

Институт (факультет)	Институт информационных технологий
Направление подготовки (специальность)	09.04.04 Программная инженерия (Разработка программно-информационных систем)
Выпускающая кафедра	Математического и программного обеспечения ЭВМ

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Название работы	Программное обеспечение планирования прокатки и нагрева на стане 5000
-----------------	---

Студента	Куцова Максима Александровича Ф.И.О.
----------	---

ДОПУСТИТЬ К ЗАЩИТЕ

Директор института (декан факультета)	Ершов Евгений Валентинович
Заведующий выпускающей кафедрой	Ершов Евгений Валентинович
Руководитель выпускной квалификационной работы	Ершов Евгений Валентинович
Нормоконтролер	Виноградова Людмила Николаевна
Выпускник	Куцов Максим Александрович

Оглавление

Введение	5
1 Характеристика проблемы планирования прокатки и нагрева слябов на стане	9
1.1 Анализ существующих моделей, методов и средств планирования прокатки и нагрева слябов	9
1.2 Описание работы прокатного стана как объекта планирования	15
1.3 Постановка задачи, разработка требований к информационному и аппаратно-программному обеспечению планирования прокатки и нагрева слябов на стане 5000	22
1.4 Выводы по разделу 1	23
2 Математическое и алгоритмическое обеспечение планирования прокатки и нагрева слябов на стане 5000.....	24
2.1 Разработка математической модели планирования прокатки и нагрева слябов.....	24
2.1.1 Имитационная модель	24
2.1.2 Математическая модель оптимизации прокатных партий	26
2.2 Адаптация и тестирование математической модели	28
2.3 Описание результатов тестирования	31
2.4 Разработка алгоритмов.....	31
2.5. Выводы по разделу 2	34
3 Проектирование и реализация информационного и программного обеспечения планирования прокатки и нагрева слябов на стане 5000	35
3.1 Концептуальное моделирование программного обеспечения прокатки и нагрева слябов на стане 5000.....	35
3.1.1 Модель черного ящика	35

3.1.2 Модель состава	37
3.1.3 Модель структуры	38
3.1.4 Модель белого ящика	38
3.2 Проектирование информационного обеспечения	39
3.3 Проектирование программного обеспечения	43
3.3.1 Выбор подхода к разработке	43
3.3.2 Выбор модели жизненного цикла	44
3.3.3 Выбор инструментальных средств	45
3.3.4 Разработка логической модели	47
3.3.4.1 Диаграмма вариантов использования	47
3.3.4.2 Контекстная диаграмма классов	52
3.3.4.3 Диаграмма последовательностей системы для вариантов использования	53
3.3.4.4 Построение диаграмм деятельности	62
3.3.4.5 Построение диаграммы пакетов	66
3.3.4.6 Детальная диаграмма классов пакета «ViewModels»	68
3.3.4.7 Детальная диаграмма классов пакета «Canva»	68
3.3.4.8 Детальная диаграмма классов пакета «Models»	69
3.3.4.9 Детальная диаграмма классов пакета «BusinessLayer»	70
3.3.4.10 Детальная диаграмма классов пакета «DataLayer»	71
3.3.5 Разработка физической модели	71
3.3.5.1. Диаграмма компонентов	71
3.3.5.2 Диаграмма размещения	72
3.3.6 Разработка интерфейсов	73
3.3.6.1 Построение графа диалога	73

3.3.6.2 Разработка форм ввода вывода информации	75
3.4 Выводы по разделу 3	77
4 Результаты экспериментальной проверки программного обеспечения планирования прокатки и нагрева слябов на стане 5000	79
4.1 Описание эксперимента и экспериментальных данных.....	79
4.2 Разработка методики настройки информационного и программного обеспечения	84
4.3 Результаты экспериментальной проверки модели	89
4.4 Перспективы применения программного обеспечения планирования прокатки и нагрева слябов на стане 5000.....	94
4.5 Выводы по разделу 4	95
Заключение	96
Список литературы	98
Приложение 1 Устав проекта	103
Приложение 2 Текст программы	115
Приложение 3 Спецификации	133
Приложение 4 Руководство пользователя	135

Введение

Актуальность темы. Оптимизация процессов производства является приоритетной задачей на любом профессиональном производстве. Обычно оптимизация производственных процессов ставит перед собой две цели: повышение эффективности производства в целом и сокращение затрат. Как правило, конечная цель достигается за счет внедрения более современных технологий и улучшения организации труда.

Оптимизация производственных процессов — это понятие, применимое к компании на любой стадии развития. Кроме того, есть ситуации, в которых просто необходимо оптимизировать производство, или предприятие окажется нерентабельным. Если в условиях дешевой рабочей силы, дешевого сырья и энергии, рентабельность достигается именно за счет доступности этих источников, то с ростом цен на эти составляющие производство становится более затратным, и, соответственно, менее рентабельным. Предприятию приходится сокращать затраты и применять более эффективные технологии производства.

Планирование загрузки листопрокатных станов является важной задачей, от решения которой зависит качество и себестоимость прокатанного металла. Прокатный план - упорядоченная последовательность слябов, составленная из слябов различных заказов.

Задача планирования относится к числу сложных, так как при создании плана загрузки, который формируется из набора заказов потребителей, состоящего из различных сортаментов прокатываемого металла и марок сталей, необходимо учесть различные требования технологии нагрева и прокатки. Задача усложняется тем, что при этом приходится обрабатывать большие массивы информации. Так, например, количество слябов, которое необходимо для формирования месячного плана, обеспечивающего выполнение заказов потребителей, может достигать до 20000 штук и более. В этих условиях выполнить все требования технологии нагрева и прокатки всех

слябов довольно сложно. Поэтому при планировании необходимо сокращать затраты, возникающие при нарушении требований. Обычно при планировании загрузки станов горячей прокатки полосы специалисты используют локальные правила, полученные формализацией требований технологических инструкций, и стараются минимизировать только количество нарушений и перевалок между компаниями.

Несмотря на то, что задача оптимального расписания заданного набора работ на имеющемся оборудовании известна давно, удовлетворительные решения получены лишь для простейших случаев, и задача сохраняет свою актуальность. Недостаточная разработанность методов решения подобных задач и отсутствие их применения для создания планов прокатных кампаний в прокатных производствах страны определили актуальность исследования и обусловили выбор темы диссертации.

На данный момент на предприятии ПАО «Северсталь» существует проблема оптимизации работы стана 5000. Необходимо оптимально планировать процессы прокатки и нагрева слэбов на стане, чтобы суммарное затраченное время на прокатку и нагрева слэбов было минимальным. Максимально эффективно это можно сделать с помощью методов и алгоритмов оптимизации в отличие от ручного планирования, которым пользуются на данный момент.

Из вышеизложенного следует, что разработка методов и алгоритмов обработки информации для программного обеспечения планирования прокатки и нагрева слэбов на стане 5000 является актуальной научно-технической задачей.

Объектом исследования являются автоматизированные системы оптимизации производственных процессов.

Предметом исследования являются методы и алгоритмы оптимизации, используемые в процессе производства.

Целью работы является повышение эффективности работы стана 5000, заключающееся в уменьшении суммарного времени прокатки запланированных прокатных партий за счет оптимизации рабочего процесса.

Для достижения поставленной цели были решены следующие задачи:

- Выполнен анализ предметной области в рамках проводимых исследовательских работ.
- Выполнено теоретическое обоснование решаемой задачи, описание методов и моделей, разработан алгоритм планирования прокатки и нагрева слэбов.
- Выполнено проектирование и реализация информационного и программного обеспечения планирования прокатки и нагрева слэбов на стане 5000.
- Реализована экспериментальная проверка программного обеспечения планирования прокатки и нагрева слэбов на стане 5000.

Методы исследования. Для решения поставленных задач в работе использовались методы системного анализа – для изучения технологических процессов и формирования требований к программному обеспечению, математической статистики – для оценки эффективности разработанного программного обеспечения, симплексный метод оптимизации – для оптимизации планирования прокатки и нагрева слэбов на стане.

Научная новизна результатов работы заключается в разработке нового алгоритма планирования прокатки и нагрева слэбов на стане 5000, отличающегося возможностью одновременного планирования процессов прокатки и нагрева слэбов и позволяющего сократить общее время прокатки запланированных партий.

Практическая значимость работы заключается в возможности применения полученного в ходе исследования программного продукта, который может с помощью применения математических моделей формировать прокатные группы с учетом всех требований, а также формировать посад слэбов в печь согласно заданным параметрам. Итоговый

результат пользователь может увидеть как в виде рисунков, так и табличном варианте.

Апробация работы. Результаты диссертационной работы обсуждены и получили положительные оценки на:

- студенческой научной конференции СНК-2022 в ЧГУ.
- Студенческой научной конференции «Регион искусственного интеллекта» (6 ноября 2022, Череповец).
- Международной научной конференции «Молодые исследователи - регионам» (18 апреля 2023).
- студенческой научной конференции СНК-2023 в ЧГУ (25 апреля 2023).

Публикации. По теме диссертации опубликована 1 статья издании из перечня ВАК.

Структура и объем работы. Диссертационная работа состоит из введения, четырех разделов, заключения, списка литературы, включающего 50 наименований, и 4 приложений. Объем диссертационной работы - 139 страниц. В тексте диссертации содержатся 42 рисунка и 52 таблицы.

1 Характеристика проблемы планирования прокатки и нагрева слэбов на стане

1.1 Анализ существующих моделей, методов и средств планирования прокатки и нагрева слэбов

Прокатка металлов — это вид обработки металлов под давлением, в ходе которого осуществляется процесс деформации металла и получение необходимой продукции (например: листы и плиты из меди, латуни, бронзы или других сплавов). В ходе такого процесса металл протягивается между сдавливающими его вращающимися валками, при этом он сжимается в вертикальном и расширяясь в поперечном направлении. Таким образом получают продукцию проката из цветных металлов и их сплавов, из стали. Форма продукции проката, ее окончательные размеры, определяются профилем отверстия между сжимающими металл валками. Сами же устройства, на которых проходит прокатка металлов называются прокатными станами [11].

В зависимости от температуры катаемого металла, прокатка делится на горячую и холодную. Прокатку металла, нагретого выше температуры рекристаллизации, называют горячей, ниже — холодной. Для горячей прокатки характерны отсутствие упрочнения и незначительные изменения механических и физико-химических свойств металла. Пластическая деформация не создаёт неравномерности микроструктуры, но в макроструктуре происходят изменения, создающие анизотропию механических свойств, при которой свойства материала вдоль проката обычно лучше его свойств в поперечном направлении. При холодной прокатке металла происходит упрочнение, изменяющее механические и физико-химические характеристики проката. Изменения затрагивают микро- и макроструктуру металла. В холодном прокате возникает анизотропия не только механических, но и физико-химических свойств. Процесс прокатки осуществляется на стане горячей прокатки.

Станы горячей и холодной прокатки подразделяют по характеру процесса прокатки и выпускаемой продукции на блюминги, слябинги, листопрокатные, проволочные, сортовые, трубопрокатные и специальные [5]. На блюмингах производят крупные квадратные заготовки – блюмы из которых на сортопрокатных станах изготавливается сортовой прокат различных сечений. На слябингах прокатывают крупные прямоугольные заготовки – слябы, из которых затем на листопрокатных станах производят более мелкие прямоугольные заготовки и листы. Проволочные станы предназначены для получения проволоки диаметром 5-10 мм. С помощью трубопрокатных станов получают бесшовные и сварные трубы. На специальных станах прокатывают железнодорожные колеса, вагонные оси и др. В свою очередь эти категории различаются видом привода, расположением рабочих клеток, количеством валков в клетях. Станы горячей прокатки в зависимости от назначения делятся на: обжимные, заготовочные, сортовые, листовые, проволочные, трубные.

Процессы прокатки классифицируют по следующим признакам:

- по температуре проведения процесса прокатку делят на
 - холодную (температура металла находится в пределах 10—30 % температуры рекристаллизации);
 - тёплую (температура металла находится в пределах 40—60 % температуры рекристаллизации);
 - горячую (температура металла выше 60—70 % температуры рекристаллизации);
- по взаимному расположению осей валков и полосы различают продольную (ось прокатываемой полосы перпендикулярна осям валков), поперечную (ось прокатываемой полосы параллельна осям валков) и поперечно-винтовую или «косую» прокатку (оси валков находятся под некоторым углом друг к другу и к оси прокатываемой полосы);
- по характеру воздействия валков на полосу и условиям деформации прокатка бывает симметричной и несимметричной. Симметричной

прокаткой называют процесс, при котором воздействие каждого из валков на прокатываемую полосу является идентичным. Если это условие нарушается, процесс следует отнести к несимметричному;

- по наличию или отсутствию внешних сил, приложенных к концам полосы, выделяют свободную и несвободную прокатку. Прокатка называется свободной, если на полосу действуют только силы, приложенные со стороны валков. Несвободная прокатка осуществляется с натяжением или подпором концов полосы [32].

Выделяют следующие группы металлопроката:

1) листовой (лист, полоса, штрипс):

- горячекатаные тонкие (толщина до 4 мм);
- горячекатаные толстые (толщина свыше 4 мм);
- холоднокатаные.

2) сортовой:

- простой (круг, квадрат, шестигранник, полоса плоского сечения);
- арматура (изделие из металла, применяемое для армирования железобетонных конструкций);
- фасонный, общего и специального назначения (угловой профиль, швеллеры, рельсы, профили машиностроения, судостроения и нефтяной промышленности).

3) трубы:

- круглые;
- бесшовные;
- сварные;
- профильные;
- с переменным размером сечения.

4) специальные виды (бандажи, колёса, шары, периодические и гнутые профили).

В большинстве современных станов процесс горячего проката включает следующие стадии: нагрев заготовок в нагревательных печах, обработку в

черновой группе клетей, прокатку в чистовой группе, смотку или порезку и горячую листоотделку. При прокатке металла, отлитого в слитки, заготовка предварительно проходит через обжимной стан – блюминг или слябинг. Назначение этого процесса – обжатие слитка в длинный брус (блюм) или пластину (сляб) [8]. Там, где работают установки непрерывной разливки стали (УНРС), получают уже готовые блюмы или слябы, которые идут в прокатные цехи, где из них делают профили – заготовки определенной толщины и формы.

Мощности горячей прокатки компании ПАО «Северсталь» представлены тремя станами горячей прокатки и одним станом прокатки холодного листа.

Листопрокатный цех №1 оборудован двумя комбинированными полунепрерывными станами. На первом из них (стан 2800) изготавливаются толстые плиты толщиной от 6,35 до 50 мм, которые служат подкатом для второго стана (стан 1700). Стан 1700 производит лист толщиной от 0,8 до 8 мм и оборудован четырьмя моталками производительностью 2 млн. т/год. Годовая производительность 400 000 т/год и 800 000 т/год соответственно [21].

В листопрокатном цехе №2 расположен стан 2000 – один из крупнейших в мире. Он предназначен для непрерывной прокатки широких листов и обладает производительностью 6 млн. т/год [43]. Он оснащен четырьмя подогревательными печами и машинами для продольной и поперечной резки. Стан 2000 производит прокатку листа толщиной от 1,2 до 16 мм в автоматическом режиме.

Цех горячей прокатки №3 оборудован станом 5000, предназначенным для прокатки толстолистовой стали. Цех располагается в г. Колпино. Стан 5000 производит широкий лист «кварто» для судостроения и ленты для изготовления труб. Производительность стана составляет до 850 000 т/год.

Аналогами представленной системы технологии являются калькулятор для составления партий слябов и модели, использующие нейросеть. Каждые из этих технологий предназначены в большей или меньшей степени представлены для решения разных задач. На примере их можно выделить основные преимущества и недостатки.

Калькулятор – это аппарат, уже укоренившийся, как в повседневной жизни, так и в профильных отраслях. С помощью него можно производить разные по сложности операции, что заметно упрощает и делает комфортной любую работу, требующую расчетов. Это изобретение является полезной находкой для нашего времени, где расчеты, цифры и числа играют далеко не последнюю роль [4].

Машинное обучение ML (ML, Machine Learning) — это процесс, в ходе которого компьютер совершенствует свои возможности посредством анализа прошлого опыта. В настоящее время разработчиками систем искусственного интеллекта принято считать, что для огромного числа приложений обучение системы путем демонстрации ей примеров желаемого поведения ввода-вывода может быть проще запрограммировать, чем предвидеть все возможные реакции на все входные данные [3].

Тем не менее процесс автоматического улучшения поведения с помощью опыта может быть дорогостоящим и длительным, особенно когда необходимо интерпретировать и обрабатывать большие объемы данных.

Существует большое разнообразие методов, которые развивались на протяжении многих лет, которые могут быть использованы для преодоления этих проблем, и выбор правильного может быть сложной задачей. Хотя первоначальная цель при обработке данных может состоять в том, чтобы найти узнаваемый паттерн, исследователи машинного обучения (ML) идут дальше и пытаются построить алгоритмы, которые могут учиться и делать прогнозы на основе имеющихся данных [40].

Чем больше данных доступно, тем лучше будет работать алгоритм. С появлением больших облачных сервисов, где пользователи имеют достаточную вычислительную мощность и неограниченное место для сбора и обработки большего количества данных, чем в прошлые десятилетия, ML сделал еще один шаг вперед [23].

Тот факт, что методы ML были разработаны для анализа экспериментальных данных с высокой пропускной способностью новыми

способами, делает их привлекательным подходом при решении более сложных задач. Тем не менее, какими бы мощными эти методы ни казались на первый взгляд, есть некоторые ограничения при их использовании. Проблема заключается в количестве данных, которые можно собрать и проанализировать. Алгоритмы ML очень сильно зависят от количества доступных данных и их характеристик. Если решаемая проблема описывается небольшим количеством исследуемых данных, будет трудно построить модель, из которой можно извлечь набор правил, которые будут использоваться для получения наилучшего возможного обобщения. Обобщение - одна из ключевых целей любой техники ML. Это означает, что модели должны быть обучены, чтобы правильно классифицировать данные, избегая при этом быть слишком специфичными для примеров, которые использовались для обучения алгоритма.

Тем не менее программное обеспечение, основанное на калькуляторе или на моделях, использующие нейросеть, не особо подходят для решения задачи оптимизации. Калькулятор не учитывает все технологические особенности процесса производства. При помощи калькулятора невозможно найти определённые партии слябов. Модель на основе нейросетей требуется в постоянном переобучении, в больших вычислительных мощностях. Основные преимущества и недостатки каждой системы представлены в табл. 1.

Таблица 1

Анализ аналогов

Название системы/модели	Преимущества	Недостатки
1	2	3
Калькулятор для составления партий слябов	1. Простота и скорость реализации 2. Тиражируемость	1. Не учитывает все особенности технологического процесса 2. Невозможно нахождение различных типов слябов

1	2	3
Модели, использующие нейросеть	1. Возможность учета множества факторов технологического процесса	1. Потребность в постоянном переобучении 2.Требуются большие вычислительные мощности
Программное обеспечение планирования прокатки и нагрева слябов на стане 5000	1. Используется комбинация алгоритмов оптимизации. 2.Получение возможных партий слябов и их редактирование при необходимости	1. Требуется развитая система АСУ ТП

Таким образом, можно сделать следующие общие выводы: анализ предметной области помог выявить, что технологии прокатки и нагрева слэбов активно развиваются, и появляются новые продукты, способные оптимизировать данные процессы, но они либо слишком дорогие, либо не учитывают все особенности технологического процесса. Поэтому разработка нового продукта является актуальной.

1.2 Описание работы прокатного стана как объекта планирования

Листопрокатный цех №3 был принят в эксплуатацию в декабре 1985 г. До 2000 г. он входил в состав Ижорских заводов и был предназначен преимущественно для проката листов из сталей специального назначения [50].

В 2000 г. цех вошел в состав компании «Северсталь», которая вложила более 10 млрд руб. в развитие данного производства, серьезно модернизировав

«стан-5000». Благодаря этому цех получил возможность производить штрипс для труб большого диаметра. В настоящий момент основным потребителем продукции ЛПЦ-3 является Ижорский трубный завод. Также прокат поставляется другим российским производителям ТБД, а также предприятиям машиностроительной, судостроительной, вагоностроительной отраслей и оборонного комплекса. Мощность ЛПЦ-3 рассчитана на выпуск 800 тыс. т проката в год в среднем сортаменте.

Основным сырьем для производства толстолистого проката ЛПЦ-3 служат слябы, выпущенные в сталеплавильном производстве Череповецкого металлургического комбината «Северстали» [37].

Слябы в ЛПЦ-3 нагревают в печах до температуры 1150-1250 °С на участке нагревательных печей. В цехе на сегодня работает несколько печей: шесть камерных печей (пять из них достались компании от Ижорских заводов), а также методическая печь, введенная в эксплуатацию в 2006 г.

Далее нагретый сляб перемещается в камеру гидросбива окалины, где с поверхности сляба удаляется окалина, образовавшаяся в процессе нагрева. Давление воды в камере достигает 253 атмостфер.

Затем следует прокат сляба. Стан «5000», установленный на «Северстали», имеет так называемую клеть «Кварто», в состав которой входят четыре валька (два опорных и два рабочих). Сам стан «5000» носит такое название из-за того, что длина бочки рабочего валька составляет 5000 мм.

Стан является автоматическим, и вся прокатка ведется в автоматическом режиме. Сортамент производимой продукции включает в себя листы толщиной от 10 до 200 мм, шириной 1500-4800 мм, длиной до 26 м [46].

Процесс прокатки включает три основные стадии:

- стадия калибровки: продольный(е) проход(ы) для обеспечения постоянной и точной толщины слябов;
- стадия поперечной прокатки: после разворота на 90° выполняются поперечные проходы для получения заданной ширины листа;
- редуционная стадия: после разворота на 90° производятся

продольные проходы до достижения заданной толщины листа.

После прокатки лист следует на установку контролируемого охлаждения либо на установку термоупрочнения листов [18]. Затем раскат подвергается горячей правке в листопрямительной машине и проходит ультразвуковой контроль. Замедленное охлаждение производится с помощью движения металла по шлипперам (холодильник) и далее режется в заданные габариты.

Марочный сортамент готовой продукции:

- низкоуглеродистые микрелегированные трубные стали;
- стали для судостроения, для котлостроения;
- стали для мостостроения;
- прочие общие конструкционные стали.

Основные марки стали, прокатываемые на стане 5000, приведены в табл. 2.

Таблица 2

Основные марки стали, прокатываемые на стане 5000 и требования, предъявляемые к ним

№п/п	Марка стали	НД на	
		Марку стали	Технические требования
1	Сталь углеродистая обыкновенного качества Ст0 – Ст5 и др.	ГОСТ 380	ГОСТ 14637
2	Сталь низколегированная для мостостроения марок 10ХСНД	ГОСТ 6713	ГОСТ 6713
3	Сталь углеродистая и низколегированная из марок 15К, 20К, 09Г2С для котлов	ГОСТ 5520	ГОСТ 5520
4	Сталь углеродистая и низколегированная для судостроения из марок А, В, D и др.	ГОСТ 5521 ГОСТ Р 52927	ГОСТ 5521 ГОСТ Р 52927
5	Сталь легированная конструкционная для судостроения АБ2	ТУ 5.961-11571-2006	ТУ 5.961-11571-2006
6	Сталь низколегированная конструкционная повышенной прочности 09Г2, 10Г2С, 16ГС и др	ГОСТ 19281	ГОСТ 19281

7	Сталь углеродистая качественная конструкционная из марок 08-25(пс, сп), 30-70	ГОСТ 1050	ГОСТ 1577
8	Сталь легированная конструкционная из марок 15Г, 20Г, 30Г и др.	ГОСТ 4543	ГОСТ 1577
9	Сталь толстолистовая из низколегированных марок стали классов прочности К52-К60 для магистральных газонефтепроводов	ТУ 14-1- 5574-2009 ТУ-14-1- 5582-2009 ТУ 14-1- 5586-2009 и др.	ТУ 14-1- 5574-2009 ТУ-14-1- 5582-2009 ТУ 14-1- 5586-2009 и др.

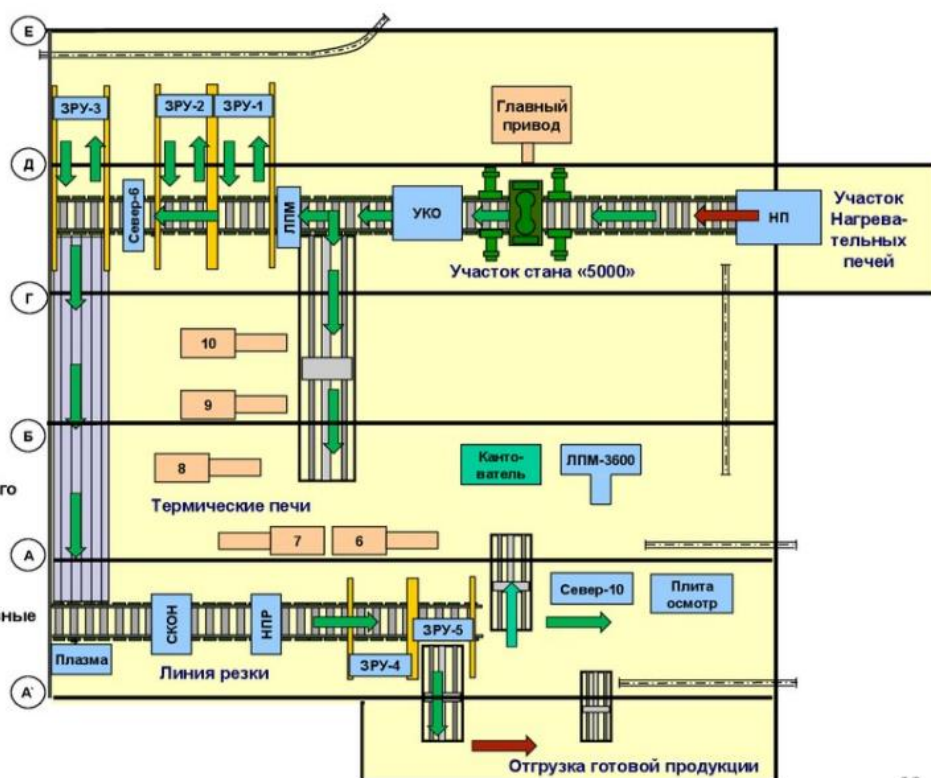
Схема производственного процесса на листопрокатном цехе №3 показана на рис. 1.

Стан-5000 предназначен для производства толстолистого проката:

• листы:
толщина от 10 до 50 мм,
ширина от 1500 до 4800 мм,
длина от 4000 до 30000 мм;

• плиты:
толщина от 50 до 450 мм,
ширина от 1500 до 4800 мм,
длина от 4000 до 30000 мм.

ЗРУ – загрузочно-разгрузочное устройство
УКО – установка контролируемого охлаждения;
Участок нагревательных печей №1-5;
ЛПМ – листопрямляющая машина
СКОН – сдвоенные кромкообрезные ножницы
НПР – ножницы поперечной резки;
Север – установка ультразвукового контроля
Термические печи №6-10



30

Рис. 1. Схема производственного процесса ЛПЦ №3

На рис. 2. представлена схема линии стана 5000, где присутствуют

следующие обозначения:

УКО – установка контролируемого охлаждения;

ЛПМ – листоправильная машина;

ГС – гидросбив;

КП – камерная печь;

МП – методическая печь.

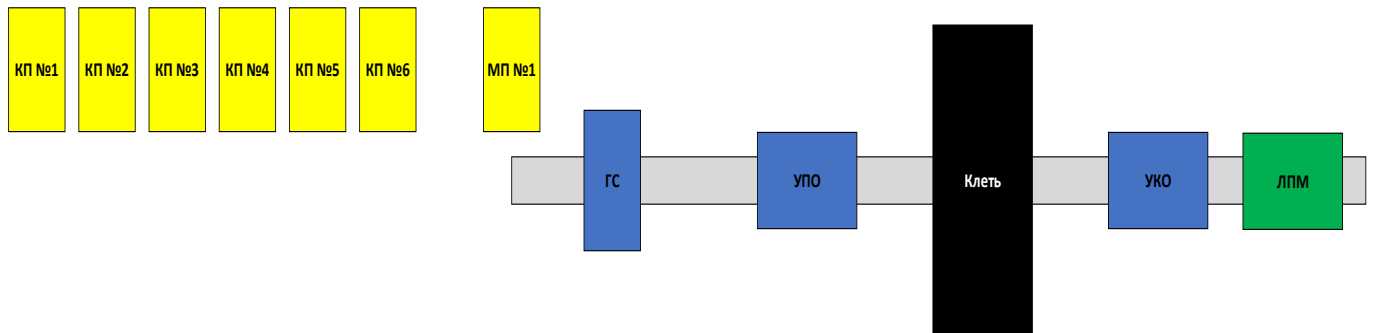


Рис. 2. Схема линии стана 5000

В целом, производство продукции на стане непрерывной прокатки характеризуется следующими показателями [30]:

- производительностью за определенный период времени;
- удельным расходом топлива и электроэнергии;
- отношением массы поступивших заготовок к массе произведенной продукции (расходный коэффициент).

Производительность стана и расходы на его техническое обслуживание связаны в первую очередь с перевалками, в то время как качество катаемого металла определяет расходный коэффициент прокатки [20]. В литературе принято разделение дефектов горячей полосы на следующие типы:

- Дефекты поверхности горячекатаной полосы и листа, образовавшиеся из дефектов литого сляба.
- Дефекты поверхности, образовавшиеся в процессе прокатки литого сляба до готовой полосы, в процессе транспортирования и порезки.
- Дефекты формы полосы и листа.

- Дефекты формы рулона.

Ниже перечислены основные дефекты, относящиеся к каждому типу, и основные причины их появления.

К дефектам поверхности, образовавшимся из дефектов литого сляба относятся раскатанное загрязнение, раскатанная трещина, раскатанный пузырь, раскатанная корочка, расслоение, гармошка, пузырь (вздутие). Все они имеют сталеплавильную природу. Появление их связано с загрязнением слэбов неметаллическими включениями, попаданием на поверхность шлака, частиц огнеупора, плохим удалением окалины, шлаковых включений, внутренних разрывов и рыхлостей в слябах [26].

К дефектам поверхности, образовавшимся в результате прохождения металла по стану, относятся [28]:

1. Вкатанные металлические частицы, возникающие из-за появления на роликах наваров и наплывов, попадания в зону проката стружки, отслоев от рваных кромок листа и др.

2. Раковина вдавливания, образующаяся при выпадении вкатанных однородных частиц во время прокатки или смотки полос.

3. Перегревы поверхности, которые образуются при превышении температуры и времени нахождения в зонах печи.

4. Рванины пережога, которые образуются при прокатке пережженного металла.

5. Сквозные разрывы, причиной которых является наличие рыхлоты и поперечная разнотолщинность.

6. Отпечатки, которые оставляет навар металла на валках, тянущих или формирующих роликах, или роликах рольганга; в т.ч. сетка отпечатков от вдавливания прокатываемого металла в трещины изношенных валков.

7. Рваная кромка, появляющаяся при прокатке стали, на которой невозможно получить необходимую пластичность.

8. Окалина, являющаяся следствием выработки валков чистовой группы и плохой работы гидросбива.

9. Прокатная плена после раскатки рванин или следов глубокой зачистки дефектов поверхности.

10. Продиры от резкого трения полосы или листа о детали прокатного и подъемного транспортного оборудования.

11. Риска от царапания поверхности металла наварями и другими выступами на прокатной арматуре и прочие царапины, образующиеся в результате механических повреждений.

12. Забоины, образующиеся в следствие ударов при транспортировке.

Неплоскостность, серповидность, загнутый угол, волнистость, порез, продольная и поперечная разнотолщинность представляют собой дефекты формы полосы, возникающие вследствие нарушений технологии нагрева и прокатки, приводящих к недопустимым перепадам температуры по длине подката и между ними, и к большим обжатиям по ширине. Недостаточный разогрев валков, прокатка широких полос после узких, перегрев середины бочки валков, неудовлетворительная профилировка валков так же оказывают значительное влияние на появление данных типов дефектов [17].

К дефектам формы рулона относятся рулон со складками, эллипс, распушенный рулон, телескопический рулон. Дефекты смотки появляются из-за наличия коробоватости, серповидности или несоответствия скоростей рольганга [36].

Таким образом появление большинства дефектов горячих полос связано как с нарушением технологии нагрева металла в печах, так и с нарушением технологии проката. Наиболее часто встречающимися видами дефектов являются плены, окалина и дефекты, вызываемые неметаллическими включениями. Из-за нарушений, вызывающих появление брака, ежегодно в отсортировку попадает несколько тысяч тонн металла. Повышение эффективности планирования могло бы снизить эту цифру за счет проката кампаний, в большей степени удовлетворяющих требованиям технологии и обеспечивающих более высокое качество готовой продукции. В зависимости от назначения катаемого металла, к качеству проката предъявляются

различные требования. Для сталей ответственного назначения важны получаемые механические свойства, для автолиста – качество поверхности, для трубных сталей – отсутствие трещин и т.д. Те или иные сталеплавильные дефекты в металле могут приводить к появлению брака в готовой полосе в зависимости от предъявляемого к металлу качества. Причины образования части дефектов берут начало в сталеплавильном производстве, оставшиеся возникают из-за тех или иных нарушений технологии прокатки.

1.3 Постановка задачи, разработка требований к информационному и аппаратно-программному обеспечению планирования прокатки и нагрева слэбов на стане 5000

Постановка задачи – это процесс формулировки назначения программного обеспечения и основных требований к нему. Описываются функциональные требования, которые должно выполнять программное обеспечение, и эксплуатационные требования, определяющие характеристики его функционирования. Этап постановки задачи заканчивается разработкой технического задания с принятием основных проектных решений.

Необходимо создать программное обеспечение, реализующее новый алгоритм планирования процессов прокатки и нагрева слэбов на стане 5000, чтобы суммарное затраченное время на прокатку и нагрев слэбов было минимальным. Максимально эффективно это можно сделать с помощью методов и алгоритмов оптимизации в отличие от ручного планирования.

Требования к информационному обеспечению:

Требования к организации выходным данным. В результате работы системы будут получены следующие виды выходных данных:

- список спланированных прокатных партий в формате .xlsx;
- скриншоты сформированных групп и заполненностей печей в формате .png;
- запись в базу данных (БД).

Требования к аппаратно-программному обеспечению:

Надежное (устойчивое) функционирование программы должно быть обеспечено выполнением совокупности организационно-технических мероприятий, перечень которых приведен ниже:

- организацией бесперебойного питания технических средств;
- использованием лицензионного программного обеспечения;
- регулярным выполнением рекомендаций Министерства труда и социального развития РФ, изложенных в Постановлении от 23 июля 1998 г. «Об утверждении межотраслевых типовых норм времени на работы по сервисному обслуживанию ПЭВМ и оргтехники и сопровождению программных средств»;
- регулярным выполнением требований ГОСТ 51188-98. Защита информации. Испытания программных средств на наличие компьютерных вирусов.

Условия эксплуатации. Условия эксплуатации данного программного обеспечения должны соответствовать условиям эксплуатации компьютера оператора. В случае затруднений у оператора, должно быть подготовлено руководство оператора, описывающее действия, которые он может выполнить и описания интерфейса с требованиями к вводимым данным [1].

1.4 Выводы по разделу 1

В данном разделе представлены основные выводы по анализу предметной области программного обеспечения планирования прокатки и нагрева слэбов на стане 5000. Были проанализированы существующие системы в данной области и выделены главные преимущества разрабатываемого программного обеспечения. Также в текущем разделе была разобрана предметная область, сформированы задачи и требования к программному обеспечению.

2 Математическое и алгоритмическое обеспечение планирования прокатки и нагрева слэбов на стане 5000

2.1 Разработка математической модели планирования прокатки и нагрева слэбов

Представленные математические модели позволяют сгруппировать прокатные группы таким образом, чтобы в итоге получилось наименьшее общее время прокатки всех сформированных партий.

В данном случае прокатные группы будут формироваться в зависимости от характеристик слэбов (время черновой, чистовой прокатки и время подстуживания).

Представленная модель формирует прокатные партии и получает наименьшее время, которое будет затрачено на выполнение всего перечня заказа.

2.1.1 Имитационная модель

Задача имитационной модели рассчитать длительность прокатки группы для заданного множества слэбов.

На вход данной имитационной модели подаются следующие данные:

- Общее количество слэбов – n ;
- Длительность черновой прокатки слэбов $(t_{\text{черн}_1}, t_{\text{черн}_2} \dots t_{\text{черн}_n})$;
- Длительность чистовой прокатки слэбов $(t_{\text{чист}_1}, t_{\text{чист}_2} \dots t_{\text{чист}_n})$;
- Длительность подстуживания слэбов $(t_{\text{подстуж}_1}, t_{\text{подстуж}_2} \dots t_{\text{подстуж}_n})$;
- Пауза между чистовыми прокатками в группе $(p_{\text{чист}})$;
- Пауза между черновыми прокатками в группе $(p_{\text{черн}})$;
- Пауза между последней черновой в группе и первой чистовой $(p_{\text{меж}})$;
- Ширина слэба $W (w_1, w_2 \dots w_n)$;
- Длина слэба $L (l_1, l_2 \dots l_n)$.

Перед тем как передать данные по характеристикам слябов в имитационную модель происходит сортировка слябов по увеличению черновой прокатки, в результате чего подтверждается линейная логика и для решения задачи потребуются методы линейного программирования.

Начальное время прокатки группы τ_0 примем за 00:00. За $i = 1 \dots c$ примем количество штук в группе. Расчет времени конца черновой прокатки первой штуки $\tau_{к.ч.}$ рассчитывается как

$$\tau_{к.ч.} = \tau_0 + t_{черн1}, \quad (1)$$

также это время равно началу подстуживания $\tau_{н.п.}$ первой штуки. Чтобы рассчитать время конца подстуживания первой $\tau_{к.п.}$, это тоже самое что и начало чистовой прокатки $\tau_{н.чист.}$, нужно ко времени конца черновой прокатки прибавить длительность подстуживания:

$$\tau_{к.п.1} = \tau_{н.чист.1} = \tau_{к.ч.1} + t_{подстуж1}. \quad (2)$$

И чтобы рассчитать время конца чистовой прокатки необходимо ко времени начала чистовой прокатки прибавить ее длительность

$$\tau_{к.чист.1} = \tau_{н.чист.1} + t_{чист1}, \quad (3)$$

в результате подсчитано общее время прокатки первой штуки в группе. Время прокатки каждой следующей штуки в группе рассчитываем следующим образом: к началу следующей чистовой прокатки прибавляем паузу между чистовыми прокатками $p_{чист}$, т.е.

$$\tau_{н.чист. i} = \tau_{к.чист. i-1} + p_{чист}, \text{ где } i \neq 1, \quad (4)$$

$$\tau_{к.чист. i} = \tau_{н.чист. i} + t_{чист i}, \text{ где } i \neq 1. \quad (5)$$

Время начала подстуживания (конца черновой прокатки) будет равно:

$$\tau_{\text{н.подстуж. } i} = \tau_{\text{к.ч. } i} = \tau_{\text{н.чист. } i} - t_{\text{подстуж } i}, \quad \text{где } i \neq 1. \quad (6)$$

Время начала черновой прокатки каждой следующей штуки равно:

$$\tau_{\text{н.ч. } i} = \tau_{\text{к.ч. } i} - t_{\text{черн } i}, \quad \text{где } i \neq 1. \quad (7)$$

После того как мы рассчитали время прокатки штуки необходимо проверить, чтобы выполнялись следующие условия:

$$\tau_{\text{н.ч. } i} - \tau_{\text{к.ч. } i-1} \geq p_{\text{черн}}, \quad \text{где } i \neq 1$$

$$\tau_{\text{н.чист. } i} > \tau_{\text{к.чист. } i-1}, \quad \text{где } i \neq 1$$

$$\tau_{\text{к.ч. } i} < \tau_{\text{н.чист. } i-1}, \quad \text{где } i \neq 1$$

$$\tau_{\text{к.ч. } i} - \tau_{\text{н.чист. } 1} \geq p_{\text{меж}}, \quad \text{где } i \neq 1.$$

Если хотя бы одно из условий не выполняется, то прокатная группа считается неисполнимой, и мы ее не рассматриваем.

В результате работы данной имитационной модели получается набор всех возможных сформированных групп $X = (x_1, x_2 \dots x_m)$, где m – количество групп. Каждая группа имеет свою суммарную длительность $d (d_1, d_2 \dots d_k)$.

После чего набор сформированных прокатных групп передается в модель оптимизации прокатных партий.

2.1.2 Математическая модель оптимизации прокатных партий

Задача данной модели оптимизации заключается в том, чтобы определить для заданного перечня слэбов, которые необходимо прокатать на следующие сутки, как их сформировать в прокатные группы, с минимальной суммарной длительностью прокатки.

На первом шаге сгенерируем все возможные варианты группировок прокатных групп для данных типов слэбов. Сколько всего типов слэбов обозначим за n .

Так как у нас есть технологическое ограничение, которое заключается в том, что много разных типов слэбов в одну прокатную группу не добавляем, в нашем случае количество типов слэбов в одной прокатной группе z равно 3.

Отсюда можно сделать вывод, что после вызова имитационной модели для генерации всех возможных вариантов остается m групп слэбов, где $m = n * (n - 1) * (n - 2)$, при $z = 3$.

Имитационная модель для каждой группы слэбов проверяет можно ли прокатать так слэбы или нет, рассчитывает сколько они будут кататься.

Дальше обозначим через $X (x_1, x_2 \dots x_m)$ сколько экземпляров таких прокатных групп необходимо взять для выполнения заказа и для решения данной оптимизационной задачи.

Для решения этой задачи необходимо следующее:

1) Целевая функция

$$F(X) = \sum_{i=1}^n X_i * d_i \rightarrow \min, \quad (8)$$

где d_i – длительность прокатки группы.

2) Задача оптимизации решается в условиях ограничений, то есть нужно сделать все слэбы, которые есть в перечне заказов, необходимо сделать за «сегодня». Для этого введем следующие обозначения:

Количество слэбов каждого типа в перечне заказов обозначим множеством $K (k_1, k_2 \dots k_n)$.

Множеством $A (a_{11}, a_{12} \dots a_{1m}; a_{21}, a_{22} \dots a_{2m}; \dots a_{n1}, a_{n2} \dots a_{nm})$ обозначим количество слэбов в m -ой группе слэбов k -го типа.

Множество ограничений можно представить в виде системы неравенств:

$$\begin{cases} \sum_{i=1}^n a_{ij} * x_i \leq k_j \\ \sum_{i=1}^n a_{ij} * x_i \geq k_j \end{cases} \quad (9)$$

Так как данные ограничения являются линейными, поэтому данная задача решалась методом линейного программирования – симплекс-методом.

Симплекс-метод – алгоритм решения оптимизационной задачи линейного программирования путем перебора вершин выпуклого многогранника в многомерном пространстве. Суть метода: построение базисных решений, на которых монотонно убывает линейный функционал, до ситуации, когда выполняются необходимые условия локальной оптимальности [35].

2.2 Адаптация и тестирование математической модели

Для адаптации новой технологии к имеющемуся оборудованию в числе прочего потребовалось разработать математические модели для планирования прокатки и нагрева слябов на стане 5000, которые были описаны ранее. Для создания математических моделей за основу был взят симплекс-метод, т.к. в области линейного программирования он достаточно хорошо зарекомендовал себя.

Симплекс-метод — алгоритм решения оптимизационной задачи линейного программирования путём перебора вершин выпуклого многогранника в многомерном пространстве [48].

Сущность метода: построение базисных решений, на которых монотонно убывает линейный функционал, до ситуации, когда выполняются необходимые условия локальной оптимальности. Задача линейного программирования в стандартной форме заключается в том, чтобы найти экстремум функции при заданных ограничениях [10].

Необходимые условия для применения симплекс-метода:

А) Задача должна иметь каноническую форму.

Б) У задачи должен быть явно выделенный базис.

Для удобства вычислений и наглядности обычно пользуются симплекс-таблицами:

Базис:	X_1	X_2	...	X_n	
X_n	a_{11}	a_{12}	...	a_{1n}	b_1
X_{n-1}	a_{21}	a_{22}	...	a_{2n}	b_2
...	...	...	...	...	...
X_{n-2}	a_{m1}	a_{m2}	...	a_{mn}	b_m
Z	c_1	c_2	...	c_n	0

Рис. 3. Пример симплекс-таблицы

- В первой строке указывают «наименование» всех переменных.
- В первом столбце указывают номера базисных переменных, а в последней ячейке – букву Z (это строка функционала).
- В «середине таблицы» указывают коэффициенты матрицы ограничений — a_{ij} .
- Последний столбец – вектор правых частей соответствующих уравнений системы ограничений.
- Крайняя правая ячейка – значение целевой функции. На первой итерации ее полагают равной 0.

Алгоритм симплекс-метода:

1. Выбираем переменную, которую будем вводить в базис. Это делается в соответствии с указанным ранее принципом: мы должны выбрать переменную, возрастание которой приведет к росту функционала. Выбор происходит по следующему правилу:

Если задача на минимум – выбираем максимальный положительный элемент в последней строке.

Если задача на максимум – выбираем минимальный отрицательный.

Такой выбор, действительно, соответствует упомянутому выше принципу: если задача на минимум, то, чем большее число вычитаем – тем быстрее

убывает функционал; для максимума наоборот – чем большее число добавляем, тем быстрее функционал растет.

Определение: Столбец симплекс-таблицы, отвечающий выбранному коэффициенту, называется ведущим столбцом.

2. Выбираем переменную, которую будем вводить в базис. Для этого нужно определить, какая из базисных переменных быстрее всего обратится в нуль при росте новой базисной переменной. Алгебраически это делается так:

- Вектор правых частей почленно делится на ведущий столбец.
- Среди полученных значений выбирают минимальное положительное (отрицательные и нулевые ответы не рассматривают).

Определение: Такая строка называется ведущей строкой и отвечает переменной, которую нужно вывести из базиса.

3. Ищем элемент, стоящий на пересечении ведущих строки и столбца.

Определение: Такой элемент называется ведущим элементом.

4. Вместо исключаемой переменной в первом столбце (с названиями базисных переменных) записываем название переменной, которую мы вводим в базис.

5. Далее начинается процесс вычисления нового базисного решения. Он происходит с помощью метода Жордана-Гаусса.

- Новая Ведущая строка = Старая ведущая строка / Ведущий элемент.
- Новая строка = Новая строка – Коэффициент строки в ведущем столбце * Новая Ведущая строка.

6. После этого проверяем условие оптимальности. Если полученное решение неоптимально – повторяем весь процесс снова.

Условие оптимальности полученного решения:

- Если задача на максимум – в строке функционала нет отрицательных коэффициентов (т.е. при любом изменении переменных значение итогового функционала расти не будет).

Если задача на минимум – в строке функционала нет положительных коэффициентов (т.е. при любом изменении переменных значение итогового функционала уменьшаться не будет) [24].

2.3 Описание результатов тестирования

Тестирование разработанных моделей было проведено во время промышленной эксплуатации, то есть непосредственно на производстве. Испытание разработанного программного решения проводилось в формате А/В-тестирования с разделением плана заказов на следующие сутки на две группы: с применением полученных результатов решения и списком подготовленным специалистом. Применение разработанного решения в рамках промышленной эксплуатации принесло уменьшение суммарного времени прокатки плана заказов на 2,5-5,0 %, поэтому внедрение решения является целесообразным.

Некоторые результаты тестирования представлены в табл. 3.

Таблица 3

Результаты тестирования

Количество штук	Время с моделями (сек)	Время без моделей (сек)
200	79 850	83 842
265	98 575	103 500
300	112 345	116 000

2.4 Разработка алгоритмов

После разработки математических моделей, необходимо разработать алгоритм, который объединит в себе решения для планирования прокатки и нагрева слэбов одновременно.

Разработанный алгоритм для планирования прокатки и нагрева слэбов на стане 5000, представлен на рис. 4.

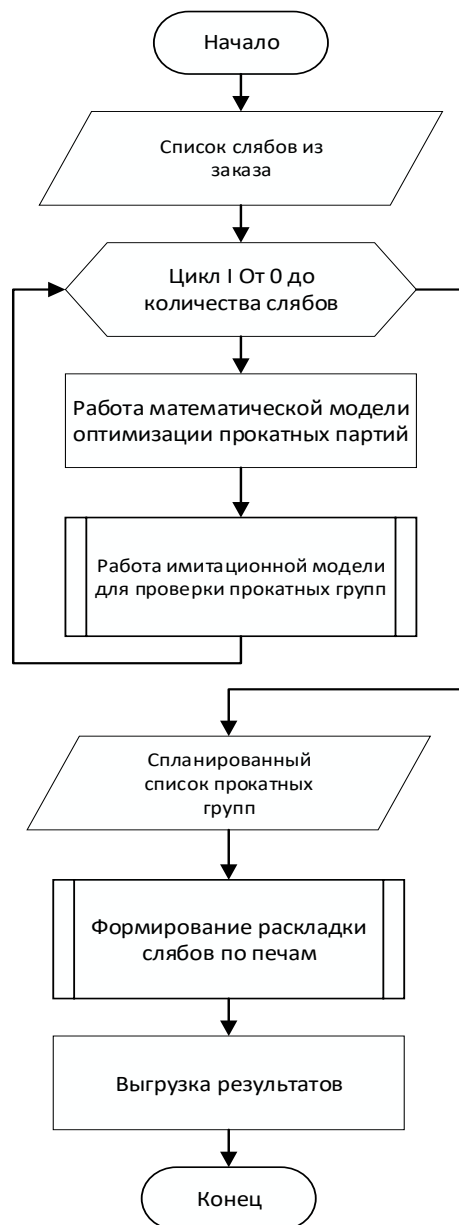


Рис. 4. Блок-схема алгоритма планирования прокатки и нагрева слябов на стане 5000

Далее будет представлено описание работы созданного алгоритма:

1. Получаем данные о технологических параметрах слябов из базы данных, файла и вводим вручную данных о печах, получаем план заказов.
2. Рассчитываем оптимальное суммарное время прокатки в соответствии с планом заказов:
 - а) формируем прокатные партии с помощью математической модели оптимизации прокатных партий;

- б) с помощью имитационной модели проверяем возможность прокатки групп по отдельности;
 - в) после проверки групп продолжает свою работу математическая модель оптимизации прокатных партий;
 - г) получаем оптимизированный график прокатных партий.
4. Полученный график передается на формирование раскладки слэбов по печам.
5. Вывод пользователю сформированных прокатных групп и посадки слэбов в печи, и сохранение результатов.

Разработанный алгоритм позволит быстрее производить необходимый план заказов, за счет наиболее выгодного формирования прокатки и нагрева слэбов.

На рис 5. представлен подробнее алгоритм для формирования прокатных групп.

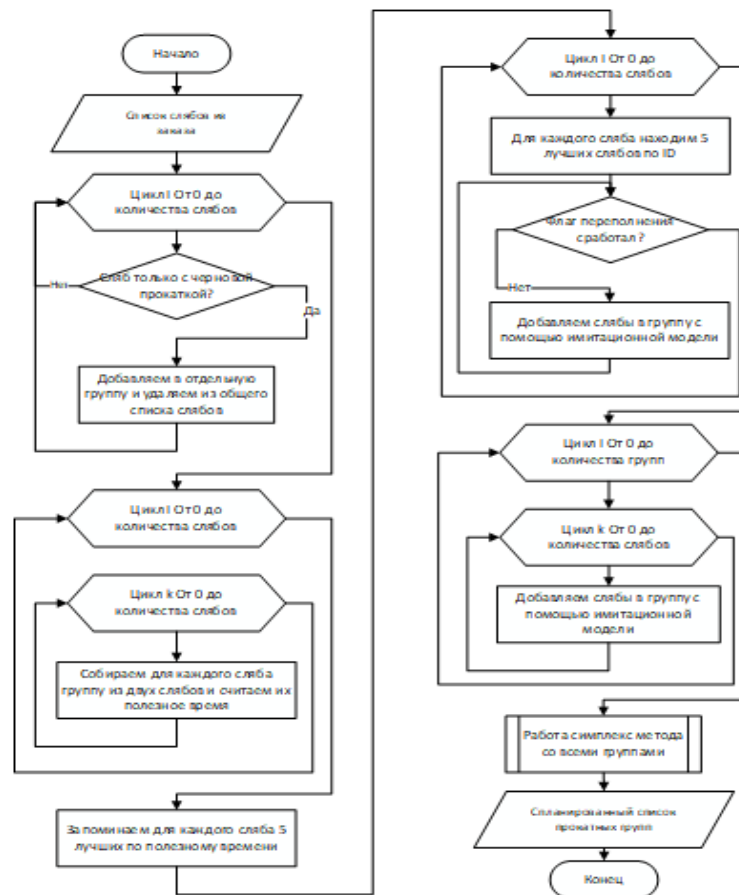


Рис. 5. Блок-схема алгоритма формирования прокатных групп

Алгоритм формирования посада слябов по печам представлен на рис. 6.

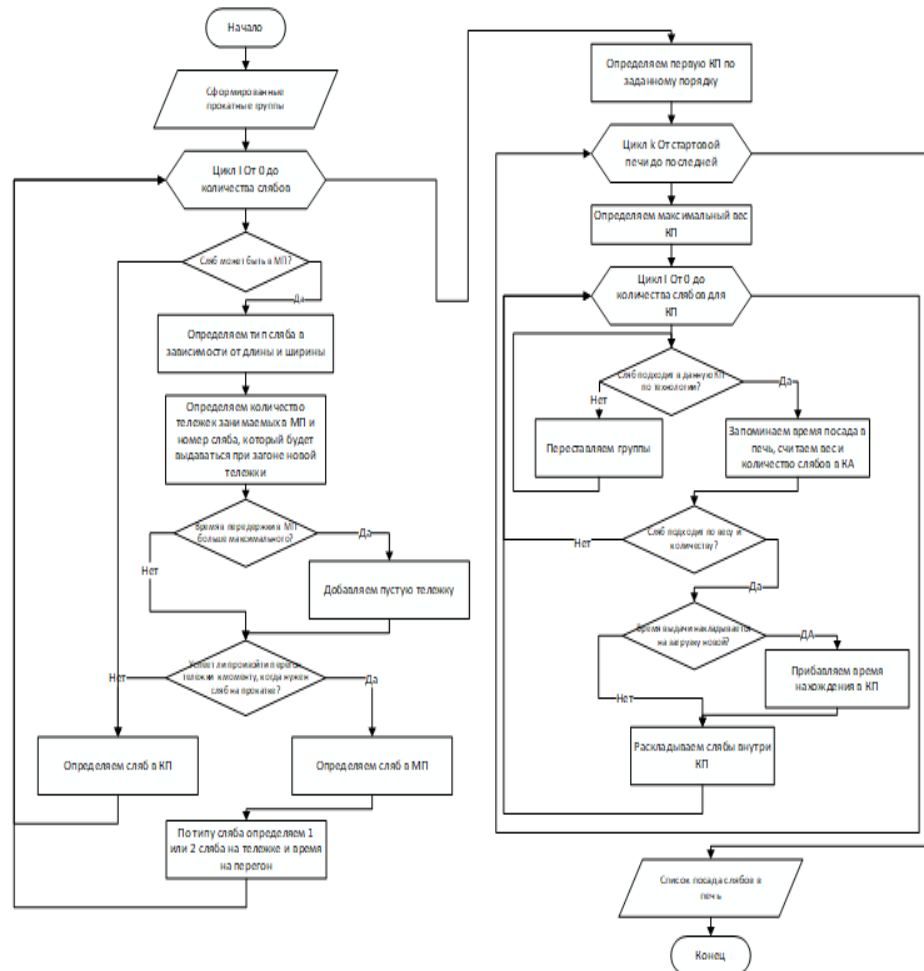


Рис. 6. Блок-схема алгоритма формирования посада по печам

2.5. Выводы по разделу 2

Для предметной области «планирование прокатки и нагрева слябов на стане 5000» были построены математические модели оптимизации формирований прокатных партий, проведены адаптация и тестирование математических моделей. Разработан алгоритм, позволяющий объединить процессы прокатки и нагрева слябов одновременно.

3 Проектирование и реализация информационного и программного обеспечения планирования прокатки и нагрева слябов на стане 5000

3.1 Концептуальное моделирование программного обеспечения прокатки и нагрева слябов на стане 5000

Концептуальное моделирование является начальным этапом в проектировании, на данном этапе принимаются решения, которые позволяют определить вид создаваемого программного обеспечения. На данном этапе проводятся исследования и согласования параметров созданных технических решений с возможной их организацией. В результате анализа алгоритмов, необходимых для планирования прокатки и нагрева слябов на стане 5000, необходимо определить границы предметной области, в которых будет осуществлено проектирование системы [49].

3.1.1 Модель черного ящика

Концептуальная (содержательная) модель – позволит получить рекомендацию по планированию прокатки и нагрева слябов на стане 5000.

Т.к. имеется понимание того какие входные и выходные данные, а также алгоритмы требуются для работы системы. Используя эти знания можно построить модель «Черного ящика» (рис. 7), в котором следует собрать все данные без углубленности в процессы работы системы.

Основные элементы системы:

1) Входы:

- Перечень слябов.
- Технологически параметры слябов.
- Расстояние в печи.
- Очередность посадки.

2) Выходы:

- График прокатных групп.
- График посадки слябов в печь.

3) Управление:

- Модель формирования групп слабов.
- Модель формирования посада слабов в печь.

4) Механизм исполнения:

- Оператор.

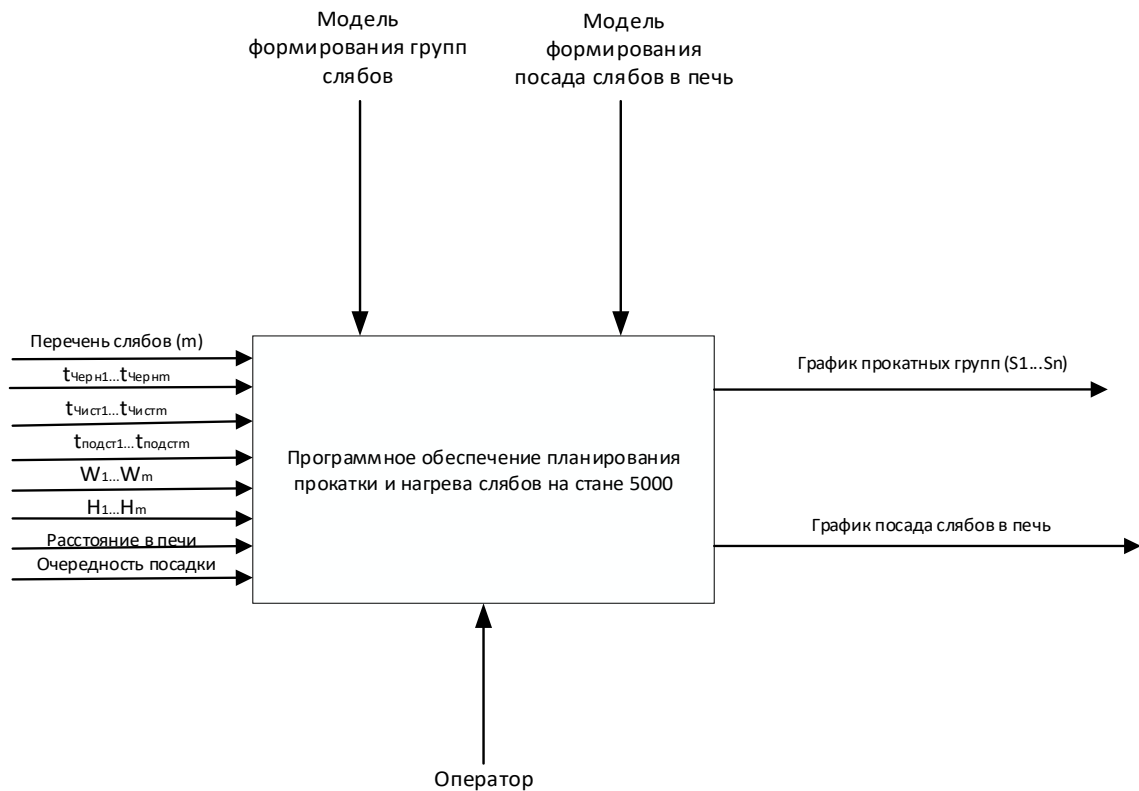


Рис. 7. «Черный ящик»

Для реализации поставленной задачи применялись методы математического моделирования, методы планирования прокатки и нагрева слабов и симплексный метод поиска оптимального значения, системный анализ – для описания модели планирования и прокатки слабов на стане 5000, математической статистики – для оценки эффективности разработанного программного обеспечения.

3.1.2 Модель состава

В начале работы системы первой срабатывает «Подсистема формирования прокатных групп» (рис. 8), которая на вход получает несколько параметров:

- Время черновой прокатки слэбов ($t_{\text{черн}_1}, t_{\text{черн}_2} \dots t_{\text{черн}_n}$).
- Время чистовой прокатки слэбов ($t_{\text{чист}_1}, t_{\text{чист}_2} \dots t_{\text{чист}_n}$).
- Время подстуживания слэбов ($t_{\text{подстуж}_1}, t_{\text{подстуж}_2} \dots t_{\text{подстуж}_n}$).
- Ширина слэба ($W_1, W_2 \dots W_n$).
- Длина слэба ($L_1, L_2 \dots L_n$).
- Высота слэба ($H_1, H_2 \dots H_n$).
- Общее количество слэбов (n).

По итогам работы данной подсистемы получается оптимизированный график прокатных групп, который передается в следующую подсистему, а также передается на выход.



Рис. 8. «Подсистема формирования прокатных групп»

Выходные данные из подсистемы формирования прокатных групп передаются в подсистему формирования посадки слэбов в печь (рис. 9), которая также на вход получает еще 2 параметра: расстояние в печи и очередность посадки.

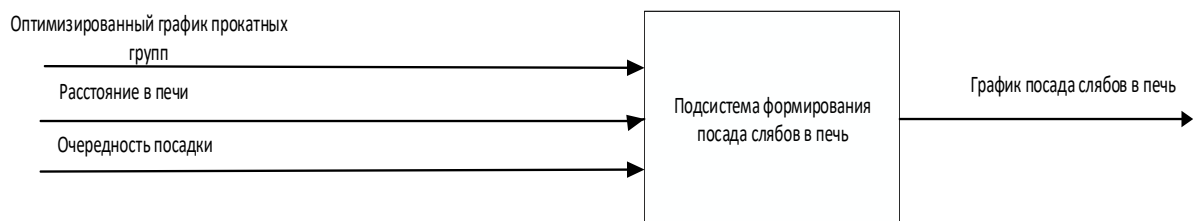


Рис. 9. Подсистема формирования посадки слэбов в печь

Выходными данными этой подсистемы является график посада слэбов в печь, который передается на выход всей системы.

3.1.3 Модель структуры

Модель структуры представляет собой объединение подсистем, описанных в модели состава, при этом обязательным условием объединения является согласованность входов и выходов подсистем (рис. 10). Оптимизированный график прокатных групп и график посада слэбов в печь, получаемые в процессе работы системы, являются основными, поскольку для его формирования необходима пошаговая работа всех, без исключения подсистем. На начальном этапе производится формирование прокатных групп, следующим этапом происходит создание графика посада слэбов в печь [39].

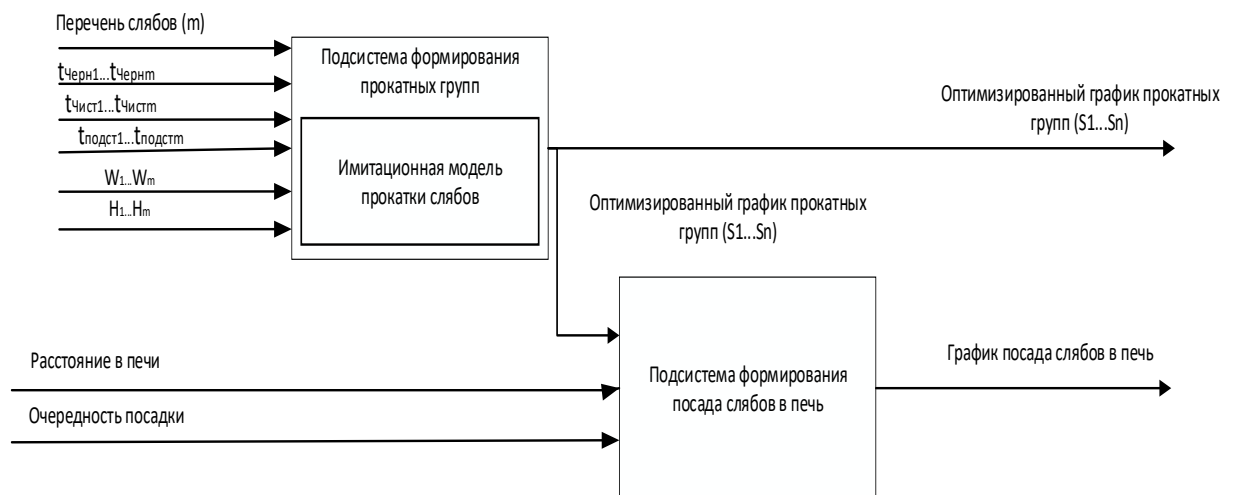


Рис. 10. Модель структуры

3.1.4 Модель белого ящика

Модель белого ящика представляет собой совмещение модели структуры и модели «чёрного ящика». При совмещении необходимо учитывать, что 39 входы и выходы модели структуры и модели «чёрного ящика» должны быть полностью согласованы (рис. 11).

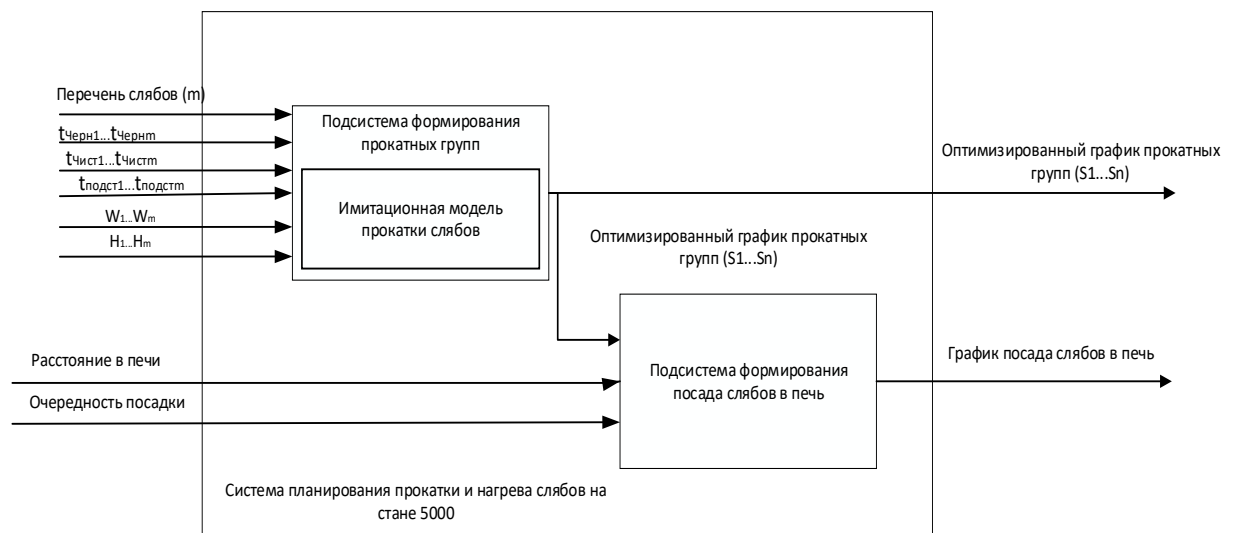


Рис. 11. Модель белого ящика

В системе планирования прокатки и нагрева слэбов на стане 5000 исходя из входных данных формируются оптимизированные прокатные партии и график посадки слэбов в печь, после чего все эти данные выводятся пользователю.

3.2 Проектирование информационного обеспечения

Проектирование информационного обеспечения заключается в разработке логической модели, которая представляет собой совокупность логических взаимосвязей между сущностями проектируемой системы. Физическая модель является совокупностью физических свойств проектируемой базы данных, таких как типы данных, размеры полей и т.д. На одну логическую модель может существовать несколько физических, поскольку свойства базы данных зависят от выбранной СУБД [47].

В проектируемом программном обеспечении, данные передаются в существующую базу данных системы.

Для правильной передачи данных в существующую БД необходимо сформировать свою структуру зависимостей и полей. Для проектирования базы данных воспользуемся реляционной моделью данных. Достоинствами данной модели являются: удобное отображение данных пользователю,

удобство работы с базой. Для начала необходимо определиться какая информация будет храниться в базе данных:

- номер сляба;
- номер группы;
- номер сляба в группе;
- время посадки сляба;
- время выдачи.

Ненормализованная таблица будет иметь следующий вид (табл. 4).

Таблица 4

Ненормализованная таблица «План заказов»

ID	Номер сляба	Марка стали	номер группы	номер сляба в группе	время посадка сляба	время выдачи	Печь
1	123456	K60	1	1	11.10.2022 15:00:00	11.10.2022 23:00:00	КП №2

Для избавления от избыточности необходимо произвести нормализацию, кроме того, она позволит лучше структурировать хранимую информацию и повысит удобство работы с ней.

Отношение находится в первой нормальной форме (1НФ) тогда и только тогда, когда все значения атрибутов в ней атомарные и ни одно из её ключевых полей не пусто.

Отношение находится во второй нормальной форме (2НФ) тогда и только тогда, когда оно находится в первой нормальной форме, и каждый не ключевой атрибут полностью зависим от первичного ключа.

Для этого разделим таблицу «План заказов» на таблицу «Марка стали» (табл.5) с полями:

- Марка стали;
- ID.

Кроме того, разделим таблицу «План заказов» на таблицу «Печи» (табл. 6) с полями:

- ID;
- Наименование печи.

Таблица 5

«Марка стали» 2НФ

ID	Марка стали
1	K60
2	C355

Таблица 6

«Печи» 2НФ

ID	Печь
1	КП №1
2	КП №2
3	МП

Таблица 7

«План заказов» 2НФ

ID	Номер сляба	Марка стали	номер группы	номер сляба в группе	время посада сляба	время выдачи	Печь
1	123456	1	1	1	11.10.2022 15:00:00	11.10.2022 23:00:00	2

Отношение находится в третьей нормальной форме (3НФ) тогда и только тогда, когда оно находится во второй нормальной форме и не содержит транзитивных зависимостей. Поскольку таблицы «Печи», «Марка» не имеют таких зависимостей, то мы имеем третью нормальную форму (табл. 7).

Третья нормальная форма является достаточной в нашем случае, так как дальнейшая нормализация приведет к повторениям данных и усложнить общую структуру базы данных.

Логическая модель. Логическая модель описывает понятия предметной области, их взаимосвязи и является представлением того, как будет построена будущая база данных. Логическая модель строится на основе общих понятий без ориентации на определённую СУБД [31].

Логическая модель представляется в виде диаграмм сущность-связь (ERD). Удобство представления логической модели в таком виде заключается в следующем:

- наглядность представления базы данных;
- облегчение проектирования больших баз данных;
- поддержка большинством систем проектирования баз данных.

В базе данных, имеющей несколько таблиц, всегда существуют определённые связи между ними, и эти связи бывают разных видов:

- связь один-к-одному;
- связь один-ко-многим;
- связь многие-к-одному;
- связь многие-ко-многим.

ER-диаграмма и схема данных базы данных, сгенерированной из неё, будут иметь следующий вид (рис. 12):

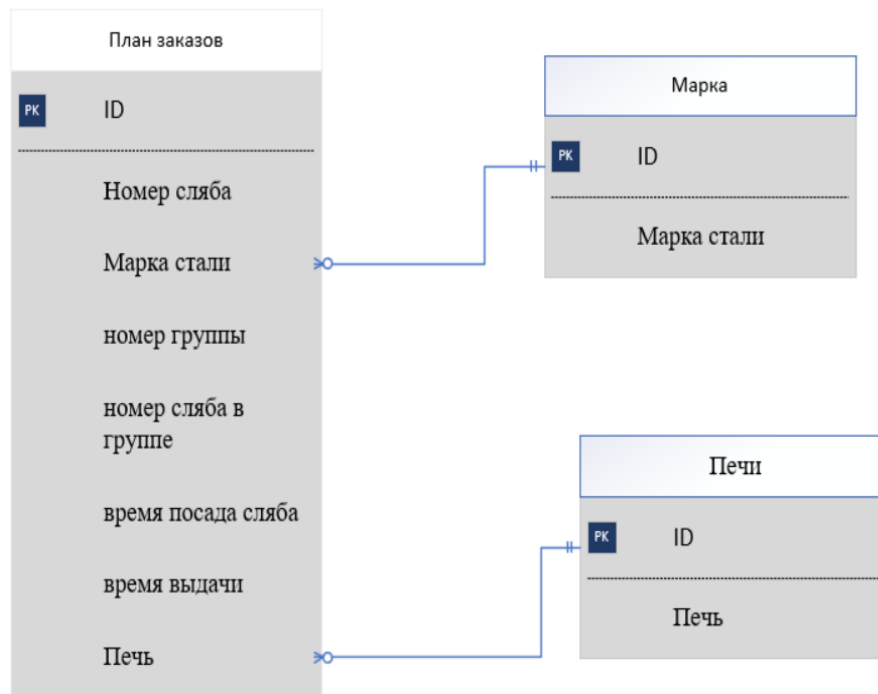


Рис. 12. ER – диаграмма базы данных

Таким образом, сформирована своя структура зависимостей и полей в существующей БД.

3.3 Проектирование программного обеспечения

3.3.1 Выбор подхода к разработке

В качестве технологии проектирования к разработке был выбран объектно-ориентированный подход. Объектный подход в проектировании программного обеспечения заключается в объектной декомпозиции, то есть представлении структуры программы в виде совокупности объектов, в процессе взаимодействия, которых с помощью передачи сообщений выполняются требуемые функции. Так же был выбран объектный подход к программированию, поскольку предметная область, разрабатываемого программного обеспечения позволяет выделить основные объекты и указать их взаимодействие [45].

3.3.2 Выбор модели жизненного цикла

Исходя из требований заказчика к реализации программного обеспечения, в частности необходимо предоставление ранних прототипов программы для ознакомления и возможное изменение или дополнение некоторых деталей, была выбрана инкрементная модель жизненного цикла (рис. 13), которая обладает следующими ключевыми преимуществами [33]:

- уменьшение затрат, которые выявляются при изменениях требований заказчиком;
- клиенты могут увидеть прототип системы и оставить комментарии для дальнейшего улучшения информационной системы;
- поскольку переход из настоящего в будущее не происходит моментально, заказчик может привыкать к новой технологии постепенно;
- снижается риск неудачи при разработке программного обеспечения.

При условии, что каждая последующая версия системы представляет собой модернизированный прототип, добавляя к предыдущей определённые функциональные возможности, то данная модель ЖЦ в полной мере подходит под разрабатываемое программное обеспечение.

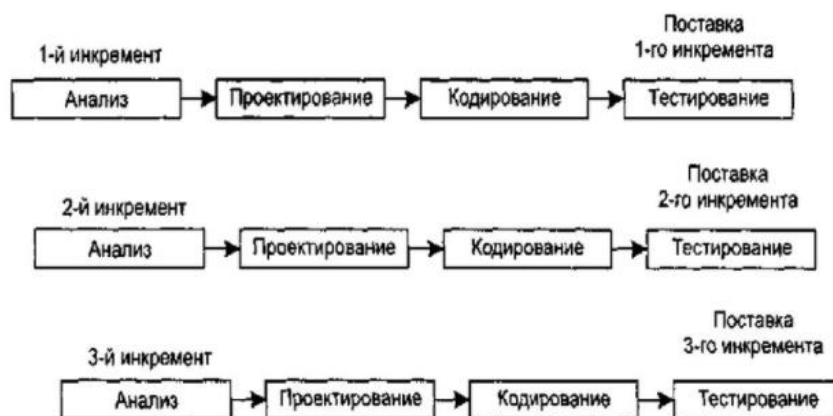


Рис. 13. Инкрементная модель

3.3.3 Выбор инструментальных средств

Для разработки данного программного обеспечения были выбраны следующие технологии и инструментальные средства:

- использование языка высокого уровня C# и платформы .Net;
- пользовательский интерфейс реализован на системе WPF;
- среда разработки Visual Studio 2019.

C# – объектно-ориентированный язык программирования. Разработан в 1998—2001 годах группой инженеров компании Microsoft под руководством Андерса Хейлсберга и Скотта Вильтаумота как язык разработки приложений для платформы Microsoft .NET Framework. Впоследствии был стандартизирован как ECMA-334 и ISO/IEC 23270 [42]. C# относится к семье языков с C-подобным синтаксисом, из них его синтаксис наиболее близок к C++ и Java. Язык имеет статическую типизацию, поддерживает полиморфизм, перегрузку операторов делегаты, атрибуты, события, переменные, свойства, обобщённые типы и методы, итераторы, анонимные функции с поддержкой замыканий, LINQ, исключения, комментарии в формате XML [6].

Windows Presentation Foundation (WPF) — это платформа пользовательского интерфейса для создания клиентских приложений для настольных систем. Платформа разработки WPF поддерживает широкий набор компонентов для разработки приложений, включая модель приложения, ресурсы, элементы управления, графику, макет, привязки данных, документы и безопасность. Эта платформа является частью платформы .NET, и если ранее вы создавали приложения в .NET с помощью ASP.NET или Windows Forms, то должны быть знакомы с принципами программирования. WPF использует расширяемый язык разметки для приложений (XAML), чтобы предоставить декларативную модель для программирования приложений [22].

XAML — это декларативный язык разметки. С точки зрения модели программирования .NET Core язык XAML упрощает создание пользовательского интерфейса для приложения .NET Core. Можно создать видимые элементы пользовательского интерфейса в декларативной XAML-

разметке, а затем отделить определение пользовательского интерфейса от логики времени выполнения, используя файлы кода программной части, присоединенные к разметке с помощью определений разделяемых классов. Язык XAML напрямую представляет создание экземпляров объектов в конкретном наборе резервных типов, определенных в сборках [7]. В этом заключается его отличие от большинства других языков разметки, которые, как правило, представляют собой интерпретируемые языки без прямой связи с системой резервных типов. Язык XAML обеспечивает рабочий процесс, позволяющий нескольким участникам разрабатывать пользовательский интерфейс и логику приложения, используя потенциально различные средства.

Вся разработка будет проводиться в среде разработки Visual Studio. Microsoft Visual Studio — линейка продуктов компании Microsoft, включающих интегрированную среду разработки программного обеспечения и ряд других инструментальных средств. Данные продукты позволяют разрабатывать как консольные приложения, так и игры, и приложения с графическим интерфейсом, в том числе с поддержкой технологии Windows Forms, а также веб-сайты, веб-приложения, веб-службы как в родном, так и в управляемом кодах для всех платформ, поддерживаемых Windows, .NET Framework, Xbox [19].

Разработка программной документации проводилась в соответствии с Единой системой программной документации (ЕСПД).

Разработка программного обеспечения в соответствии с общепринятыми стандартами позволяет облегчить процесс сопровождения программного решения как сами создателем ПО, так и сторонним лицам, сопровождающим разработанный программный продукт.

3.3.4 Разработка логической модели

3.3.4.1 Диаграмма вариантов использования

Диаграмма вариантов использования (сценариев выполнения, прецедентов) является исходным концептуальным представлением системы в процессе ее проектирования и разработки. На диаграммах вариантов использования (ВИ) изображаются актеры и варианты использования, между которыми существуют отношения. На диаграмме можно показывать и другие элементы UML (например, классы могут показывать, какие сущности порождаются или используются в конкретных ВИ) [25].

Суть данной диаграммы состоит в следующем: проектируемая система представляется в виде множества актеров, взаимодействующих с системой с помощью вариантов использования. Актером (действующим лицом, пользователем, актером) называется любой объект, субъект или система, взаимодействующая с моделируемой системой извне. В свою очередь вариант использования – это спецификация сервисов (функций), которые система предоставляет действующему лицу. Другими словами, каждый вариант использования определяет некоторый набор действий, совершаемых системой при взаимодействии с действующим лицом. При этом в модели никак не отражается то, каким образом будет реализован этот набор действий.

Диаграмма вариантов использования ПО для планирования прокатки и нагрева слэбов на стане 5000 представлена на рис.14.

На данной диаграмме показано одно действующее лицо: мастер нагревательных печей, который может загружать необходимые данные и вводить необходимые технологические параметры, после чего может запустить данное расчет планирования прокатки и нагрева слэбов на стане 5000, а затем ознакомиться с результатами работы как в виде рисунков, так и в табличном формате.



Рис. 14. Диаграмма вариантов использования программного обеспечения планирования прокатки и нагрева слябов на стане 5000

Вместе с диаграммой вариантов использования есть таблицы, которые описывают вариант использования. В такой таблице имеется описание ВИ в краткой форме. Для основных вариантов использования применяется подробная форма описания, которая включает в себя типичный ход событий. Таблицы с описанием вариантов использования представлены далее (табл. 8 -17).

Краткое описание варианта использования «Ввод технологических параметров» представлено в табл. 8.

Таблица 8

Краткое описание варианта «Ввод технологических параметров»

Название	Ввод технологических параметров
Назначение	Обеспечить возможность пользователя ввести технологические параметры в системе для разрешения последующего её использования
Действующие лица	Пользователь
Краткое описание	Обеспечивает механизм ввода пользователя в систему

Таблица 9

Типичный ход событий варианта «Ввод технологических параметров»

Действие пользователя	Отклик системы
1) Ввод технологических параметров	2) Проверка корректности введенных данных
3) Нажатие кнопки «Старт»	4) Уведомление пользователя об успешном вводе данных

Альтернативные варианты:

1. Если пользователь ввел неправильные данные, выведется сообщения об ошибке.

Краткое описание варианта использования «Выбор перечня слябов для прокатки» представлено в табл. 10.

Таблица 10

Краткое описание варианта «Выбор перечня слябов для прокатки»

Название	Выбор перечня слябов для прокатки
Назначение	Обеспечить возможность выбора перечня слябов для прокатки
Действующие лица	Пользователь
Краткое описание	Обеспечивает выбор перечня слябов для прокатки

Таблица 11

Типичный ход событий варианта «Выбор перечня слябов для прокатки»

Действие пользователя	Отклик системы
1) Удаление ненужных слябов	2) Удаляются слябы
3) Нажатие кнопки «Сохранить»	4) Уведомление пользователя об успешной загрузке плана слябов

Альтернативные варианты:

1. Если выбран неправильный формат файла, выдаст соответствующее сообщение об ошибке.

2. Если есть внутри файла некорректные данные и их не удастся считать, выведется сообщение об ошибке.

Краткое описание варианта использования «Просмотр графика посада» представлено в табл. 12.

Таблица 12

Краткое описание варианта «Просмотр графика посада»

Название	Просмотр графика посада
Назначение	Обеспечить возможность просмотра графиков посада в табличном или графическом виде
Действующие лица	Пользователь
Краткое описание	Обеспечивает возможность просмотра графиков посада

Таблица 13

Типичный ход событий варианта «Просмотр графика посада»

Действие пользователя	Отклик системы
1) Нажатие кнопки «Таблица»	2) Открывается список слябов со временем посада
3) Нажатие кнопки «Группы»	4) Возможность просмотреть как выглядят сформированные группы

Альтернативные варианты:

1. Пользователь закрыл программу, поэтому данные не сохранились.

Краткое описание варианта использования «Расчет графика посада слябов в печь» представлено в табл. 14.

Таблица 14

Краткое описание варианта «Расчет графика посада слябов в печь»

Название	Расчет графика посада слябов в печь
Назначение	Рассчитывает время посада слябов в печь
Действующие лица	Пользователь
Краткое описание	Позволяет рассчитать время посада слябов в печь

Таблица 15

Детальное описание варианта «Расчет графика посадки слябов в печь»

Действие пользователя	Отклик системы
1) Нажатие кнопки «Старт»	2) Формируется время посадки слябов

Альтернативные варианты:

1. Если пользователь не добавил файл с данными, то кнопка старт будет не активна и не будет возможности использовать данный вариант использования.

Краткое описание варианта использования «Расчет оптимальных прокатных групп» представлено в табл. 16.

Таблица 16

Краткое описание варианта «Расчет оптимальных прокатных групп»

Название	Расчет оптимальных прокатных групп
Назначение	Формирует оптимальных группы с помощью алгоритма
Действующие лица	Пользователь
Краткое описание	Позволяет сформировать оптимальные группы с помощью алгоритма

Таблица 17

Детальное описание варианта «Расчет оптимальных прокатных групп»

Действие пользователя	Отклик системы
1) Нажатие кнопки «Старт»	2) Формируются прокатные группы

Альтернативные варианты:

1. Если пользователь не добавил файл с данными, то кнопка старт будет не активна и не будет возможности использовать данный вариант использования.

3.3.4.2 Контекстная диаграмма классов

Диаграммы классов – центральное звено объектно-ориентированных методов разработки программного обеспечения, поэтому все существующие методы используют диаграммы классов в одной из известных нотаций. Однако в основном диаграммы классов в этих методах применяют на этапе проектирования, для того чтобы показать особенности построения конкретных классов [27].

Класс при этом традиционно понимают, как совокупность общих признаков заданной группы объектов предметной области. В соответствии с этим определением на диаграмме классов каждому классу соответствует группа объектов, общие признаки которых и фиксирует класс.

Контекстная диаграмма классов подтверждает возможность разделения предметной области на взаимодействующие классы, таким образом, выбор объектно-ориентированного подхода является оправданным.

Для демонстрации общей структуры иерархий классов и их взаимодействия служит контекстная диаграмма классов, которая представлена на рис. 15.

Класс-понятие «Система планирования прокатки и нагрева слэбов на стане 5000» является разрабатываемой системой. Класс-понятие «Группы прокатки» связывается с основным, потому что тот его использует. Из данного класса вытекают и следующие понятия «Правило прокатки» и «Технология прокатки», которые отвечают за технологические процессы данного производства. Основным класс-понятием «Стан 5000» является ключевым, потому что все процессы происходят именно здесь. Класс-понятие «Сляб» является неотъемлемой частью прокатки и используется на всем этапе производства. Класс-понятие «Печь» использует «Сляб», чтобы нагревать его в соответствии с нормами. Класс-понятие «Лист» является конечным продуктом производства. Класс-понятие «УКО» это установка контрольного охлаждения, которая необходима в процессе производства.

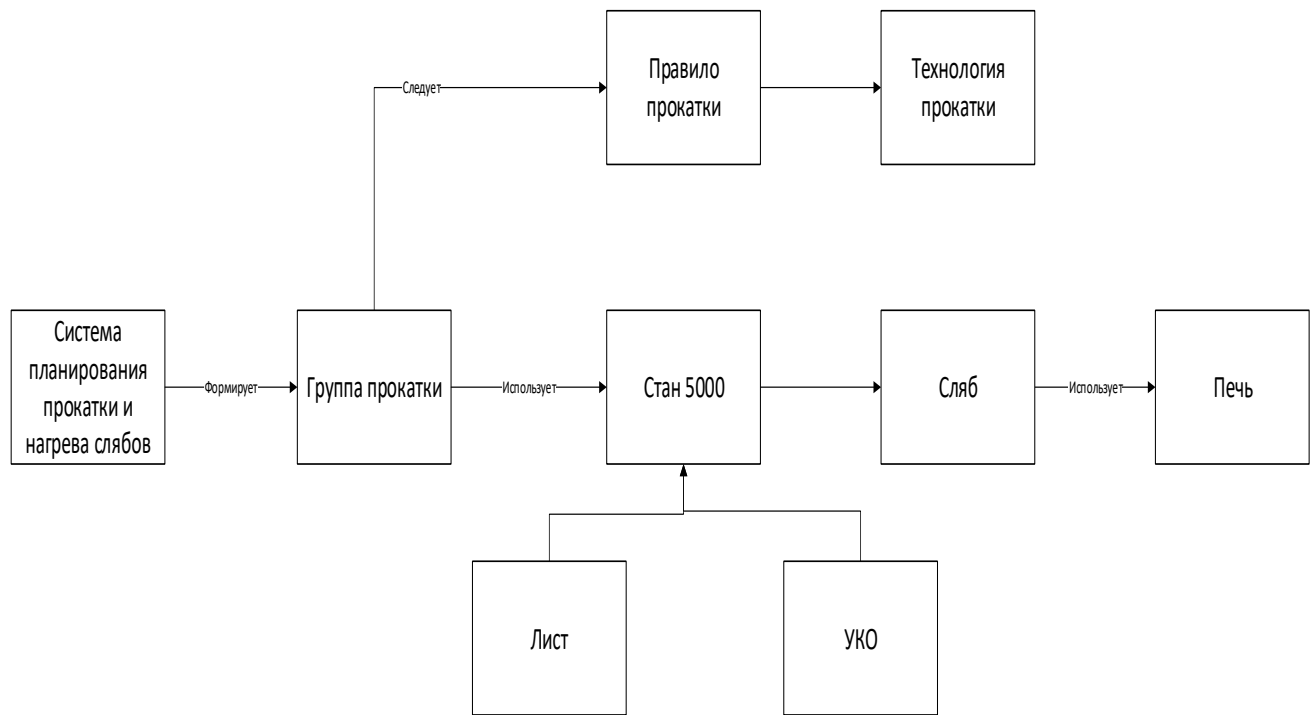


Рис. 15. Контекстная диаграмма классов

3.3.4.3 Диаграмма последовательностей системы для вариантов использования

Диаграмма последовательности — диаграмма, на которой для некоторого набора объектов на единой временной оси показан жизненный цикл какого-либо определённого объекта (создание-деятельность-уничтожение некой сущности) и взаимодействие актёров (действующих лиц) ИС в рамках какого-либо определённого прецедента (отправка запросов и получение ответов) [44].

Диаграммы последовательностей действий для соответствующих вариантов использования изображены на рис. 16-20. Описания операций и событий, изображённых на диаграммах, представлены в табл. 18-32.

Диаграмма последовательностей действий для варианта использования «Ввод технологических параметров» отображена на рис. 16.



Рис. 16. Диаграмма последовательностей действий для варианта использования «Ввод технологических параметров»

Описание операций данного варианта использования представлено в табл. 18 - 20.

Таблица 18

Описание операции «Ввести порядок печей»

Раздел	Описание
Имя	Ввести порядок печей
Обязанности	Запоминает порядок печей в котором необходимо раскладывать слябы
Тип	Системная
Ссылки	Вариант использования «Ввод технологических параметров»
Примечание	-
Исключения	-
Вывод	Текстбокс
Предусловие	-
Постусловие	-

Таблица 19

Описание операции «Ввести дату загрузки»

Раздел	Описание
Имя	Ввести дату загрузки
Обязанности	Запоминает дату от которой необходимо рассчитывать загрузку
Тип	Системная
Ссылки	Вариант использования «Ввод технологических параметров»
Примечание	-
Исключения	-
Вывод	Текстбокс
Предусловие	-
Постусловие	-

Таблица 20

Описание операции «Ввод минимального расстояния в печи»

Раздел	Описание
Имя	Ввод минимального расстояния в печи
Обязанности	Запоминает минимальное расстояние необходимое для раскладки слэбов
Тип	Системная
Ссылки	Вариант использования «Ввод технологических параметров»
Примечание	-
Исключения	-
Вывод	Текстбокс
Предусловие	-
Постусловие	-

Диаграмма последовательностей действий для варианта использования «Выбор перечня слэбов для прокатки» отображена на рис. 17.

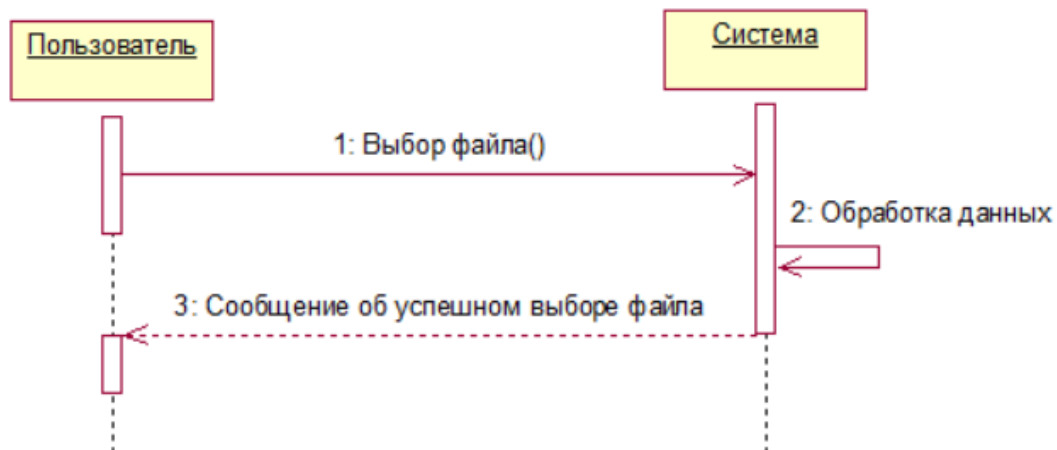


Рис. 17. Диаграмма последовательностей действий для варианта использования «Выбор перечня слябов для прокатки»

Таблица 21

Описание операции «Выбор файла»

Раздел	Описание
Имя	Выбор файла
Обязанности	Позволяет выбрать нужный файл в определенном формате
Тип	Системная
Ссылки	Вариант использования «Выбор перечня слябов для прокатки»
Примечание	-
Исключения	Файл формата .xls или .xlsx
Вывод	Окно с выбором файла
Предусловие	Система должна быть запущена
Постусловие	Система выбирает файл

Таблица 22

Описание операции «Обработка данных»

Раздел	Описание
Имя	Обработка данных
Обязанности	Обрабатывает данные, полученные из файла
Тип	Внутренняя

Ссылки	Вариант использования «Выбор перечня слябов для прокатки»
Примечание	-
Исключения	-
Вывод	-
Предусловие	Должен быть загружен файл
Постусловие	Информация об успешной обработке

Таблица 23

Описание операции «Сообщение об успешном выборе файла»

Раздел	Описание
Имя	Сообщение об успешном выборе файла
Обязанности	Оповещает пользователя об успешной обработке
Тип	Системная
Ссылки	Вариант использования «Выбор перечня слябов для прокатки»
Примечание	-
Исключения	-
Вывод	Диалоговое окно с сообщением
Предусловие	Система должна быть запущена
Постусловие	-

Диаграмма последовательностей действий для варианта использования «Просмотр графика посада» отображена на рис. 18.

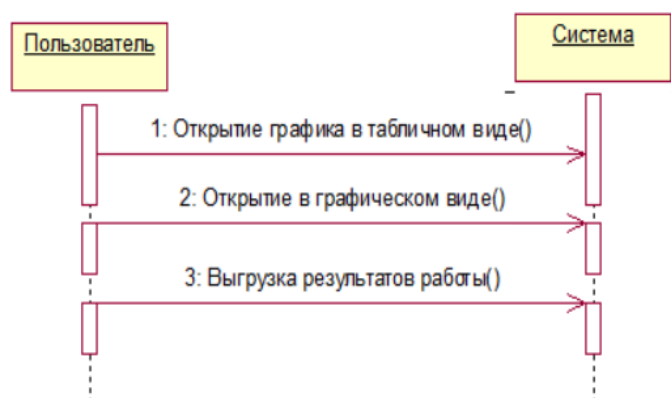


Рис. 18. Диаграмма последовательностей действий для варианта использования «Просмотр графика посада»

Таблица 24

Описание операции «Открытие графика в табличном виде»

Раздел	Описание
Имя	Открытие графика в табличном виде
Обязанности	Открывает вкладку таблица, где отображается план загрузки в виде таблицы
Тип	Системная
Ссылки	Вариант использования «Просмотр графика посадки»
Примечание	-
Исключения	-
Вывод	Таблица
Предусловие	Должен отработать алгоритм
Постусловие	-

Таблица 25

Описание операции «Открытие графика в табличном виде»

Раздел	Описание
Имя	Открытие графика в графическом виде
Обязанности	Открывает вкладку группы, где отображается план загрузки в виде рисунка
Тип	Системная
Ссылки	Вариант использования «Просмотр графика посадки»
Примечание	-
Исключения	-
Вывод	Окно с изображением
Предусловие	Должен отработать алгоритм
Постусловие	-

Описание операции «Выгрузка результатов работы»

Раздел	Описание
Имя	Выгрузка результатов работы
Обязанности	Позволяет сохранять результат работы в .xlsx файл
Тип	Системная
Ссылки	Вариант использования «Просмотр графика посадки»
Примечание	-
Исключения	-
Вывод	Диалоговое окно
Предусловие	Должен отработать алгоритм
Постусловие	Сохраняет файл в указанное место

Диаграмма последовательностей действий для варианта использования «Расчет оптимальных прокатных групп» отображена на рис. 19.

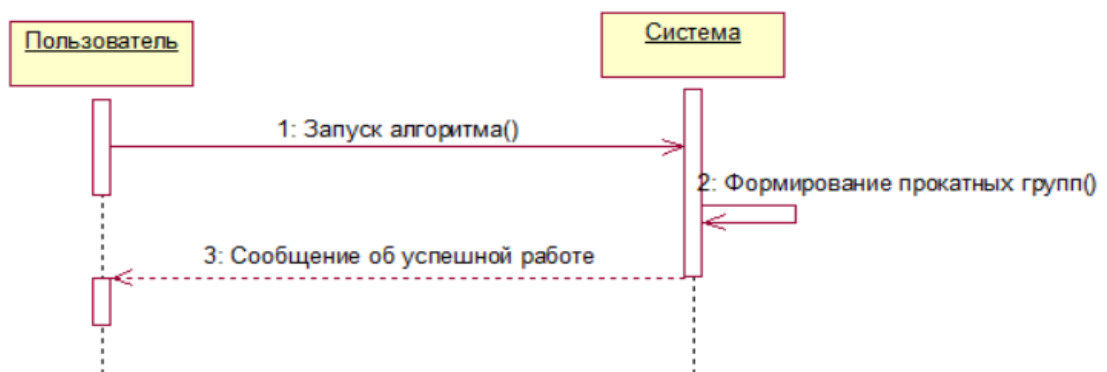


Рис. 19. Диаграмма последовательностей действий для варианта использования «Расчет оптимальных прокатных групп»

Описание операции «Запуск алгоритма»

Раздел	Описание
Имя	Запуск алгоритма
Обязанности	Запускает алгоритм формирования оптимальных прокатных групп
Тип	Системная
Ссылки	Вариант использования «Расчет оптимальных прокатных групп»

Продолжение табл. 27

Примечание	-
Исключения	-
Вывод	Диалоговое окно
Предусловие	Должен быть загружен перечень слябов
Постусловие	Сформированные прокатные группы

Таблица 28

Описание операции «Формирование прокатных групп»

Раздел	Описание
Имя	Формирование прокатных групп
Обязанности	Формирует прокатные группы с помощью симплекс-метода
Тип	Внутренняя
Ссылки	Вариант использования «Расчет оптимальных прокатных групп»
Примечание	-
Исключения	-
Вывод	-
Предусловие	Должен быть загружен перечень слябов
Постусловие	Сформированные прокатные группы

Таблица 29

Описание операции «Сообщение об успешной работе»

Раздел	Описание
Имя	Сообщение об успешной работе
Обязанности	Информирует пользователя об успешной работе алгоритма
Тип	Системная
Ссылки	Вариант использования «Расчет оптимальных прокатных групп»
Примечание	-
Исключения	-
Вывод	Диалоговое окно
Предусловие	Должен быть загружен перечень слябов
Постусловие	Сформированные прокатные группы

Диаграмма последовательностей действий для варианта использования «Расчет графика посадки слябов в печь» отображена на рис. 20.

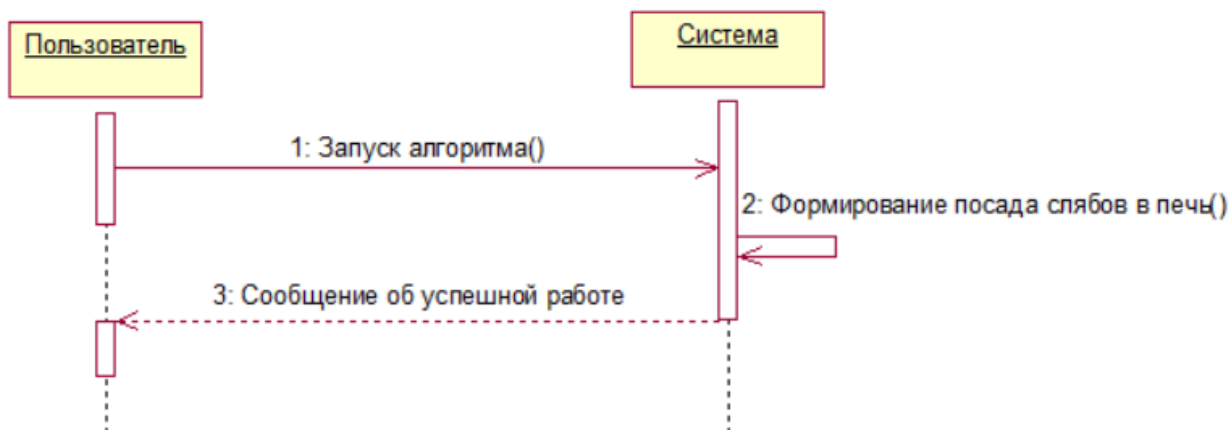


Рис. 20. Диаграмма последовательностей действий для варианта использования «Расчет графика посадки слябов в печь»

Таблица 30

Описание операции «Запуск алгоритма»

Раздел	Описание
Имя	Запуск алгоритма
Обязанности	Запускает алгоритм формирования посадки слябов в печь
Тип	Системная
Ссылки	Вариант использования «Расчет графика посадки слябов в печь»
Примечание	-
Исключения	-
Вывод	Диалоговое окно
Предусловие	Должен быть загружен перечень слябов и сформированы группы
Постусловие	Сформированный график посадки

Таблица 31

Описание операции «Формирование посадки слябов в печь»

Раздел	Описание
Имя	Формирование посадки слябов в печь
Обязанности	Формирование посадки слябов в печь
Тип	Внутренняя

Продолжение табл. 31

Ссылки	Вариант использования «Расчет графика посада слябов в печь»
Примечание	-
Исключения	-
Вывод	-
Предусловие	Должен быть загружен перечень слябов и сформированы группы
Постусловие	Сформированный график посада

Таблица 32

Описание операции «Сообщение об успешной работе»

Раздел	Описание
Имя	Сообщение об успешной работе
Обязанности	Информирует пользователя об успешной работе алгоритма
Тип	Системная
Ссылки	Вариант использования «Расчет графика посада слябов в печь»
Примечание	-
Исключения	-
Вывод	Диалоговое окно
Предусловие	Должен быть загружен перечень слябов и сформированы группы
Постусловие	Сформированный график посада

3.3.4.4 Построение диаграмм деятельности

Каждому варианту использования соответствует последовательность задач. Диаграммы деятельности являются обобщенным представлением алгоритма, реализующего анализируемый вариант использования. Диаграммы деятельности хорошо подходят для описания сложных потоков с циклами и ветвлениями, поскольку они позволяют наглядно представить последовательность действий. Варианты использования, без ветвлений и циклов, нет смысла описывать при помощи диаграмм деятельности [9].

На рис. 21 изображена диаграмма деятельности для варианта использования «Выбор перечня слябов».

На рис. 22 изображена диаграмма деятельности для варианта использования «Просмотр графика посада».

На рис. 23 изображена диаграмма деятельности для варианта использования «Расчет оптимальных прокатных групп».

На рис. 24 изображена диаграмма деятельности для варианта использования «Расчет графика посада слябов в печь».

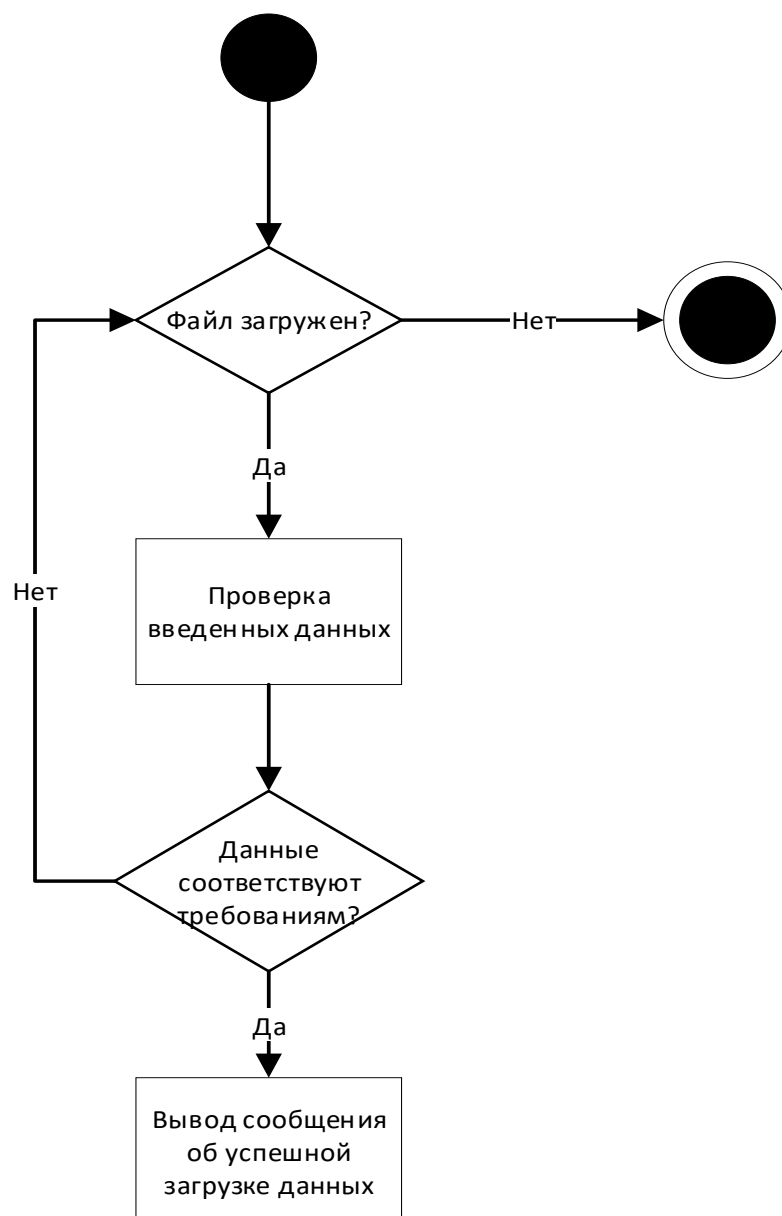


Рис. 21. Диаграмма деятельности для варианта использования «Выбор перечня слябов»

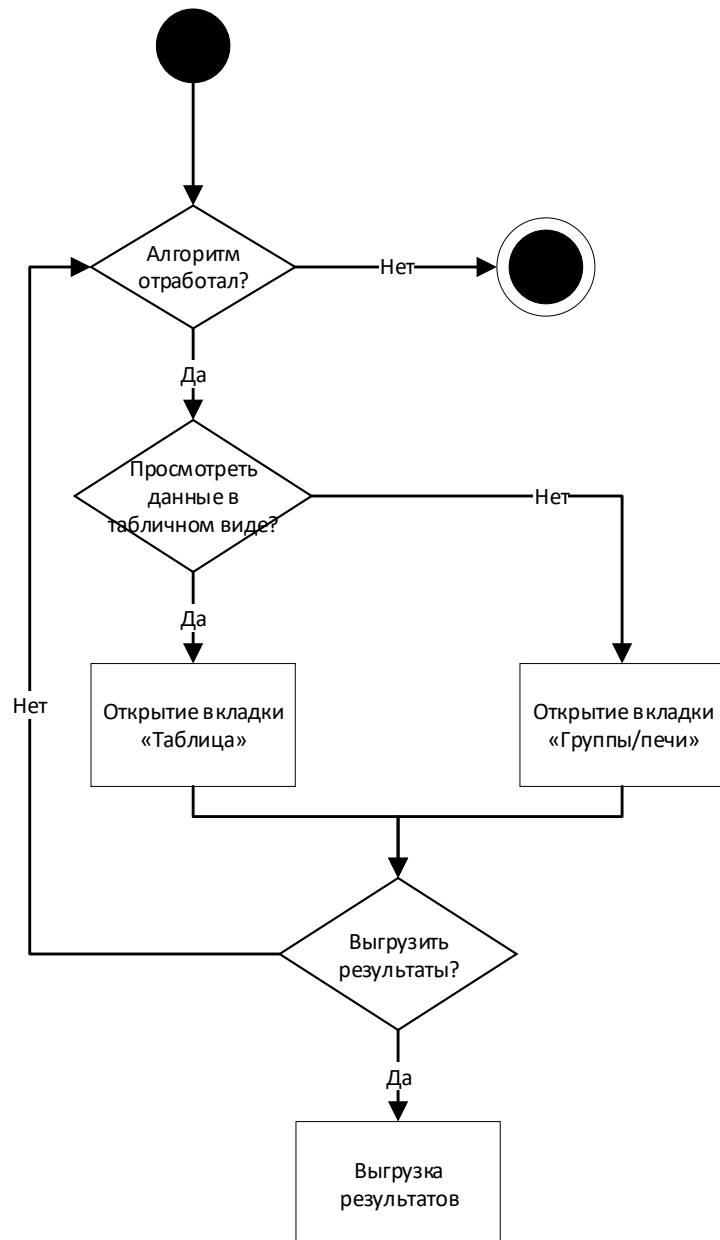


Рис. 22. Диаграмма деятельности для варианта использования «Просмотр графика посадки»



Рис. 23. Диаграмма деятельности для варианта использования «Расчет оптимальных прокатных групп»

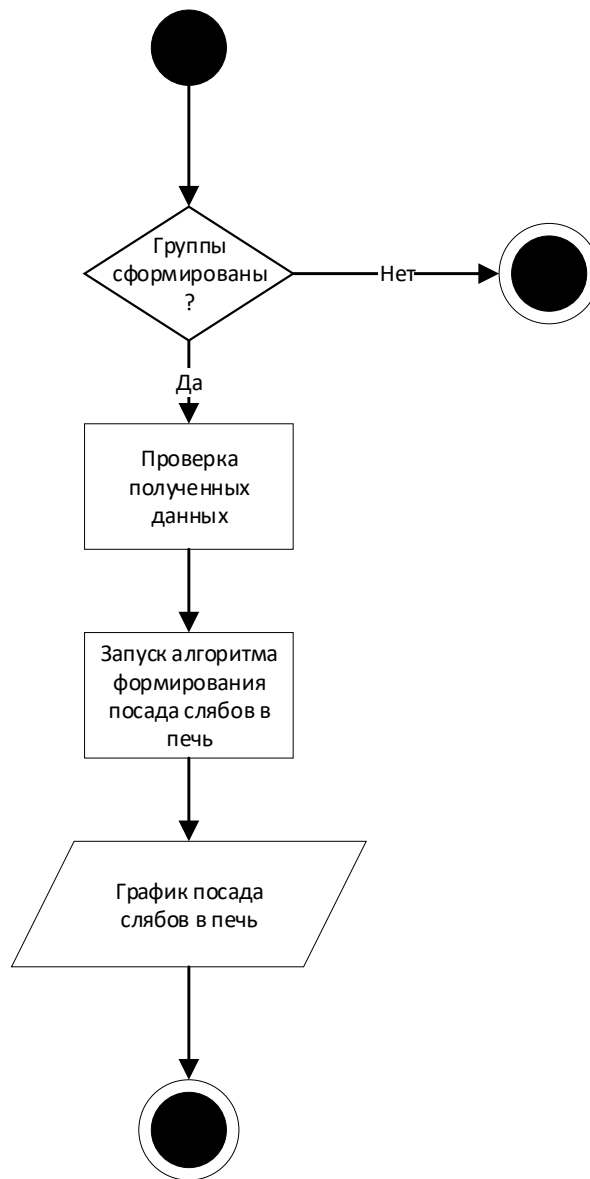


Рис. 24. Диаграмма деятельности для варианта использования «Расчет графика посада слябов в печь»

3.3.4.5 Построение диаграммы пакетов

Диаграмма пакетов представляет собой диаграмму, в которой содержатся пакеты и указываются отношения между ними, она показывает, из каких частей состоит проектируемое программное обеспечение, и как эти части связаны друг с другом [16]. Диаграмма пакетов классов для программного обеспечения системы планирования прокатки и нагрева слябов на стане 5000 показана ниже (рис. 25). Описание пакетов представлено в таблице 33.

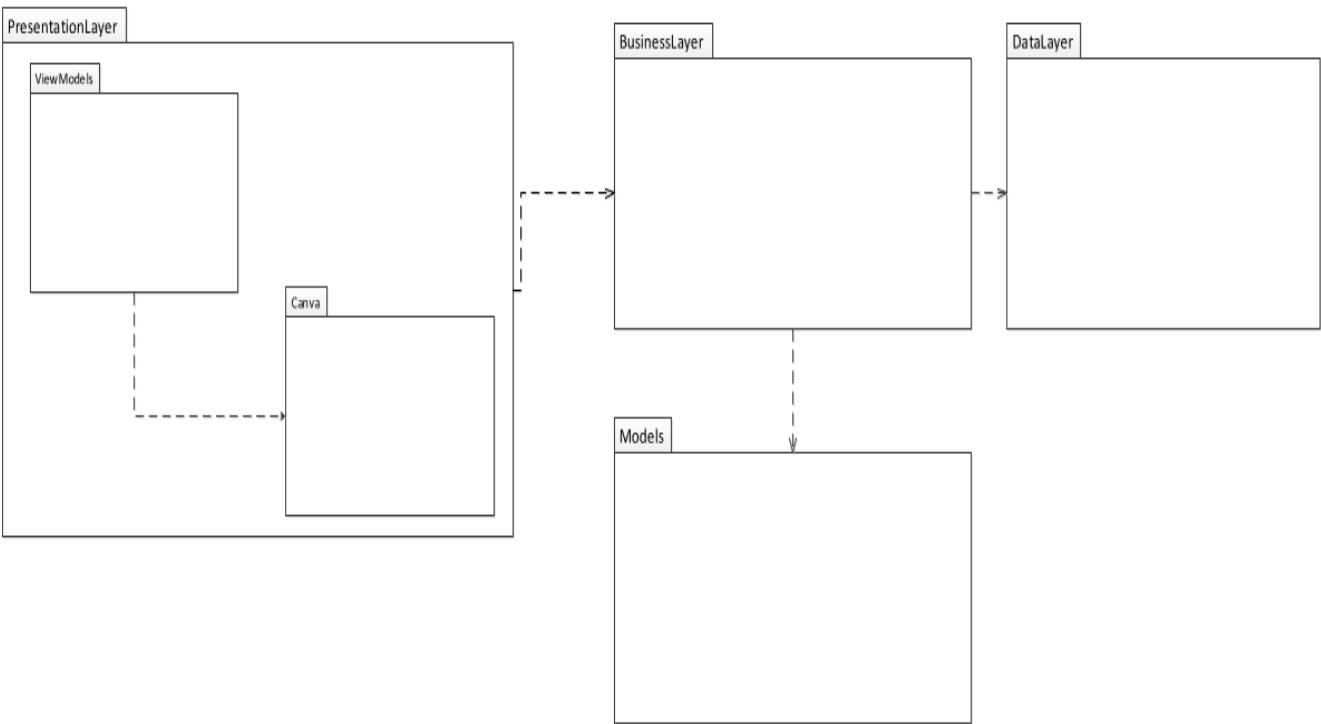


Рис. 25. Диаграмма пакетов

Таблица 33

Описание пакетов

Пакет	Описание
Presentation Layer	Пакет, включает в себя два пакета: ViewModels и Canva. Классы данных пакетов отвечают за взаимодействие пользователя с интерфейсом программы, отображение рабочей плоскости и обработку видеопотока
Business Layer	Пакет, содержащий классы, которые содержат логику работы программы и являющиеся связующим звеном между пакетами Presentation Layer и Data Layer. Определяют механизмы по работе с данными и их отображению
Data Layer	Пакет, содержащий классы, описывающие данные предметной области используемые в программе, а также логику по взаимодействию с ними
Models	Пакет, содержащий классы, отвечающие за представление данных в системе о виде объектов соответствующих классов

3.3.4.6 Детальная диаграмма классов пакета «ViewModels»

Пакет «ViewModels» предназначен для связывания модели и пользовательской формы через механизм привязки данных. ViewModels также содержит логику по получению данных из модели, которые потом передаются в представление. И также ViewModels определяет логику по обновлению данных в модели. Исходная диаграмма классов для пакета «ViewModels» приведена на рис. 26. Описание классов представлено в табл. 34.

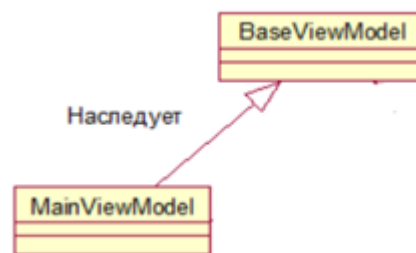


Рис. 26. Диаграмма классов пакета «ViewModels»

Таблица 34

Описание классов пакета «ViewModels»

Название	Описание
BaseViewModel	Базовая модель представления, в которой реализованы функции изменения состояния объекта на экране и функция присваивания нового значения объекта для изменения его состояния
MainViewModel	Модель представления главного экрана

3.3.4.7 Детальная диаграмма классов пакета «Canva»

Пакет «Canva» предназначен для визуализации групп прокатки и информации о посадке слябов в печь. Исходная диаграмма классов пакета «Canva» представлена на рис. 27. Описание классов представлено в табл. 35.

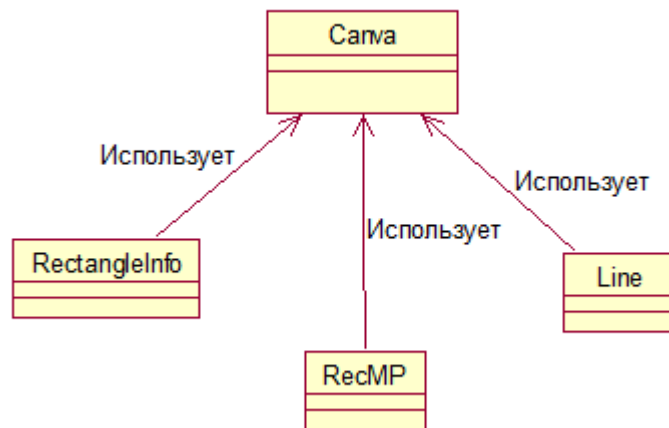


Рис. 27. Детальная диаграмма классов пакета «Canva»

Таблица 35

Описание классов пакета «Canva»

Название	Описание
Canva	Класс для отрисовки данных на канвасе
RectangleInfo	Класс для отрисовки прямоугольника
RecMP	Класс для отрисовки прямоугольника в МП
Line	Класс для отрисовки линий на канвасе

3.3.4.8 Детальная диаграмма классов пакета «Models»

Пакет, содержащий классы, отвечающие за представление данных в системе о виде объектов соответствующих классов.

Исходная диаграмма классов данного пакета представлена на рис. 28. Описание классов представлено в таблице 36.

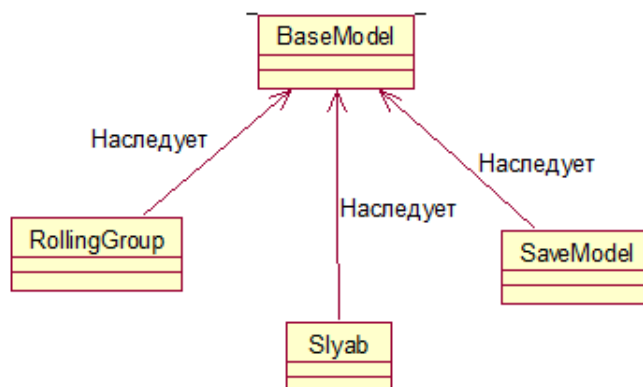


Рис. 28. Детальная диаграмма классов пакета «Models»

Описание классов пакета «Models»

Класс	Описание
BaseModel	Базовая модель, в которой реализованы функции изменения состояния свойств объекта и функция присваивания нового значения объекта для изменения его состояния
RollingGroup	Модель прокатной группы
Slyab	Модель сляба
SaveModel	Модель сохранения данных в БД

3.3.4.9 Детальная диаграмма классов пакета «BusinessLayer»

Пакет, содержащий классы, которые содержат логику работы программы и являющиеся связующим звеном между пакетами Presentation Layer и Data Layer. Определяют механизмы по работе с данными и их отображению.

Исходная диаграмма классов данного пакета представлена на рис. 29. Описание классов представлено в табл. 37.

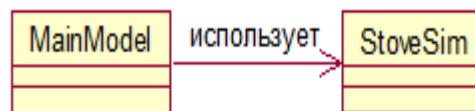


Рис. 29. Детальная диаграмма классов пакета «BusinessLayer»

Описание классов пакета «BusinessLayer»

Класс	Описание
MainModel	Класс, в котором реализуется работа математических моделей по формированию групп слябов
StoveSim	Класс для раскладки сформированных групп по печам

3.3.4.10 Детальная диаграмма классов пакета «DataLayer»

Пакет, содержащий классы, описывающие данные предметной области используемые в программе, а также логику по взаимодействию с ними.

Исходная диаграмма классов данного пакета представлена на рис. 30. Описание классов представлено в табл. 38.

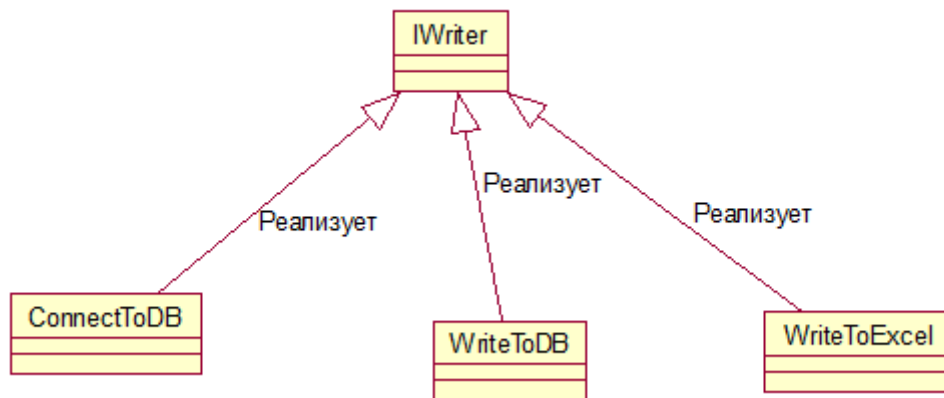


Рис. 30. Детальная диаграмма классов пакета «DataLayer»

Таблица 38

Описание классов пакета «DataLayer»

Класс	Описание
IWriter	Класс, в котором реализован функций для записи в нужный тип файла
ConnectToDB	Класс, отвечающий за соединение с БД
WriteToDB	Класс, отвечающий за запись данных в БД
WriteToLog	Класс, отвечающий за запись в Excel файл

3.3.5 Разработка физической модели

3.3.5.1. Диаграмма компонентов

Диаграммы компонентов применяют при проектировании физической структуры разрабатываемого программного обеспечения. Эти диаграммы показывают, как выглядит программное обеспечение на физическом уровне, т. е. из каких частей оно состоит и как эти части связаны между собой.

Диаграммы компонентов оперируют понятиями компонент и зависимость. Под компонентами при этом понимают физические заменяемые части программного обеспечения, которые соответствуют некоторому набору интерфейсов и обеспечивают их реализацию [15].

Диаграмма компонентов разрабатываемого программного обеспечения представлена на рис. 31.

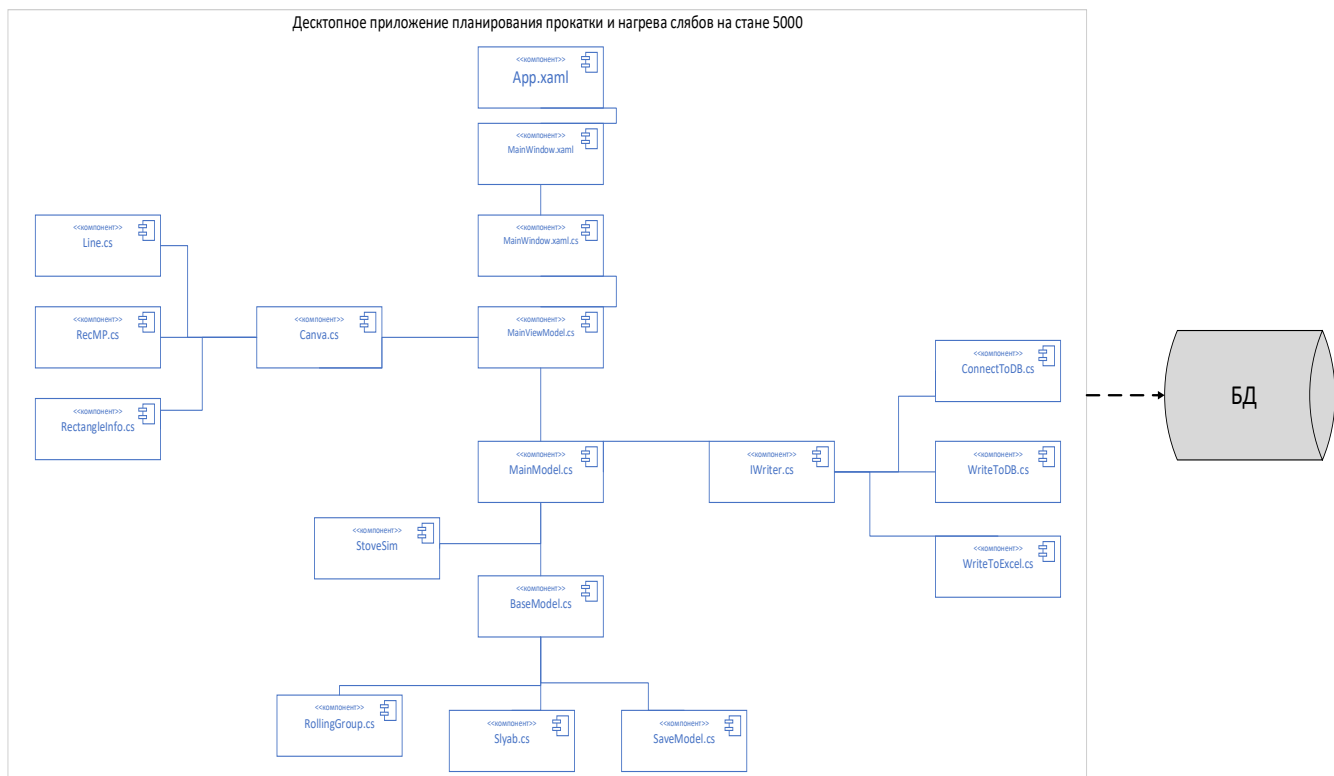


Рис. 31. Диаграмма компонентов

3.3.5.2 Диаграмма размещения

Диаграмма размещения отражает физические взаимосвязи между программными и аппаратными компонентами системы. Каждой части аппаратных средств системы, например, компьютеру или датчику, на диаграмме размещения соответствует узел. Соединения узлов означают наличие в системе соответствующих коммуникационных каналов. Внутри узлов указывают размещенные на данном оборудовании программные

компоненты разрабатываемой программной системы, сохраняя указанные на диаграмме компонентов отношения зависимости [14].

Диаграмма размещения представлена на рис. 32.

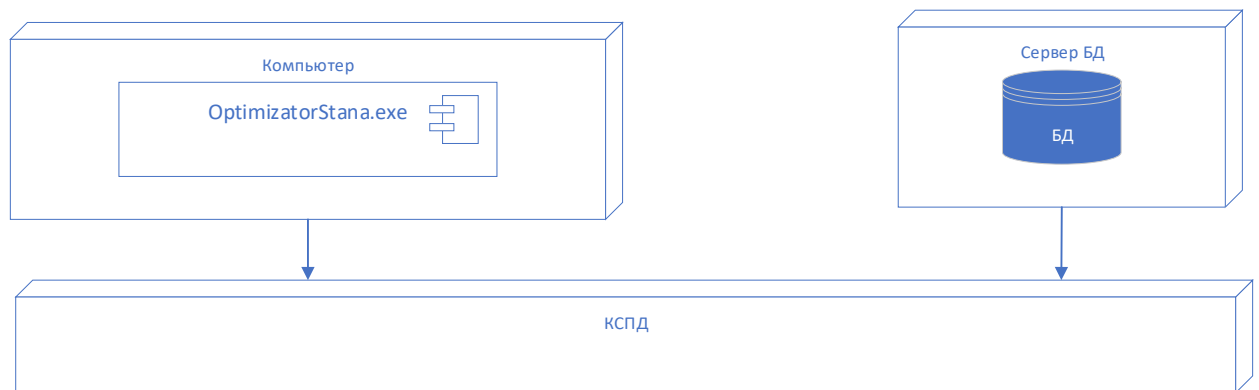


Рис. 32. Диаграмма размещения

ПК оператора по корпоративной сети передачи данных будет связываться с базой данных для сохранения результатов.

3.3.6 Разработка интерфейсов

3.3.6.1 Построение графа диалога

Граф диалога – ориентированный взвешенный граф, каждой вершине которого сопоставлена конкретная картина на экране (кадр) или определенное состояние диалога, характеризующееся набором доступных пользователю действий [34]. Дуги, исходящие из вершин, показывают возможные изменения состояний при выполнении пользователем указанных действий. Дуги, исходящие из вершин, показывают возможные изменения состояний при выполнении пользователем указанных действий. В качестве весов дуг указывают условия переходов из состояния в состояние и операции, выполняемые во время перехода. Под диалогом понимают регламентированный обмен информацией между человеком и программным обеспечением, осуществляемый в реальном масштабе времени и направленный на совместное решение конкретных задач. Каждый диалог состоит из отдельных процессов ввода-вывода, которые физически

обеспечивают связь между пользователем программного обеспечения и компьютером. Обмен информацией между пользователем и компьютером осуществляется с помощью передачи сообщений и управляющих сигналов. Сообщение — это определенная порция информации в диалоговом обмене человека и компьютера [38]. Пользователь в основном генерирует сообщения следующих типов: запрос информации, запрос операции или функции, ввод или изменение информации, выбор зоны и т. д. В ответ он получает: подсказки или справки; информационные сообщения, не требующие ответа; приказы, требующие действий; сообщения об ошибках, нуждающиеся в ответных действиях; изменение формата кадра и т. д. Так как работа с программой имеет четкую последовательность действий, то тип диалога пользователя с программой является «Управляемый программой».

Граф диалога программы представлен на рис. 33.

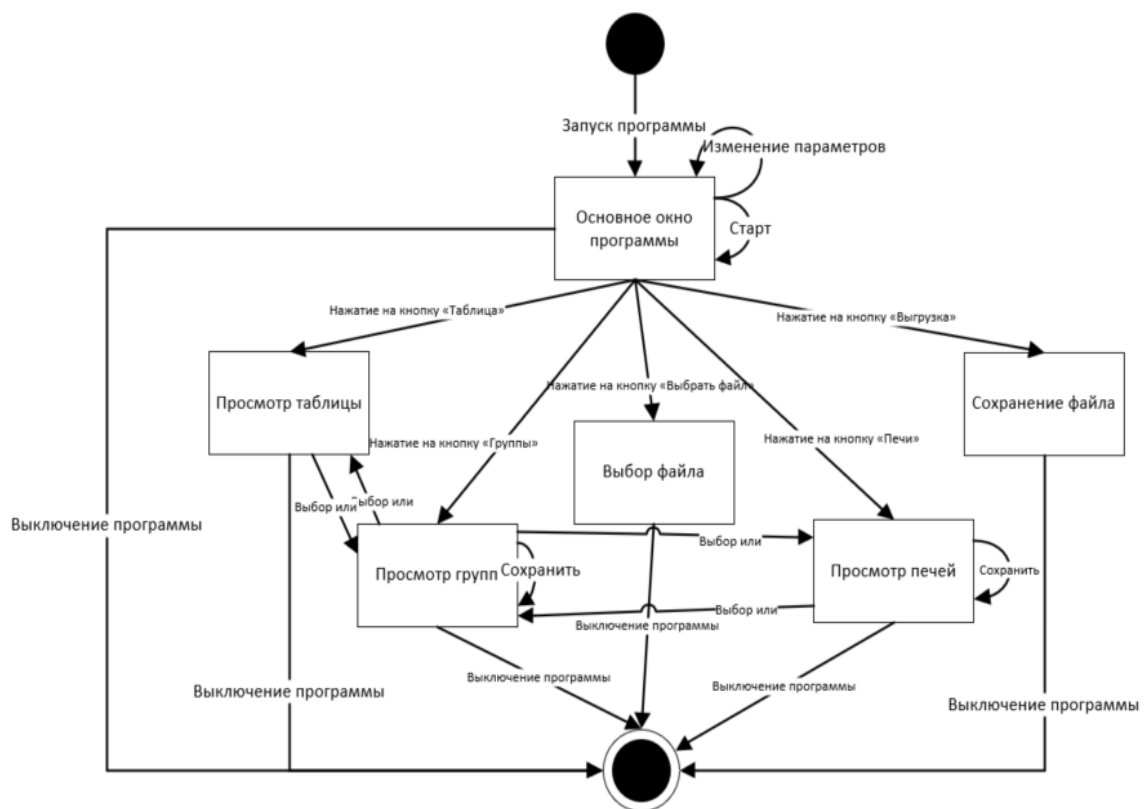


Рис. 33. Граф диалога

3.3.6.2 Разработка форм ввода вывода информации

Интерфейс разрабатываемого программного обеспечения является интерфейсом со свободной навигацией, предназначенный для планирования прокатки и нагрева слябов на стане 5000. На рис. 34 представлено главное окно программы. На данной форме пользователь может ввести необходимые технологические параметры (порядок печей при посадке, время пауз, паузы между группами и т.д.), также с этого окна пользователь может загрузить файл с данными и запустить алгоритм.

Формирование прокатных партий	
Общее время с оптимизацией	98585
Порядок посадки в КП	1 2 3 4 5
Отступ от конца печи	100
Отступ от начала печи	100
Количество сортировок	3
Время на перегон тележки	320
Время посадки 1-го сляба в КП	
Макс время выдачи, ч	6
Время на перемещение МП	100
Время на перемещение КП1	200
Время на перемещение КП2	190
Время на перемещение КП3	180
Пауза между чистой и черновой	35
Пауза между слябами черновой	30
Пауза между слябами чистой	40
Доп пауза между черновой и чистой	15
Допустимая разница длины	10000
Время посадки 1-го сляба в МП	14.02.21 15:00
Пауза между группами	40
Время на перемещение КП4	170
Время на перемещение КП5	160
Время на перемещение КП6	150
Минимальное расстояние в печи	700
Время МП (ч)	8

Рис. 34. Главное окно программы

После того как алгоритм отработает, можно просмотреть результаты в любом виде во вкладках «Таблица» (рис. 35) (данные представлены в табличном варианте), «Группы» (рис. 36) (сформированные группы отображаются в виде рисунка) и «Печи» (рис. 37) (данные по посадке отображаются графически).

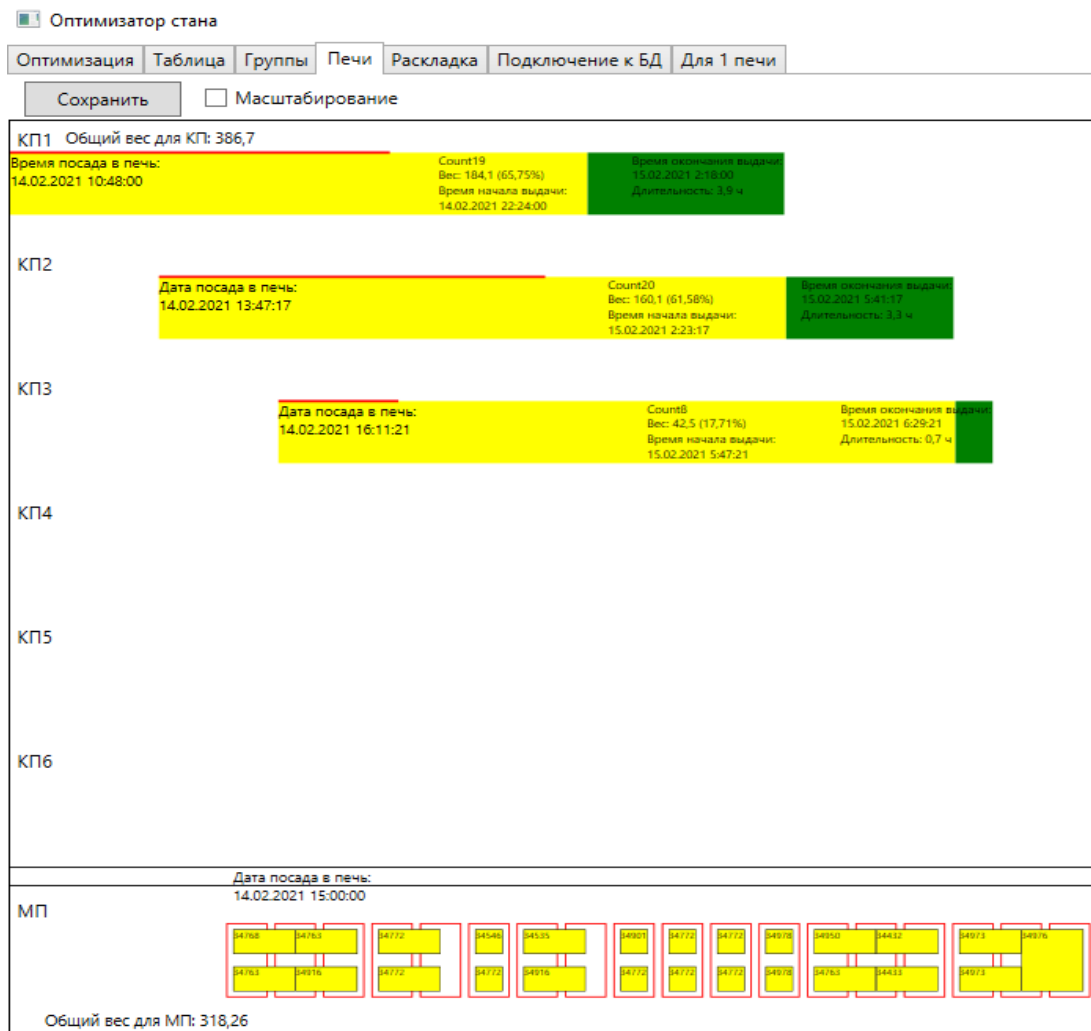


Рис. 37. Вкладка «Печи»

Полное руководство пользователя по данному программному обеспечению представлено в прил. 4.

3.4 Выводы по разделу 3

В данном разделе выполнены все этапы проектирования разрабатываемого программного обеспечения на основе объектно-ориентированного подхода к анализу и проектированию систем. В качестве средств реализации использованы язык высокого уровня программирования C#, интерфейс реализован в системе WPF, вся разработка в среде Visual Studio 2019.

На этапе концептуального моделирования были разработаны модели «черного» и «белого» ящика, состав и структура системы. На этапе

проектирования был произведен выбор подхода к разработке и модели жизненного цикла, а также используемых инструментов.

Были разработаны диаграммы потоков данных в системе. С помощью языка моделирования UML были разработаны: диаграмма вариантов использования, контекстная диаграмма классов, диаграмма пакетов, диаграммы классов пакетов, диаграммы компонентов и диаграмма размещения.

В ходе разработки интерфейса был описан граф диалога пользователя и все формы ввода и вывода информации для программного обеспечения планирования прокатки и нагрева слябов на стане 5000.

4 Результаты экспериментальной проверки программного обеспечения планирования прокатки и нагрева слэбов на стане 5000

4.1 Описание эксперимента и экспериментальных данных

Для подтверждения работы, разработанного алгоритма планирования прокатки и нагрева слэбов на стане 5000, необходимо выполнить сравнительный анализ суммарного времени прокатки слэбов, полученных в процессе работы модели с суммарным временем, полученным при формировании плана специалистами.

Экспериментальными данными будут списки слэбов, из которых будет сформирован заказ на следующие сутки. Данные списки не пересекаются с тренировочными и валидационными выборками и как следствие являются реальным набором данных для подтверждения работы методов и алгоритмов системы планирования прокатки и нагрева слэбов на стане 5000 [2].

Для проведения экспериментальных исследований, как обучения, так и получения результатов работы модели, необходимо использовать ПК, удовлетворяющий минимальным системным требованиям, а именно:

- операционная система (OS) Windows 10;
- процессор (CPU) Intel core I5, не ниже 8-ого поколения, тактовая частота 3.6 ГГц. ;
- видеокарта (GPU) nvidia geforce gtx 1080 ti, с объемом памяти, не менее 8 гигабайт, тактовая частота 4400 ГГц. – 6600 ГГц.;
- оперативная память (RAM) объемом не менее 16 гигабайт, с тактовой частотой, не менее 2200 ГГц.

Весь эксперимент будет проводиться на стане 5000, схема которого представлена на рис. 38.

УКО -Установка контролируемого охлаждения;

ЛПМ – листоправильная машина;

ГС – гидросбив;

КП – камерная печь;

МП – методическая печь.

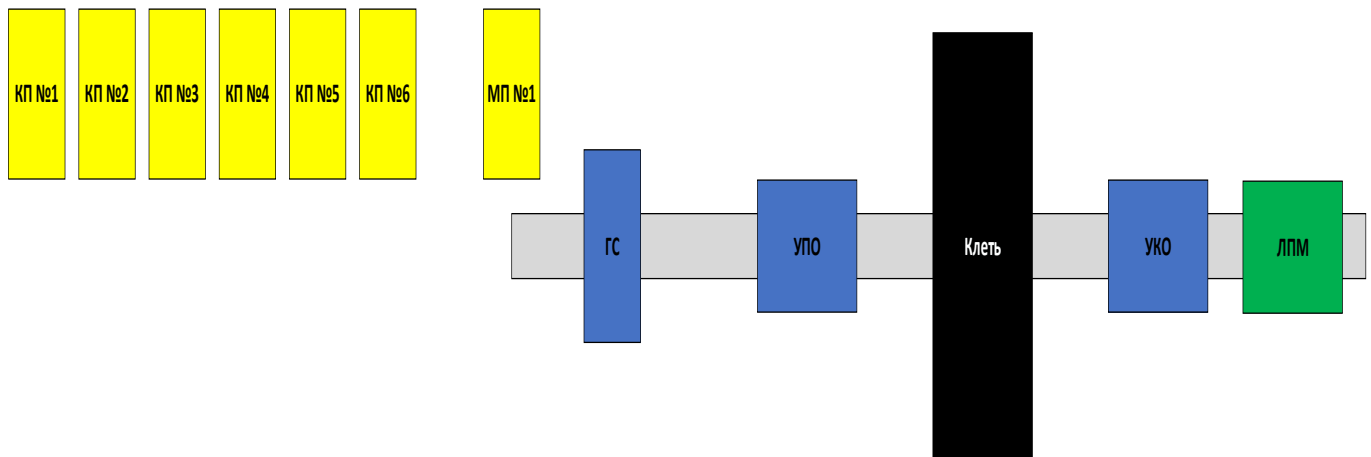


Рис. 38. Структурно-функциональная схема эксперимента

Мастер нагревательных печей, запускает разработанное ПО на установленном там компьютере, выставляя необходимые параметры, а также план заказов, которые необходимо выполнить на следующие сутки.

После чего получает список сформированных прокатных групп и график посада слябов в печь, и в соответствии с полученным планом управляет загрузкой камерных и методической печей, а также работой стана.

Испытание разработанного решения проводилось в формате А/В-тестирования с разделением плана заказов на следующие сутки на две группы: с применением полученных результатов решения и списком подготовленным специалистом.

Для выполнения эксперимента был разработан стандарт для предоставления исходного набора заказов. Для удобства набор заказов предоставляется в виде файла формата Microsoft Excel, состоящего из набора следующих столбцов, как показано в табл. 39.

Информация, необходимая для использования в программе исходного
набора заказов

Название столбца	Размерность	Расшифровка	Примечание
МАРКА	-	марка стали	Марка стали
ТОЛЩИНА	Мм	толщина	Толщина сляба
ШИРИНА	мм	ширина	Ширина сляба
ДЛИНА	Мм	длина	Длина сляба
ВЕС	Т	вес заказа	-
Продолжительность черновой прокатки	сек	Время черновой прокатки	-
Продолжительность чистовой прокатки	сек	Время чистовой прокатки	-
Продолжительность подстуживания	сек	Время подстуживания	
КП1	мин	Время нахождения сляба в печи	
КП2	мин	Время нахождения сляба в печи	
КП3	мин	Время нахождения сляба в печи	
КП4	мин	Время нахождения сляба в печи	
КП5	мин	Время нахождения сляба в печи	
КП6	мин	Время нахождения сляба в печи	
МП	мин	Время нахождения сляба в печи	
УКО\УТУ	-	Принадлежность сляба к УКО или УТУ	

Выходные данные так же сохраняются в файл формата Microsoft Excel, для удобства их дальнейшего использования и возможности редактирования. Файл выводится на экран по окончании расчета. Столбцы файла описаны в табл. 40.

Информация, необходимая для использования в программе выходного
набора заказов

Название столбца	Размерность	Расшифровка	Примечание
МАРКА	-	марка стали	Марка стали
ТОЛЩИНА	Мм	толщина	Толщина сляба
ШИРИНА	мм	ширина	Ширина сляба
ДЛИНА	Мм	длина	Длина сляба
ВЕС	Т	вес заказа	-
Продолжительность черновой прокатки	сек	Время черновой прокатки	-
Продолжительность чистовой прокатки	сек	Время чистовой прокатки	-
Продолжительность подстуживания	сек	Время подстуживания	-
GroupNum	-	Номер группы	-
SlyabNum	-	Номер сляба в группе	-
Stove	-	Какая печь КП/МП	-
StoveNum	-	Номер КП	-
Дата посада сляба в печь	Дата	Время посада сляба в печь	-
Дата забора сляба из печи	Дата	Время забора сляба из печи	-
Время прихода на стан	Дата	Время прихода сляба на стан	-

Для проведения эксперимента представим часть набора слябов для
формирования прокатных групп и графика посада слябов в печь в табл. 41.

Таблица 41

Набор слябов

Марка стали	Сляб, Н	Сляб, W	Сляб, L	Прод-ть черновой	Прод-ть чист	Прод-ть подстуж	КП1 min	КП2 min	КП3 min	КП4 min	КП5 min	КП6 min	МП min	вес сляба	УКО/УТУ
22К	313	1995	4051	442	0	0	690	750	810	780	750	810	480	19,279	N/N
22К	313	1995	2020	339	0	0	690	750	810	780	750	810	480	9,838	N/N
22К	313	1996	1038	302	0	0	690	750	810	780	750	810	480	5,059	N/N
22К	313	1996	2019	339	0	0	690	750	810	780	750	810	480	9,838	N/N
22К	313	1996	2019	339	0	0	690	750	810	780	750	810	480	9,838	N/N
22К	313	1996	2019	339	0	0	690	750	810	780	750	810	480	9,838	N/N
22К	313	1996	2019	339	0	0	690	750	810	780	750	810	480	9,838	N/N
22К	313	1996	2019	339	0	0	690	750	810	780	750	810	480	9,838	N/N
22К	313	1996	2019	339	0	0	690	750	810	780	750	810	480	9,838	N/N
22К	313	1996	2019	339	0	0	690	750	810	780	750	810	480	9,838	N/N
22К	313	1996	2019	339	0	0	690	750	810	780	750	810	480	9,838	N/N
09Г2С	313	1995	3977	60	80	780	690	750	0	780	750	0	0	19,371	Y/N
09Г2С	313	1995	3977	60	80	780	690	750	0	780	750	0	0	19,371	Y/N
22К	313	1995	4051	442	0	0	690	750	810	780	750	810	480	19,279	N/N
09Г2С	313	1995	4000	60	80	790	690	750	0	780	750	0	0	19,48	Y/N
20	313	2002	1838	80	90	480	690	750	810	780	750	810	0	8,983	N/N
10ХСНД	309	1715	2830	326	0	0	0	750	810	780	750	810	0	11,698	Y/Y

4.2 Разработка методики настройки информационного и программного обеспечения

Требования к информационному обеспечению:

Требования к организации выходным данным. В результате работы системы будут получены следующие виды выходных данных:

- список спланированных прокатных партий в формате .xlsx;
- скриншоты сформированных групп и заполненностей печей в формате .png;
- запись в БД.

Требования к аппаратно-программному обеспечению:

Надежное (устойчивое) функционирование программы должно быть обеспечено выполнением совокупности организационно-технических мероприятий, перечень которых приведен ниже:

- организацией бесперебойного питания технических средств;
- использованием лицензионного программного обеспечения;
- регулярным выполнением рекомендаций Министерства труда и социального развития РФ, изложенных в Постановлении от 23 июля 1998 г. «Об утверждении межотраслевых типовых норм времени на работы по сервисному обслуживанию ПЭВМ и оргтехники и сопровождению программных средств»;
- регулярным выполнением требований ГОСТ 51188-98. Защита информации. Испытания программных средств на наличие компьютерных вирусов.

Условия эксплуатации. Условия эксплуатации данного программного обеспечения должны соответствовать условиям эксплуатации компьютера оператора. В случае затруднений у оператора, должно быть подготовлено руководство оператора, описывающее действия, которые он может выполнить и описания интерфейса с требованиями к вводимым данным.

Для проверки работоспособности разработанного программного обеспечения предполагается следующий состав мероприятий по

тестированию:

- тестирование разработанных модулей;
- тестирование программы на соответствие функциональных требований;
- тестирование надежности программы на соответствие требований к надежности;
- проверка программной документации на соответствие требований ЕСПД.

Тестирование проводилось на персональном компьютере со следующими характеристиками:

- 1.Объем оперативной памяти 16 Гб.
- 2.Монитор с разрешением 1920x1080.
- 3.Операционная система Windows10 Домашняя.
- 4.Твердотельный накопитель объемом 1 Тб.
- 5.Процессор AMD Ryzen 5 2600 (4 ядер, 8 потоков, тактовая 3.4 ГГц).
- 6.Видеокарта Gigabyte GeForce GTX 1660Ti с поддержкой DirectX 12.

Для тестирования программной документации использовался метод ручного контроля. Для тестирования программного обеспечения использовалось два метода: модульное тестирование и системное тестирование [13].

- 2 Модульное тестирование, иногда блочное тестирование или юнит-тестирование (англ. Unit testing) — процесс в программировании, позволяющий проверить на корректность отдельные модули исходного кода программы, наборы из одного или более программных модулей вместе с соответствующими управляющими данными, процедурами использования и обработки [29].

Цель модульного тестирования — изолировать отдельные части программы и показать, что по отдельности эти части работоспособны. Идея состоит в том, чтобы писать тесты для каждой нетривиальной функции или метода. Это позволяет достаточно быстро проверить, не привело ли очередное изменение кода к регрессии, то есть к появлению ошибок в уже

оттестированных местах программы, а также облегчает обнаружение и устранение таких ошибок.

3 Системное тестирование программного обеспечения — это тестирование программного обеспечения (ПО), выполняемое на полной, интегрированной системе, с целью проверки соответствия системы исходным требованиям.

Системное тестирование выполняется методом «Черного ящика», т.к. проверяемое множество является «внешними» сущностями, которые не требуют взаимодействия с внутренним устройством программы. Выполняя системное тестирование, можно обнаружить следующие типы дефектов:

- непредусмотренные комбинации пользовательских данных;
- проблемы с совместимостью окружения;
- несоответствие с функциональными требованиями;
- плохое удобство использования.

Результаты проведения испытаний:

В результате проверки программной документации методом ручного контроля были исправлены найденные грамматические и пунктуационные ошибки, было проверено соответствие документации требованиям ЕСПД. Также были проверены построенные схемы и диаграммы на соответствие ГОСТ.

Результаты тестирования компонентов ПО представлены в табл. 42.

Таблица 42

Тестирование компонентов

Дата	Модуль	Кто проводил тестирование	Способ тестирования	Результаты тестирования
1	2	3	4	5
13.05.23	App.xml	Разработчик	Функциональное тестирование	Успех
13.05.23	MainWindow.xaml	Разработчик	Функциональное тестирование	Успех

Продолжение табл. 42

1	2	3	4	5
13.05.23	MainWindow.xaml .cs	Разработчик	Функциональное тестирование	Успех
13.05.23	ExcelLoader.cs	Разработчик	Функциональное тестирование	Успех
14.05.23	MainModel.cs	Разработчик	Функциональное тестирование	Успех
14.05.23	RollingGroup.cs	Разработчик	Функциональное тестирование	Успех
14.05.23	SlyabSim.cs	Разработчик	Функциональное тестирование	Успех
14.05.23	SaveModel.cs	Разработчик	Функциональное тестирование	Успех
14.05.23	OracleDataBase.cs	Разработчик	Функциональное тестирование	Успех
14.05.23	RectangleSim.cs	Разработчик	Функциональное тестирование	Успех
14.05.23	RectangleInfo.cs	Разработчик	Функциональное тестирование	Успех
14.05.23	MLine.cs	Разработчик	Функциональное тестирование	Успех
15.05.23	RecMPSimplex.cs	Разработчик	Функциональное тестирование	Успех
15.05.23	FindInDB.cs	Разработчик	Функциональное тестирование	Успех

Результаты тестирования программного обеспечения на выполнение требований к функциональным характеристикам представлены в табл. 43.

Тестирование функциональных характеристик

Дата и время тестирования	Тестируемое требование	Кто проводил тестирование	Способ тестирования	Результаты тестирования
10.05.23	Считывание данных с Excel файла	Разработчик	Системное тестирование	Успех
11.05.23	Работа алгоритма с заданными параметрами	Разработчик	Системное тестирование	Успех
16.05.23	Формирование групп слябов согласно заданным требованиям	Разработчик	Системное тестирование	Успех
16.05.23	Формирование посада слябов в печь согласно заданным требованиям	Разработчик	Системное тестирование	Успех
17.05.23	Вывод данных в табличном виде	Разработчик	Системное тестирование	Успех
17.05.23	Вывод сформированных групп в графическом виде	Разработчик	Системное тестирование	Успех
17.05.23	Вывод посада слябов в печь в графическом виде	Разработчик	Системное тестирование	Успех
17.05.23	Сохранение данных в Excel файл	Разработчик	Системное тестирование	Успех
18.05.23	Подключение к БД	Разработчик	Системное тестирование	Успех

Результаты тестирования программного обеспечения на выполнение требований к надежности продукта представлены в табл. 44.

Тестирование требований к надежности

Дата и время тестирования	Тестируемое требование из ТЗ	Кто проводил тестирование	Способ тестирования	Результаты тестирования
20.05.23	Оперативное сохранение информации о результатах работы программы	Разработчик	Системное тестирование	Успех
20.05.23	Обеспечение проверки вводимых пользователем данных	Разработчик	Системное тестирование	Успех
21.05.23	Система должна иметь защиту от некорректных действий оператора и ошибочных исходных данных	Разработчик	Системное тестирование	Успех

В результате модульного тестирования пакетов были выявлены недостатки в разработанном программном обеспечении, которые были устранены в результате доработки. В ходе системного тестирования были также устранены недостатки в работе системы и установлен факт соблюдения всех предъявляемых требований. Таким образом, можно сделать вывод, что разработанное программное обеспечения выполняет все заявленные требования в полном объеме.

4.3 Результаты экспериментальной проверки модели

После получения данных, описанных выше, запустим разработанный алгоритм и получим суммарное время прокатки, а также сформированные прокатные группы и график посяда слябов в печь. Часть полученных результатов представим в табл. 45, а также продемонстрируем работу программы на рис. 39 – 41.

Оптимизатор стана

Оптимизация Таблица Группы Печи Раскладка Подключение к БД

Формирование прокатных партий

Старт

Выгрузить результат

Загрузить файл

Общее время с оптимизацией: 98580

Порядок посадки в КП: 1 2 3 4 5

Отступ от конца печи: 100

Отступ от начала печи: 100

Количество сортировок: 3

Время на перегон тележки: 320

Время посадки 1-го сляба в КП:

Макс время выдачи, ч: 6

Время на перемещение МП: 100

Время на перемещение КП1: 200

Время на перемещение КП2: 190

Время на перемещение КП3: 180

Пауза между чистой и черновой: 35

Пауза между слябами черновой: 30

Пауза между слябами чистой: 40

Доп пауза между черновой и чист: 15

Допустимая разница длины: 10000

Время посадки 1-го сляба в МП: 14.02.21 15:00

Пауза между группами: 40

Время на перемещение КП4: 170

Время на перемещение КП5: 160

Время на перемещение КП6: 150

Минимальное расстояние в печи: 700

Время МП (ч): 8

Рис. 39. Главное окно программы

Как можно заметить на рис. 36 отображается общее суммарное время прокатки с оптимизацией для заданного набора слябов в данном случае оно равняется 98580 секундам.



Рис. 40. Сформированные прокатные группы

КП1 Общий вес для КП: 491,9

Время посадки в печь:
14.02.2021 11:58:20

Count24
Вес: 246,3 (87,96%)
Время начала выдачи:
14.02.2021 23:34:20

Время окончания выдачи:
15.02.2021 5:22:20
Длительность: 5,8 ч

КП2

Дата посадки в печь:
14.02.2021 17:08:18

Count18
Вес: 169,6 (65,23%)
Время начала выдачи:
15.02.2021 5:44:18

Время окончания выдачи:
15.02.2021 8:32:18
Длительность: 2,8 ч

КП3

Дата посадки в печь:
14.02.2021 18:59:18

Count11
Вес: 76 (31,67%)

Время окончания выдачи:
15.02.2021 9:29:18
Длительность: 6,0 ч

КП4

Марка	Номер прокатки	Геом Листа Н	Геом Листа W	Геом Листа L	Сляб, Н	Сл
20	34983	20	2000	6000	248	145
20	34983	20	2000	6000	248	145
20	34984	20	2000	6000	248	145
20	34983	20	2000	6000	248	145
20	34983	20	2000	6000	248	145
10ХСНД	34519	90	2200	6000	309	170
10ХСНД	34519	90	2200	6000	309	170
10ХСНД	34523	30	2000	6000	309	170
20	34778	12	2000	6000	170	169
20	34778	12	2000	6000	170	169
10ХСНД	34518	110	2000	4200	309	170
K60	35005	28	3923	12050	313	199
K60	35006	28	3923	12050	313	199
K60	35006	28	3923	12050	313	199
K60	35006	28	3923	12050	313	199
K60	35006	28	3923	12050	313	199
K60	35006	28	3923	12050	313	199

КП6

Дата посадки в печь:
14.02.2021 15:00:00

МП

Б5005	Б5006	Б5006	Б4768	Б4763	Б4763	Б4546	Б4772	Б4772	Б4772	Б4433	Б4432	Б4916	Б4978	Б4950
Б5006	Б5006	Б5006	Б4763	Б4763	Б4772	Б4772	Б4772	Б4772	Б4772	Б4916	Б4535	Б4535	Б4978	Б4822

Рис. 41. Раскладка по печам

На рис. 41 можно увидеть, как раскладываются слябы по печам, а также полную информацию по каждому слябу.

Таблица 45

Результаты

Марка	Вес	Номер группы	Номер сляба в группе	Печ ь	Номер КП	Сляб, Н	Сляб, W	Сляб, L	Прод-ть черн-овой	Прод-ть чистой	Прод-ть подстуж ивания	Дата посады в печь	Время забора из печи	Время прихода на стан
S355K2	7,652	29	1	МП	0	313	1715	1828	110	95	240	14.02.2021 15:00:00	14.02.2021 22:01:40	14.02.2021 22:03:20
S355J2	8,077	29	2	МП	0	313	1990	1662	105	95	175	14.02.2021 15:00:00	14.02.2021 22:05:05	14.02.2021 22:06:45
S355J2	8,078	31	1	МП	0	313	1990	1663	105	95	175	14.02.2021 15:10:20	14.02.2021 22:12:00	14.02.2021 22:13:40
S355J2	7,841	32	1	МП	0	313	1710	1878	115	95	240	14.02.2021 15:10:20	14.02.2021 22:18:55	14.02.2021 22:20:35
S355J2	7,841	32	2	КП	1	313	1715	1873	115	95	240	14.02.2021 10:48:00	14.02.2021 22:24:00	14.02.2021 22:27:20
S355J2	7,785	33	1	МП	0	313	1995	1598	110	95	175	14.02.2021 15:27:50	14.02.2021 22:29:30	14.02.2021 22:31:10
S355J2	7,785	33	2	МП	0	313	1995	1598	110	95	175	14.02.2021 15:27:50	14.02.2021 22:31:50	14.02.2021 22:33:30
S355J2	4,602	35	1	КП	1	313	1715	1099	120	105	510	14.02.2021 10:48:00	14.02.2021 22:36:00	14.02.2021 22:39:20
S355J2	7,091	35	2	КП	1	313	1710	1699	100	95	240	14.02.2021 10:48:00	14.02.2021 22:48:00	14.02.2021 22:51:20
S355J2	6,976	36	1	МП	0	313	1995	1432	110	95	180	14.02.2021 15:52:20	14.02.2021 22:54:00	14.02.2021 22:55:40

В итоге полученное суммарное время прокатки и нагрева слэбов получилось 98580 секунд. В то время как при планировании плана специалистом было затрачено 103500 секунд.

Тестирование разработанных моделей было проведено во время промышленной эксплуатации, то есть непосредственно на производстве. Испытание разработанного интеллектуального решения проводилось в формате А/В-тестирования с разделением плана заказов на следующие сутки на две группы: с применением полученных результатов решения и списком подготовленным специалистом [41].

Было проведено еще несколько экспериментов, результаты которых представлены в табл. 46 и на графике (рис. 42).

Таблица 46

Результаты тестирования

Количество штук	Время с новым алгоритмом (сек)	Время без алгоритма (сек)
200	79 850	83 842
265	98 575	103 500
300	112 345	116 000

Применение разработанного алгоритма с использованием математических моделей в рамках промышленной эксплуатации принесло уменьшение суммарного времени прокатки плана заказов на 2,5-5,0 %, в отличии от ручного планирования специалистами, поэтому внедрение решения является целесообразным.

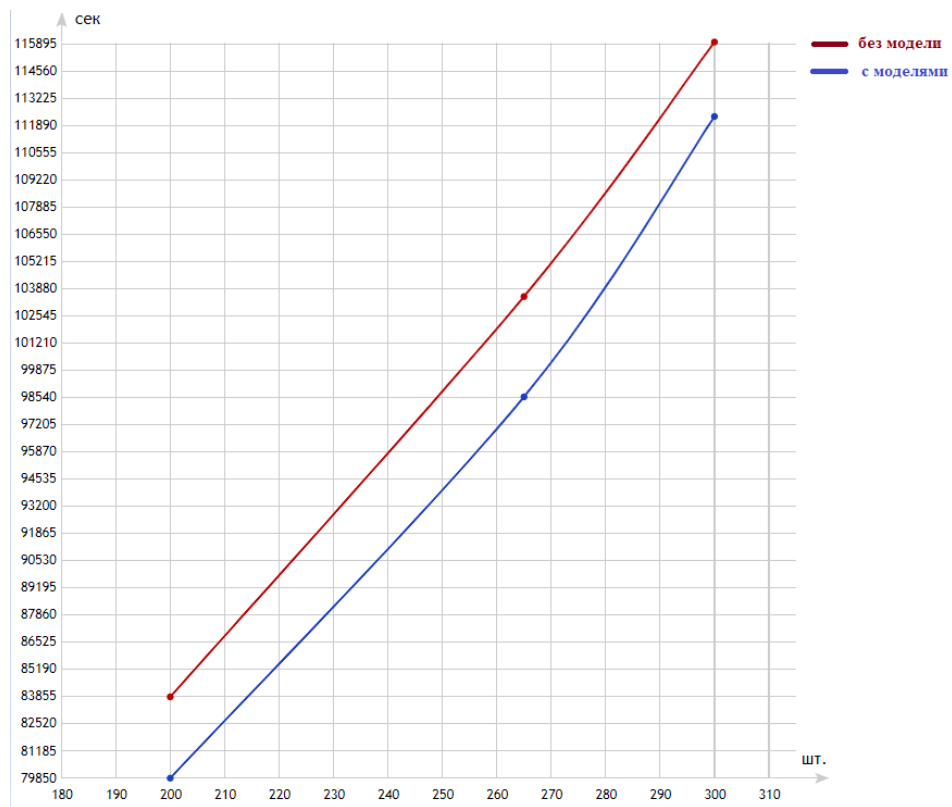


Рис. 42. Результаты тестирования

4.4 Перспективы применения программного обеспечения планирования прокатки и нагрева слэбов на стане 5000

Предложенный математический аппарат для планирования прокатки и нагрева слэбов позволит предприятию быстрее выполнять заказы, в следствие чего за тот же самый отрезок времени появляется возможность производить больше заказов.

Практическая значимость работы заключается в возможности применения полученного в ходе исследования программного продукта, который может с помощью применения математических моделей формировать прокатные группы с учетом всех требований, а также формировать посад слэбов в печь согласно заданным параметрам. Итоговый результат пользователь может увидеть как в отрисованном, так и табличном варианте.

Описанные математические модели и алгоритм позволяют планировать прокатку и нагрев слэбов на стане 5000 в результате чего получается

суммарное затраченное время на выполнение заказа меньше, чем при ручном планировании специалистами.

4.5 Выводы по разделу 4

В разделе 4 данной работы была выполнена апробация предложенных ранее моделей программного обеспечения планирования прокатки и нагрева слябов на стане 5000.

Разработанный алгоритм, объединяющий процессы прокатки и нагрева слябов одновременно, доказал свою пригодность в сравнении с тем, что используется на данный момент, так как суммарное время выполнения заказа на 2,5-5,0 % быстрее, в отличие от того, если формирование партий будет происходить специалистом вручную.

Заключение

Результатом выпускной квалификационной работы является разработанное программное обеспечение планирования прокатки и нагрева слябов на стане 5000, для платформ с операционной системой Windows.

По результатам выполнения работы поставленная цель достигнута и получены следующие результаты:

1. Проведенный анализ существующих технических решений помог выявить необходимость разработки программного обеспечения планирования прокатки и нагрева слябов на стане.

2. Выполнено описание математического аппарата и алгоритмов его работы для планирования прокатки и нагрева слябов на стане 5000, с помощью данных моделей уменьшается суммарное время прокатки, что позволяет выполнять большее количество заказов.

3. Выполнено проектирование системы, которое включает в себя:

- Концептуальное моделирование: модели «черного ящика», состава, структуры «белого ящика» системы.
- Проектирование информационного обеспечения: выбор подхода к разработке и модели жизненного цикла программного обеспечения, а также используемых инструментов.
- Разработка диаграмм потоков данных в системе.
- Разработка логической модели системы: диаграмма использования, контактная диаграмма классов, структуры данных, диаграммы пакетов, диаграммы классов пакетов и физическая модели системы.
- Разработка интерфейса: графы диалога пользователя, формы ввода и вывода информации

4. Выполнена апробация разработанной модели.

По итогу выполнения работы мною были освоены следующие компетенции [12]:

- Способен самостоятельно приобретать, развивать и применять математические, естественнонаучные, социально-экономические и профессиональные знания для решения нестандартных задач, в том числе, в новой или незнакомой среде и в междисциплинарном контексте.
- Способен разрабатывать оригинальные алгоритмы и программные средства, в том числе с использованием современных интеллектуальных технологий, для решения профессиональных задач.
- Способен разрабатывать и модернизировать программное и аппаратное обеспечение информационных и автоматизированных систем.
- Способен самостоятельно приобретать с помощью информационных технологий и использовать в практической деятельности новые знания и умения, в том числе в новых областях знаний, непосредственно не связанных со сферой деятельности.
- Способен применять при решении профессиональных задач методы и средства получения, хранения, переработки и трансляции информации посредством современных компьютерных технологий, в том числе, в глобальных компьютерных сетях.
- Способен осуществлять эффективное управление разработкой программных средств и проектов.
- Способен владеть навыками выбора инструментальных средств разработки, определения набора библиотек повторно используемых модулей, выбора средств создания и ведения репозитория, учета задач, сборки и непрерывной интеграции, базы знаний, организации процесса использования инфраструктуры, мониторинга функционирования инфраструктуры, принятия управленческих решений, определения областей применения процесса управления рисками, определения стратегий и приоритетов управления рисками, выявления и отслеживания рисков в процессе разработки программного обеспечения, анализа и оценки выявленных рисков, выбора способов реагирования на них и выделения необходимых ресурсов, структурной декомпозиции работ.

Список литературы

1. Абрамов, Г.В. Проектирование информационных систем : учебное пособие / Г.В. Абрамов, И.Е. Медведкова, Л.А. Коробова. – Воронеж : Воронежский государственный университет инженерных технологий, 2012. – 172 с.
2. Альфред В. Ахо, Джон Хопкрофт, Джеффри Д. Ульман. Структуры данных и алгоритмы = Data Structures and Algorithms. — М.: Вильямс, 2000. — 384 с.
3. Антамошкин, О.А. Программная инженерия. Теория и практика : учебник / О.А. Антамошкин ; Сибирский федеральный университет. – Красноярск : Сибирский федеральный университет (СФУ), 2012. –247 с.
4. Балдин, К.В. Математическое программирование : учебник / К.В. Балдин, Н.А. Брызгалов, А.В. Рукусуев ; под общ. ред. К.В. Балдина. – 2-е изд. – Москва : Дашков и К, 2018. – 218 с.
5. Березин А.А., Вакула И.А., Леонова С.И. Задача планирования горячей прокатки // Современные проблемы математики и её приложений - Екатеринбург: Институт математики и механики УрО РАН им. Н.Н. Красовского, 2015. - С. 98-104.
6. Бишоп, Дж. C# в кратком изложении / Дж. Бишоп, Н. Хорспул. – М.: Бином. Лаборатория знаний, 2011. 472 с.
7. Боггс, М. UML и Rational Rose / М. Боггс. – Москва: РГГУ, 2010. –385 с.
8. Борисов И.А., Львова С.С. Выбор стали для валков прокатных станов // Металловедение и термическая обработка металлов. - 2009. - №6 (648). - С. 8-14.
9. Буч, Г., Язык UML. Руководство пользователя / Грейди Буч, Джеймс Рамбо, Айвар Джекобсон. – М.: ДМК, 2015. – 432 с
10. Гладков, Л.А. Методы решения задач оптимизации: учебное пособие: / Л.А. Гладков, Н.В. Гладкова; Южный федеральный университет. –

- Ростов-на-Дону ; Таганрог : Южный федеральный университет, 2019. – 119 с.
11. Грудев А. П., Машкин Л. Ф., Ханин М. И. Технология прокатного производства изд., М: Металлургия, 1994 г. – 656 с.
 12. Ершов Е.В. Программа практики. Производственная. 09.04.04 Программная инженерия. / Ершов Е.В. – Череповец.: Череповецкий государственный университет, 2020. – 56 с
 13. Ершов Е.В., д-р техн. наук, проф.; Виноградова Л.Н. и др. Методика и организация самостоятельной работы студентов – Коллектив авторов, ФГБОУ ВПО «Череповецкий государственный университет», 2012. – 208 с.
 14. Ефимова, И.Ю. Компьютерное моделирование: сборник практических работ / И.Ю. Ефимова, Т.Ю. Варфоломеева. – 2-е изд., стер. – Москва : Флинта, 2014. – 68 с.
 15. Иванова, Г.С. Технология программирования: Учебник для вузов. - М.: Изд-во МГТУ им. Н.Э. Баумана, 2002. -320 с.: ил. (Сер. Информатика в техническом университете.).
 16. Камаев, В.А. Технологии программирования / В.А. Камаев, В.В. Костерин. – М.: Высшая школа, 2006. – 454 с.
 17. Канторович Л. В. Математические методы организации и планирования производства: репринтное издание. Санкт-Петербург гос. ун-т.: изд. дом Санкт-Петербург гос. ун-та, 2012 С 7-13.
 18. Колобов А.В., Варфоломеев И.А. Повышение эффективности бизнес-системы предприятия на основе применения цифровых инструментов в металлургии.: журнал «Сталь», н.10, 2020. 69-73с.
 19. Люгер Дж. Ф. Искусственный интеллект. Стратегии и методы решения сложных проблем. – 4-е изд. – М.: Вильямс, 2005. – 864 с.
 20. Мазур И.П. Проблемы контроля качества поверхности при производстве листового проката.//[Сталь](#). 2011. № 4. С. 31.

21. Мазур И.П. Развитие теории и совершенствование технологии производства листового проката на литейно-прокатных комплексах : дис. д.т.н./ Липецк: ЛГТУ, 2003. – 316 с.
22. Мак-Дональд М. WPF – Windows Presentation Foundation в .NET 4.5 с примерами на C# 5.0 для профессионалов / М. Мак-Дональд. – ИД Вильямс, 2013. – 1024 с.
23. Маляренко И. Планирование и оптимизация // Корпоративные системы. – 2006. – № 27. – С. 29-32.
24. Математический энциклопедический словарь / Гл. ред. Прохоров Ю. В.. — М.: Сов. энциклопедия, 1988. — 847 с.
25. Назаров, С.В. Архитектура и проектирование программных систем: Монография. / С.В. Назаров. - М.: ИНФРА-М, 2013. - 351 с.
26. Новокщенова С.М., Виноградов М.И Дефекты стали. Справ. изд. М.: Металлургия, 1984. 199с.
27. Орлов С.А. Технологии разработки программного обеспечения / С. Орлов. – СПб.: Питер, 2002. – 464 с.
28. Павосудович В.В., Сокуренок В.П., Данчен «Дефекты стальных слитков и проката. Справочник» Интермет Инжиниринг, 2006г. – 384 с.
29. Пероцкая В.Н., Градусов Д.А. Основы тестирования программного обеспечения: Учебное пособие – Владимирский государственный университет, 2017. – 100 с.
30. Потехин Б.А. Металловедение. Екатеринбург, 2019 г., 70-75 с.
31. Пушников А.Ю. Введение в системы управления базами данных. Часть 1. Реляционная модель данных: Учебное пособие/Изд-е Башкирского ун-та. – Уфа, 1999. – 108 с.
32. Р. Л. Шаталов, А. Л. Генкин. Автоматизация процесса горячей прокатки плоского металла. - М.: Изд-во МГОУ, 2009. - 256 с.
33. Рассел, Дж., Жизненный цикл программного обеспечения / Джесси Рассел. – М.: Книга по Требованию, 2012. – 89 с.

34. Рассел, Дж., Интерфейс программирования приложений / Джесси Рассел. – М.: Книга по Требованию, 2012. – 574 с.
35. Самарин В.И., Игнатенко А.М., Макарова И.Л., Якунина Н.Ф. Технология раскрытия темы "симплекс - метод решения задачи линейного программирования // Вопросы гуманитарных наук. - 2016. - №2 (83). - С. 118-126.
36. Суворов И.К. Обработка металлов давлением: Учебник для вузов.-3-е изд. М.: Высш. школа, 1980 – 364 с.
37. Суворова Я.А. Бережливое производство как эффективный метод оптимизации производства // Международный молодежный симпозиум по управлению, экономике и финансам. Сборник научных статей. Том 1.. - Казань: Издательство Казанского университета, 2019. - С. 266-268.
38. Тидвелл Дж. Разработка пользовательских интерфейсов; Питер. Санкт-Петербург, 2011. 480 с.
39. Фомин Г.П. Математические методы и модели в коммерческой деятельности, Финансы и статистика, 2008 г. – 616 с.
40. Carson J.S. Model verification and validation // Proceedings of the 2002 Winter Simulation Conference. – 2002. – pp. 52–58.
41. Lukas Mathis. A/B Testing // Designed for Use. — Pragmatic Bookshelf, 2011. — 344 p.
42. C# [Электронный ресурс]. URL: <https://habr.com/ru/post/56694/> Дата доступа 15.02.2023.
43. Severstal [Электронный ресурс]. URL: <https://chermk.severstal.com/about/business-system/> Дата доступа 10.05.2023
44. Wikipedia [Электронный ресурс] — Режим доступа: URL: https://ru.wikipedia.org/wiki/Диаграмма_последовательности. Дата доступа: 12.05.2023.
45. Wikipedia [Электронный ресурс] — Режим доступа: URL: https://ru.wikipedia.org/wiki/Объектно-ориентированное_программирование. Дата доступа 25.05.2023.

46. Wikipedia [Электронный ресурс] – Режим доступа: URL: https://ru.wikipedia.org/wiki/Прокатный_стан. Дата доступа 25.05.2023
47. Wikipedia [Электронный ресурс] – Режим доступа: URL: https://ru.wikipedia.org/wiki/Программное_обеспечение. Дата доступа 25.05.2023
48. Wikipedia [Электронный ресурс] – Режим доступа: URL: <https://ru.wikipedia.org/wiki/Симплекс-метод>. Дата доступа 25.05.2023
49. В. А. Пепеляев, Ю. М. Чёрный О современных подходах к оценке достоверности имитационных моделей. Электронная статья // <http://www.gpss.ru/immod%2703/028.html> Дата доступа: 21.05.2023
50. ЛПЦ №3 [Электронный ресурс] – Режим доступа: URL: <https://www.metalinfo.ru/ru/news/85919/> / Дата доступа: 09.10.2022

УСТАВ ПРОЕКТА

Разработка программного обеспечения планирования прокатки и нагрева
слябов на стане 5000

Общие положения

Данный документ является соглашением между основными участниками проекта, определяющим цели проекта, рамки проекта, обязанности участников проекта, основные этапы выполнения работ по проекту.

Соблюдение положения устава является обязательным для всех участников проекта.

Данный документ может меняться в ходе выполнения проекта. Внесение изменений и дополнений в устав проекта инициируется руководителем проекта и утверждается заказчиком (решением управляющего комитета).

Настоящий устав действует, начиная с момента подписания договора на проектные работы и до окончания срока реализации проекта (до подписания акта сдачи-приемки всех работ по проекту).

Требования, удовлетворяющие потребности, пожелания и ожидания
заказчика и других участников проекта

Программное обеспечения планирования прокатки и нагрева слэбов на
стане 5000 должно выполнять следующие задачи:

- формировать прокатные группы в соответствии с технологическими требованиями;
- при формировании посадки слэбов в печь необходимо учитывать все технологические нюансы производства по отношению к камерным и методической печам;

- суммарное полученное время прокатки должно быть меньше текущих результатов при одинаковых планах заказов;
- возможность просмотра полученных результатов в понятной для оператора форме.

Производственная необходимость, самое общее описание проекта или требования к продукту, который является предметом проекта

На данный момент на предприятии ПАО «Северсталь» существует проблема оптимизации работы стана 5000. Необходимо оптимально планировать процессы прокатки и нагрева слэбов на стане, чтобы суммарное затраченное время на прокатку и нагрева слэбов было минимальным. Максимально эффективно это можно сделать с помощью методов и алгоритмов оптимизации в отличие от ручного планирования, которым пользуются на данный момент. Поэтому необходимо разработать соответствующий программный продукт.

Практическая значимость разработанного программного обеспечения заключается в возможности применения полученного в ходе исследования программного продукта, который может с помощью применения математических моделей формировать прокатные группы с учетом всех требований, а также формировать посад слэбов в печь согласно заданным параметрам. Итоговый результат пользователь может увидеть как в отрисованном, так и табличном варианте.

Описанные математические модели и алгоритм позволяют планировать прокатку и нагрев слэбов на стане 5000 в результате чего получается суммарное затраченное время на выполнение заказа меньше, чем при ручном планировании специалистами.

Требования к продукту, который является предметом проекта

Требования к функциональным характеристикам:

- удобный пользовательский интерфейс;
- вывод на экран сформированных прокатных групп;
- вывод на экран сформированного посада слябов в печь;
- хранение результатов работы в Excel-файлы;
- возможность принимать введенные технологические параметры;
- сохранять результаты работы в виде файлов в формате .png.

Входные данные:

- сформированный Excel файл с планом заказов;

Выходные данные:

- полученный в результате работы алгоритма Excel файл;
- скриншоты сформированных прокатных групп и посада слябов в печь.

Цель проекта

Целью работы является повышение эффективности работы стана 5000, состоящее из уменьшения суммарного времени прокатки запланированных прокатных партий за счет оптимизации рабочего процесса.

Для достижения поставленной цели необходимо решить следующие задачи:

- Выполнить анализ предметной области в рамках проводимых исследовательских работ.
- Выполнить теоретическое обоснование решаемой задачи, описание методов и моделей, разработан алгоритм планирования прокатки и нагрева слябов.

- Выполнить проектирование и реализация информационного и программного обеспечения планирования прокатки и нагрева слябов на стане 5000.
- Реализовать экспериментальную проверку программного обеспечения планирования прокатки и нагрева слябов на стане 5000.

Расписание контрольных событий представлено в табл. П1.1.

Таблица П1.1

Расписание контрольных событий

Этап	Срок	Ответственный
1 этап: Характеристика проблемы планирования прокатки и нагрева слябов на стане	октябрь – декабрь 2021	Кустов М.А.
2 этап: Разработка математического и алгоритмического обеспечения планирования прокатки и нагрева слябов на стане 5000	ноябрь – май 2022	Кустов М.А.
3 этап: Проектирование и реализация информационного и программного обеспечения планирования прокатки и нагрева слябов на стане 5000	июнь 2022 – февраль 2023	Кустов М.А.
4 этап: Экспериментальная проверка программного обеспечения планирования прокатки и нагрева слябов на стане 5000	март – май 2022	Кустов М.А.

Информация об участниках проекта

Информация о заказчике проекта представлена в табл. П1.2, информация о руководителе проекта представлена в табл. П1.3, информация о разработчике проекта представлена в табл. П1.4.

Таблица П1.2

Заказчик проекта

Организация:	Публичное акционерное общество «Северсталь»
Полномочия:	• формирование общих требований к результату проекта; • назначение руководителя проекта; • контроль хода выполнения работ по проекту; • приемка результатов проекта.
Ответственность:	• за своевременное обеспечение проекта ресурсами.

Таблица П1.3

Руководитель проекта

Организация:	ФГБОУ ВПО «Череповецкий государственный университет»
ФИО, должность	Ершов Евгений Валентинович, заведующий кафедрой МПО ЭВМ
Контактная информация	Тел: +7(8202) 51-90-69
Полномочия:	• формирование команды проекта; • распределения ресурсов по этапам проекта; • планирование, организация и контроль выполнения работ по достижению целей проекта; • оценка результативности и разработка отчетных документов по проекту.

Продолжение табл. П1.3

Ответственность:	качественное и своевременно выполнение работ по проекту, достижение запланированных результатов.
------------------	--

Таблица П1.4

Разработчик проекта

Организация:	ФГБОУ ВПО «Череповецкий государственный университет»
ФИО, должность	Кустов Максим Александрович, студент
Контактная информация	Тел: +7(8202) 51-90-69
Полномочия:	- разработка, проектирование ПО согласно требованиям; - создание дорожной карты для планирования и организации выполнения работ по достижению целей; - коммуникация с заказчиком, получение обратной связи по реализации проекта; - распределение ресурсов по стадиям проекта.
Ответственность:	качественное и своевременно выполнение работ по проекту, достижение запланированных результатов.

Функциональные организации и их участие

Проект реализуется на базе ФГБОУ ВО «Череповецкий государственный университет».

Заказчиком проекта является ПАО «Северсталь».

Стратегия «Северстали» заключается в повышении финансовой эффективности, создании максимальной добавленной стоимости и увеличении вознаграждения акционеров в сочетании с заботой о сотрудниках компании и минимизацией негативного воздействия на окружающую среду.

Реальная бизнес-ситуация, служащая обоснованием проекта с данными о прибыли на инвестиции

Проект не является коммерческим, доходы по проекту в период его реализации не предусмотрены.

Бюджет проекта

Проект реализуется в рамках освоения магистерской программы «Разработка программно-информационных систем» по направлению 09.04.04 Программная инженерия.

Разработка программного обеспечения ведется на ноутбуке HP Elitebook 14 (стоимость – 95000 руб.).

Разработка производилась в среде Visual Studio Professional 2019 (стоимость лицензии – 2500 руб.).

Трудозатраты на разработку и отладку программы

Расчет экономической эффективности и срока окупаемости проектируемой программы для ЭВМ начинается с расчета трудовых затрат.

Затраты на оплату труда при разработке программного продукта вычисляются по формуле:

$$З_{тр} = (З_{общ} + Отч) * Т_H,$$

где $Z_{\text{общ}}$ – общая зарплата работника за час;

Отч – отчисления с зарплаты, %;

T_H – время написания программы.

Заработная плата программиста за час определяется по следующей формуле:

$$Z_{\text{пр}} = \frac{C_{\text{т пр}}}{\Phi_{\text{вм}}},$$

где $C_{\text{т пр}}$ – ставка программиста;

$\Phi_{\text{вм}}$ – фонд рабочего времени в месяц, ч.

Заработная плата дополнительная определяется по следующей формуле:

$$Z_{\text{доп}} = \frac{Z_{\text{пр}} * N_{\text{доп}}}{100},$$

где $Z_{\text{пр}}$ – заработная плата программиста;

$N_{\text{доп}}$ – норма отчислений на дополнительную зарплату (10%).

Зарплата общая вычисляется по следующей формуле:

$$Z_{\text{общ}} = Z_{\text{пр}} + Z_{\text{доп}},$$

Отчисления на соцстрах, фонд занятости и пенсионный фонд вычисляются по следующей формуле:

$$\text{Отч} = O_{\text{сс}} + O_{\text{фз}} + O_{\text{пф}},$$

где $O_{\text{сс}}$ – отчисления на социальное страхование (0,5% от $Z_{\text{общ}}$);

$O_{\text{фз}}$ – отчисления в фонд занятости (0,5% от $Z_{\text{общ}}$);

$O_{\text{пф}}$ – отчисления в пенсионный фонд (2% от $Z_{\text{общ}}$)

Данные по заработной плате представлены в табл. П1.5.

Таблица П1.5

Данные по заработной плате

Должность	Время работы, мес.	Ст _{пр} , руб.	З _{пр} , руб.	З _{доп} , руб.	З _{общ} , руб.	Отч, руб.	З _{тр} , руб.
Программист	11	54 000	303	30,33	333,33	10,01	594001,4

Затраты на использование машинного времени вычисляются по формуле:

$$Z_{\text{м.вр}} = C_{\text{м.вр}} * \text{Вр}_{\text{в.т}},$$

где $Z_{\text{м.вр}}$ - затраты на использование машинного времени, руб;

$C_{\text{м.вр}}$ - стоимость одного часа машинного времени, руб/ч;

$\text{Вр}_{\text{в.т}}$ - время использования вычислительной техники, ч.

Стоимость одного часа машинного времени рассчитывается по формуле:

$$C_{\text{м.вр}} = \frac{C_{\text{к}}}{C_{\text{сп.к}} * K_{\text{р.д}} * \text{Вр}_{\text{с}}} + \text{Ст}_{\text{э}} * M_{\text{вс}},$$

где $C_{\text{к}}$ - покупная цена компьютера;

$C_{\text{сп.к}}$ - срок службы компьютера, год;

$K_{\text{р.д}}$ - количество рабочих дней в году;

$\text{Вр}_{\text{с}}$ - время работы компьютера в течение суток, ч;

$\text{Ст}_{\text{э}}$ - стоимость одного кВт/ч электроэнергии, руб;

$M_{\text{вс}}$ - мощность вычислительной системы, кВт.

$$C_{\text{м.вр}} = \frac{95\,000}{4 * 250 * 8} + 4,87 * 0,65 = 26,91 \text{ руб/ч}$$

Время использования вычислительной техники ($V_{\text{в.т}}$) рассчитывается по следующей формуле:

$$V_{\text{в.т}} = K_{\text{д.р}} * V_{\text{р.с}},$$

где $K_{\text{д.р}}$ - количество дней разработки ПО;

$V_{\text{р.с}}$ - время работы компьютера в течение суток, ч.

$$V_{\text{в.т}} = 8 * 328 = 2624 \text{ ч.}$$

$$Z_{\text{м.вр}} = 26,91 * 2624 = 70626,27 \text{ руб.}$$

Затраты на носители информации принимаются в размере 2% от цены вычислительной техники.

$$Z_{\text{н.и}} = C_{\text{к}} * 2\%$$

$$Z_{\text{н.и}} = 95\,000 * 2\% = 1900 \text{ руб.}$$

Затраты на текущий и профилактический ремонт принимаются в размере 4% от цены вычислительной техники.

$$Z_{\text{рем}} = C_{\text{к}} * 4\%$$

$$Z_{\text{рем}} = 95\,000 * 4\% = 3800 \text{ руб.}$$

Прочие эксплуатационные расходы включают в себя затраты на освещение, отопление, охрану, уборку и текущий ремонт помещений. Они

принимаются в размере 10% от стоимости помещения (или его аренды), где происходит разработка программного продукта.

$$З_{пр} = C_{мп} * 10\%,$$

$$З_{пр} = 15000 * 10\% = 1500 \text{ руб.}$$

где $C_{мп}$ - стоимость помещения.

Себестоимость разработки определяется по следующей формуле:

$$C_{п.п} = З_{тр} + З_{м.вр} + З_{н.и} + З_{рем} + З_{пр}$$

$$C_{п.п} = 594001,4 + 70626,27 + 1900 + 3800 + 1500 = 671827,7 \text{ руб.}$$

Расчет цены программного продукта

Установление определенной цены на программный продукт служит для последующей его продажи и получения прибыли. Очень важно назначить цену таким образом, чтобы она не оказалась слишком высокой или слишком низкой. Для определения минимальной цены, ниже которой разработчику будет невыгодно продавать программный продукт, используется следующая формула:

$$Ц_{п.п} = C_{п.п} * (1 + Н_{пр}),$$

где $Н_{пр}$ - норматив прибыли (20%).

$$Ц_{п.п} = 671827,7 * (1 + 0,2) = 806193,24 \text{ руб.}$$

Основные риски проекта

Риски в рамках реализуемого проекта представлены в табл. П1.6.

Таблица П1.6

Риски проекта

№ п/п	Риски проекта	Мероприятия по управлению рисками
1	Риски, связанные с выполнением участниками своих обязательств в рамках проекта по причине форс-мажорных обстоятельств (не зависящих от воли участников проекта)	Замена участников проекта. Пересмотр сроков реализации проекта
2	Риски, связанные с недостаточным финансированием проекта	Приостановление проекта / Пересмотр сроков реализации проекта

Текст программы

Текст модуля SaveModel.cs представлен на рис. П2.1.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Xml.Serialization;

namespace OptimizatorStana
{
    [Serializable]
    public class SaveModel
    {
        public SaveModel(int iteration, int shirina, int pauseCB,
            int pauseDirty, int pauseClear, int dopPause, double allTimes,
            double allTimesWithOpt, double timecar, string port, string ids,
            string host)
        {
            Iteration = iteration;
            Shirina = shirina;
            PauseCB = pauseCB;
            PauseDirty = pauseDirty;
            PauseClear = pauseClear;
            DopPause = dopPause;
            AllTimes = allTimes;
            AllTimesWithOpt = allTimesWithOpt;
            TimeToCar = timecar;
            Port = port;
            Ids = ids;
            Host = host;
            //MaxSlabs = mx;
        }

        public SaveModel() { }
        // [XmlElement]
        public int Iteration { get; set; }
        // [XmlElement]
        public int Shirina { get; set; }
        // [XmlElement]
        public int PauseCB { get; set; }
        // [XmlElement]
        public int PauseDirty { get; set; }
        // [XmlElement]
        public int PauseClear { get; set; }
        // [XmlElement]
        public int DopPause { get; set; }
        // [XmlElement]
        public double AllTimes { get; set; }
        // [XmlElement]
        public double AllTimesWithOpt { get; set; }
        // [XmlElement]
        public double TimeToCar { get; set; }
        public string Port { get; set; }
        public string Ids { get; set; }
        public string Host { get; set; }
        // [XmlElement]
        // public int MaxSlabs { get; set; }
    }
}

```

Рис. П2.1. Модуль SaveModel.cs

Текст модуля RollingGroup.cs представлен на рис. П2.2.

```

using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Xml.Serialization;
using CommonClasses.MVVM;

namespace Optimizator
{
    public class RollingGroup
    {
        [XmlIgnore]
        public int pausemezhdz { get; set; }
        [XmlIgnore]
        public int pausedirty { get; set; }
        [XmlIgnore]
    }
}

```

Рис. П2.2. Модуль RollingGroup.cs

П2.2. Продолжение

```

public int pauseclear { get; set; }
[XmlIgnore]
public int doppause { get; set; }

public int Id { get; set; }      public
ObservableCollection<SlyabSim> Slabs { get; set; }
public bool IsOk { get; set; }

public double Result { get; set; }

public double UsefulTimeFirstTwoSlabs
{
    get
    {
        var res = (Slabs[0].RoughtRollingDuration =
Slabs[0].FinishRollingDuration +
        Slabs[1].RoughtRollingDuration +
Slabs[1].FinishRollingDuration).TotalSeconds
        /
        ((Slabs[1].CoolingStartTime -
Slabs[0].RoughtRollingStartTime) +
        (Slabs[1].FinishRollingFinishTime -
Slabs[0].FinishRollingStartTime)).TotalSeconds;
        return res;
    }
}

public double GroupTotalSeconds { get { return
(Slabs.Last().FinishRollingFinishTime -
Slabs.First().RoughtRollingStartTime).TotalSeconds; } }

public RollingGroup()
{
    Slabs = new ObservableCollection<SlyabSim>();
    IsOk = true;
}

/// <summary>
/// Моделирует прокатную группу из слабов и
возвращает ее
/// </summary>
/// <param name="rollingGroup">Полученная
прокатная группа</param>
/// <returns>Смог ли промоделировать всю группу
(если false - группа не исполнима)</returns>
public bool SimulateRollingGroup(out
ObservableCollection<SlyabSim> rollingGroup)
{
    bool isOk = true;
    var rgStartTime = DateTime.MinValue;
    var rgRoughRollingFinishTimeMax =
DateTime.MinValue;

    rollingGroup = new
ObservableCollection<SlyabSim>();
    foreach (var s in Slabs)
    {
        if (rollingGroup.Count == 0)
        {
            s.RoughtRollingStartTime = rgStartTime;
            rollingGroup.Add(s);
            if (s.FinishRollingStartTime ==
DateTime.MinValue) {
                s.FinishRollingStartTime =
s.FinishRollingStartTime.AddSeconds(pausemezhd);
            }
            rgRoughRollingFinishTimeMax =
s.FinishRollingStartTime.Subtract(new TimeSpan(0, 0,
pausemezhd));
        }
        else
        {
            s.RoughtRollingStartTime =
rollingGroup.Last().CoolingStartTime.Add(new TimeSpan(0, 0,
pausedirty));
            if (s.FinishRollingStartTime <=
rollingGroup.Last().FinishRollingFinishTime.Add(new
TimeSpan(0, 0, pauseclear)))
            {
                var d =
rollingGroup.Last().FinishRollingFinishTime.Add(new
TimeSpan(0, 0, pauseclear)) - s.FinishRollingStartTime;
                s.RoughtRollingStartTime =
s.RoughtRollingStartTime.Add(d);
            }

            if (s.CoolingStartTime <=
rgRoughRollingFinishTimeMax)
            {
                rollingGroup.Add(s);
                if (rollingGroup.Count >= 2)
                {
                    rgRoughRollingFinishTimeMax =
rgRoughRollingFinishTimeMax.Subtract(new TimeSpan(0, 0,
doppause));
                }
            }
            else
            {
                isOk = false;
                break;
            }
        }
    }
    IsOk = isOk;
    return isOk;
} }

```

Текст модуля ExcelLoader.cs представлен на рис. П2.3.

```

using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.IO;
using System.Windows.Controls;
using System.Linq;
using System.Windows;
using C1.WPF.Excel;
using System.Reflection;
using System.Data;
using OptimizatorStana;

namespace Optimizator
{
    public static class ExcelLoader
    {
        /// <summary>
        /// Генерирует имя файла для сохранения
        /// </summary>
        public static string GetFileName()
        {
            var saveDir = System.IO.Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location) + @"Results\";
            var path = saveDir + string.Format(@"{0}_{1}\{2}\",
                new object[] { DateTime.Now.Year, DateTime.Now.Month,
                    DateTime.Now.Day });
            if (!Directory.Exists(path))
            {
                Directory.CreateDirectory(path);
            }
            var fileName = path + string.Format("{0}", new
                object[] { DateTime.Now.ToString("HH.mm").Replace(":", ".")
            });
            var counter = 0;
            while (File.Exists(fileName + (counter > 0 ? " (" +
                counter + ")" : string.Empty) + ".xls"))
            {
                counter++;
            }
            return counter > 0 ? fileName + " (" + counter + ").xls"
                : fileName + ".xls";
        }

        public static void ReadDataInDG(string fileName, out
            System.Data.DataTable dataGrid)
        {
            // slyabs = new ObservableCollection<Slyab>();
            var book = new C1XLBook();
            book.Load(fileName, FileFormat.OpenXml);
            var sheet = book.Sheets[0];
            var index = 0;
            int kol = 0;
            var unitName = string.Empty;
            dataGrid = new DataTable();

            for (var i = 0; i < 1000; i++) {
                if (sheet[i, 0].Text == string.Empty) // считываем
                    пока есть записи
                {
                    break;
                }
                kol++;
            }
            for (var i = 0; i < kol - 1; i++)
            {
                dataGrid.Rows.Add();
            }
            for (var i = 0; i < sheet.Columns.Count; i++)
            {
                dataGrid.Columns.Add(Convert.ToString(sheet[index, i].Value),
                    typeof(string));
            }
            index++;
            var index1 = 0;
            while (true)
            {
                if (sheet[index, 0].Text == string.Empty) //
                    считываем пока есть записи
                {
                    break;
                }
                dataGrid.Rows[index1].SetField(0,
                    Convert.ToString(sheet[index, 0].Value));
                dataGrid.Rows[index1].SetField(1,
                    Convert.ToString(sheet[index, 1].Value));
                dataGrid.Rows[index1].SetField(2,
                    Convert.ToString(sheet[index, 2].Value));
                dataGrid.Rows[index1].SetField(3,
                    Convert.ToString(sheet[index, 3].Value));
                dataGrid.Rows[index1].SetField(4,
                    Convert.ToString(sheet[index, 4].Value));
                dataGrid.Rows[index1].SetField(5,
                    Convert.ToString(sheet[index, 5].Value));
                dataGrid.Rows[index1].SetField(6,
                    Convert.ToString(sheet[index, 6].Value));
                dataGrid.Rows[index1].SetField(7,
                    Convert.ToString(sheet[index, 7].Value));
                dataGrid.Rows[index1].SetField(8,
                    Convert.ToString(sheet[index, 8].Value));
                dataGrid.Rows[index1].SetField(9,
                    Convert.ToString(sheet[index, 9].Value));
                dataGrid.Rows[index1].SetField(10,
                    Convert.ToString(sheet[index, 10].Value));
                dataGrid.Rows[index1].SetField(11,
                    Convert.ToString(sheet[index, 11].Value));
                dataGrid.Rows[index1].SetField(12,
                    Convert.ToString(sheet[index, 12].Value));
                dataGrid.Rows[index1].SetField(13,
                    Convert.ToString(sheet[index, 13].Value));
                dataGrid.Rows[index1].SetField(14,
                    Convert.ToString(sheet[index, 14].Value));
            }
        }
    }
}

```

Рис. П2.3. Модуль ExcelLoader.cs

П2.3. Продолжение

```

        dataGrid.Rows[index1].SetField(15,
Convert.ToString(sheet[index, 15].Value));
        dataGrid.Rows[index1].SetField(16,
Convert.ToString(sheet[index, 16].Value));
        dataGrid.Rows[index1].SetField(17,
Convert.ToString(sheet[index, 17].Value));
        dataGrid.Rows[index1].SetField(18,
Convert.ToString(sheet[index, 18].Value));
        dataGrid.Rows[index1].SetField(19,
Convert.ToString(sheet[index, 19].Value));
        dataGrid.Rows[index1].SetField(20,
Convert.ToString(sheet[index, 20].Value));
        dataGrid.Rows[index1].SetField(21,
Convert.ToString(sheet[index, 21].Value));
        dataGrid.Rows[index1].SetField(22,
Convert.ToString(sheet[index, 22].Value));
        dataGrid.Rows[index1].SetField(23,
Convert.ToString(sheet[index, 23].Value));
        dataGrid.Rows[index1].SetField(24,
Convert.ToString(sheet[index, 24].Value));
        dataGrid.Rows[index1].SetField(25,
Convert.ToString(sheet[index, 25].Value));
        dataGrid.Rows[index1].SetField(26,
Convert.ToString(sheet[index, 26].Value));
        dataGrid.Rows[index1].SetField(27,
Convert.ToString(sheet[index, 27].Value));
        dataGrid.Rows[index1].SetField(28,
Convert.ToString(sheet[index, 28].Value));
        dataGrid.Rows[index1].SetField(29,
Convert.ToString(sheet[index, 29].Value));
        dataGrid.Rows[index1].SetField(30,
Convert.ToString(sheet[index, 30].Value));
        dataGrid.Rows[index1].SetField(31,
Convert.ToString(sheet[index, 31].Value));
        dataGrid.Rows[index1].SetField(32,
Convert.ToString(sheet[index, 32].Value));
        dataGrid.Rows[index1].SetField(33,
Convert.ToString(sheet[index, 33].Value));
        dataGrid.Rows[index1].SetField(34,
Convert.ToString(sheet[index, 34].Value));
        dataGrid.Rows[index1].SetField(35,
Convert.ToString(sheet[index, 35].Value));
        dataGrid.Rows[index1].SetField(36,
Convert.ToString(sheet[index, 36].Value));
        dataGrid.Rows[index1].SetField(37,
Convert.ToString(sheet[index, 37].Value));
        dataGrid.Rows[index1].SetField(38,
Convert.ToString(sheet[index, 38].Value));
        dataGrid.Rows[index1].SetField(39,
Convert.ToString(sheet[index, 39].Value));

```

```

        dataGrid.Rows[index1].SetField(40,
Convert.ToString(sheet[index, 40].Value));
        dataGrid.Rows[index1].SetField(41,
Convert.ToString(sheet[index, 41].Value));
        dataGrid.Rows[index1].SetField(42,
Convert.ToString(sheet[index, 42].Value));
        dataGrid.Rows[index1].SetField(43,
Convert.ToString(sheet[index, 43].Value));
        dataGrid.Rows[index1].SetField(44,
Convert.ToString(sheet[index, 44].Value));
        dataGrid.Rows[index1].SetField(45,
Convert.ToString(sheet[index, 45].Value));
        dataGrid.Rows[index1].SetField(46,
Convert.ToString(sheet[index, 46].Value));
        dataGrid.Rows[index1].SetField(47,
Convert.ToString(sheet[index, 47].Value));
        index++;
        index1++;    }    }
    public static void ReadData(string fileName,
                                out ObservableCollection<SlyabSim>
slyabs, int cs)    {
        slyabs = new ObservableCollection<SlyabSim>();
        var book = new C1XLBook();
        book.Load(fileName, FileFormat.OpenXml);
        var sheet = book.Sheets[0];
        var index = 1;
        var unitName = string.Empty;
        if (cs == 1)    {
            while (true)    {
                if (sheet[index, 0].Text == string.Empty) //
считываем пока есть записи    {
                    break;    }
                slyabs.Add(new SlyabSim()    {
                    ID = Convert.ToInt32(sheet[index, 0].Value)
                    RoughtRollingDuration = new TimeSpan(0,
0, Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
11].Value), 0))),
                    CoolingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
12].Value), 0))),
                    FinishRollingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
13].Value), 0))),
                    Count = Convert.ToInt32(sheet[index, 14].Value),
                }); ;
                index++;    }    }
        else if (cs == 2)    {
            while (true)    {
                if (sheet[index, 0].Text == string.Empty) //
считываем пока есть записи    {
                    break;    }
                slyabs.Add(new SlyabSim()    {

```

П2.3. Продолжение

```

        Marka = Convert.ToString(sheet[index, 0].Value),
        NumberRolling =
Convert.ToInt32(sheet[index, 1].Value),
        H = Convert.ToDouble(sheet[index, 2].Value),
        W = Convert.ToDouble(sheet[index, 3].Value),
        L = Convert.ToDouble(sheet[index, 4].Value),
        H_Slyab = Convert.ToInt32(sheet[index, 5].Value),
        W_Slyab = Convert.ToInt32(sheet[index, 6].Value),
        L_Slyab = Convert.ToInt32(sheet[index, 7].Value),
        H_podst = Convert.ToString(sheet[index, 8].Value),
        Tnachprok = Convert.ToString(sheet[index, 9].Value),
        T_nchp = Convert.ToString(sheet[index, 10].Value),
        T_kp = Convert.ToString(sheet[index, 11].Value),
        RoughtRollingDuration = new TimeSpan(0,
0, Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
12].Value), 0))),
        CoolingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
14].Value), 0))),
        FinishRollingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
13].Value), 0))),
        KP1 = Convert.ToDouble(sheet[index, 15].Value) / 60,
        KP1Max = Convert.ToDouble(sheet[index, 16].Value) / 60,
        KP2 = Convert.ToDouble(sheet[index, 17].Value) / 60,
        KP2Max = Convert.ToDouble(sheet[index,
18].Value) / 60,
        KP3 = Convert.ToDouble(sheet[index,
19].Value) / 60,
        KP3Max = Convert.ToDouble(sheet[index,
20].Value) / 60,
        KP4 = Convert.ToDouble(sheet[index,
21].Value) / 60,
        KP4Max = Convert.ToDouble(sheet[index, 22].Value) / 60,
        KP5 = Convert.ToDouble(sheet[index, 23].Value) / 60,
        KP5Max = Convert.ToDouble(sheet[index, 24].Value) / 60,
        KP6 = Convert.ToDouble(sheet[index, 25].Value) / 60,
        KP6Max = Convert.ToDouble(sheet[index, 26].Value) / 60,
        MP = Convert.ToDouble(sheet[index, 27].Value) / 60,
        MPMax = Convert.ToDouble(sheet[index, 28].Value) / 60,
        WeightSl = Math.Round(Convert.ToDouble(sheet[index,
29].Value), 1),
        TimeCoolingPrevUKO = Convert.ToDouble(sheet[index,
30].Value),
        StrategyProk = Convert.ToString(sheet[index, 31].Value),
        UKOorUTU = Convert.ToString(sheet[index, 32].Value),
        TimeCoolingInUTU = Convert.ToDouble(sheet[index,
33].Value),
        PlaceCooling = Convert.ToString(sheet[index, 34].Value),
        TimeStorageZRU = Convert.ToDouble(sheet[index, 35].Value),
        TaskPrav = Convert.ToString(sheet[index, 36].Value),

```

```

        CountPass = Convert.ToInt32(sheet[index, 37].Value),
        TnachCoolingUPO = Convert.ToString(sheet[index,
38].Value)TkonCoolingUPO = Convert.ToString(sheet[index,
39].Value),
        SpeedTrans = Convert.ToString(sheet[index, 40].Value),
        RashodWater = Convert.ToString(sheet[index, 41].Value),
        TnachCoolingUKO = Convert.ToString(sheet[index,
42].Value),
        TkonCoolingUKO = Convert.ToString(sheet[index,
43].Value),
        VCoolingUKOms = Convert.ToString(sheet[index,
44].Value),
        VCoolingUKOgrads = Convert.ToString(sheet[index,
45].Value),
        Numberthing = Convert.ToString(sheet[index, 46].Value),
        Tehnumber = Convert.ToString(sheet[index, 47].Value),
        }); ; index++; } }
        else if (cs == 3) {
            while (true) {
                if (sheet[index, 0].Text == string.Empty) //
считываем пока есть записи { break; }
                slyabs.Add(new SlyabSim) {
                    ID = Convert.ToInt32(sheet[index, 1].Value),
                    W = Convert.ToDouble(sheet[index, 3].Value),
                    L_Slyab = Convert.ToInt32(sheet[index, 7].Value),
                    RoughtRollingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
12].Value), 0))),
                    CoolingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
14].Value), 0))),
                    FinishRollingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
13].Value), 0))),
                }); ;
                index++; } } }
        public static void
SaveResults(ObservableCollection<SlyabSim> slyabSims,
ObservableCollection<SlyabSim> slabs,
ObservableCollection<RollingGroup> rollingGroups, string
fileName) {
            try {
                var book = new C1XLBook();
                book.Sheets.Clear();
                #region Вывод результатов по группам слябов
                var sheet = book.Sheets.Add();
                sheet.Name = "Результат";
                #region Создаем шапку
                var cell = 0;
                sheet[0, cell].Value = cell + 1;
                sheet[1, cell++].Value = "Id";
                sheet[0, cell].Value = cell + 1;

```

П2.3. Продолжение

```

sheet[1, cell++].Value = "Длительность черновой";
    sheet[0, cell].Value = cell + 1;
sheet[1, cell++].Value = "Длительность подстуживания";
    sheet[0, cell].Value = cell + 1;
sheet[1, cell++].Value = "Длительность чистовой";
    sheet[0, cell].Value = cell + 1;
    sheet[1, cell++].Value = "Количество план";
    sheet[0, cell].Value = cell + 1;
    sheet[1, cell++].Value = "Количество факт";
#endregion
#region Сохраняем данные
var row = 2;
foreach (var s in slabs)      {
    double cnt1 = 0;
    foreach (var rg in rollingGroups)      {
        cnt1 += (rg.Slabs.Count(ss => ss.ID == s.ID) * rg.Result);
    }
    sheet[row, 0].Value = s.ID;
        sheet[row, 1].Value =
s.RoughtRollingDuration.ToString();
        sheet[row, 2].Value =
s.CoolingDuration.ToString();
        sheet[row, 3].Value =
s.FinishRollingDuration.ToString();
        sheet[row, 4].Value = s.Count;
        sheet[row, 5].Value = cnt1;
        row++;
    }
#endregion
#region Прокатные партии
sheet = book.Sheets.Add();
sheet.Name = "Прокатные партии";
#region Создаем шапку
cell = 0;
sheet[0, cell++].Value = "Марка";
sheet[0, cell++].Value = "Номер проката";
sheet[0, cell++].Value = "Id";
sheet[0, cell++].Value = "Н листа";
sheet[0, cell++].Value = "W листа";
sheet[0, cell++].Value = "L листа";
sheet[0, cell++].Value = "Н, Сляб";
sheet[0, cell++].Value = "W, Сляб";
sheet[0, cell++].Value = "L, Сляб";
sheet[0, cell++].Value = "Н подст";
sheet[0, cell++].Value = "Т нчп";
sheet[0, cell++].Value = "Т кп";
sheet[0, cell++].Value = "Длит. черн.";
sheet[0, cell++].Value = "Длит. подст.";
sheet[0, cell++].Value = "Длит. чист.";
sheet[0, cell++].Value = "Начало черн.";
sheet[0, cell++].Value = "Начало подст.";
sheet[0, cell++].Value = "Начало чист.";
sheet[0, cell++].Value = "Конец чист.";

sheet[0, cell++].Value = "КП1";
sheet[0, cell++].Value = "КП1Max";
sheet[0, cell++].Value = "КП2";
sheet[0, cell++].Value = "КП2Max";
sheet[0, cell++].Value = "КП3";
sheet[0, cell++].Value = "КП3Max";
sheet[0, cell++].Value = "КП4";
sheet[0, cell++].Value = "КП4Max";
sheet[0, cell++].Value = "КП5";
sheet[0, cell++].Value = "КП5Max";
sheet[0, cell++].Value = "КП6";
sheet[0, cell++].Value = "КП6Max";
sheet[0, cell++].Value = "МП";
sheet[0, cell++].Value = "МПMax";
sheet[0, cell++].Value = "Вес";
sheet[0, cell++].Value = "Номер группы";
sheet[0, cell++].Value = "Номер сляба в группе";
sheet[0, cell++].Value = "Время черновой паузы";
sheet[0, cell++].Value = "Время чистовой паузы";
sheet[0, cell++].Value = "Время прокатки группы";
sheet[0, cell++].Value = "Время черновой прокатки";
sheet[0, cell++].Value = "Печь";
sheet[0, cell++].Value = "Тип сляба для МП";
sheet[0, cell++].Value = "Время посады в печь";
sheet[0, cell++].Value = "Номер тележки для МП";
sheet[0, cell++].Value = "Номер печи для КП";
sheet[0, cell++].Value = "Дата/Время посады в печь";
sheet[0, cell++].Value = "Время забора из печи";
sheet[0, cell++].Value = "Время прихода на стан";
sheet[0, cell++].Value = "Стратегия прокатки";
sheet[0, cell++].Value = "УКО/УТУ";
sheet[0, cell++].Value = "Время охлаждения в УТУ";
sheet[0, cell++].Value = "Место охлаждения";
sheet[0, cell++].Value = "Время хранения под ЗРУ";
sheet[0, cell++].Value = "Задание на правку";
sheet[0, cell++].Value = "Количество проходов УПО";
sheet[0, cell++].Value = "Тнач охл УПО";
sheet[0, cell++].Value = "Ткон охл УПО";
sheet[0, cell++].Value = "V охл УКО (м/с)";
sheet[0, cell++].Value = "V охл УКО (град/с)";
sheet[0, cell++].Value = "Номер штуки";
sheet[0, cell++].Value = "Технический номер штуки";
#endregion
#region Сохраняем данные
row = 0;
int k = 0;
#endregion
foreach (var rgg in slyabSims)      {
    row++;
    k++;
    sheet[row, 0].Value = rgg.Marka.ToString();
    sheet[row, 1].Value = rgg.NumberRolling;

```

П2.3. Продолжение

```

sheet[row, 2].Value = rgg.ID;
sheet[row, 3].Value = rgg.H;
sheet[row, 4].Value = rgg.W;
sheet[row, 5].Value = rgg.L;
sheet[row, 6].Value = rgg.H_Slyab;
sheet[row, 7].Value = rgg.W_Slyab;
sheet[row, 8].Value = rgg.L_Slyab;
sheet[row, 9].Value = rgg.H_podst.ToString();
sheet[row, 10].Value = rgg.T_nchp.ToString();
sheet[row, 11].Value = rgg.T_kp.ToString();
sheet[row, 12].Value =
rgg.RoughtRollingDuration.ToString();
sheet[row, 13].Value =
rgg.CoolingDuration.ToString();
sheet[row, 14].Value =
rgg.FinishRollingDuration.ToString();
sheet[row, 15].Value =
rgg.RoughtRollingStartTime.ToLongTimeString();
sheet[row, 16].Value =
rgg.CoolingStartTime.ToLongTimeString();
sheet[row, 17].Value =
rgg.FinishRollingStartTime.ToLongTimeString();
sheet[row, 18].Value =
rgg.FinishRollingFinishTime.ToLongTimeString();
sheet[row, 19].Value = rgg.KP1;
sheet[row, 20].Value = rgg.KP1Max;
sheet[row, 21].Value = rgg.KP2;
sheet[row, 22].Value = rgg.KP2Max;
sheet[row, 23].Value = rgg.KP3;
sheet[row, 24].Value = rgg.KP3Max;
sheet[row, 25].Value = rgg.KP4;
sheet[row, 26].Value = rgg.KP4Max;
sheet[row, 27].Value = rgg.KP5;
sheet[row, 28].Value = rgg.KP5Max;
sheet[row, 29].Value = rgg.KP6;
sheet[row, 30].Value = rgg.KP6Max;
sheet[row, 31].Value = rgg.MP;
sheet[row, 32].Value = rgg.MPMax;
sheet[row, 33].Value = rgg.WeightSl;
sheet[row, 34].Value = rgg.GroupNum;
sheet[row, 35].Value = rgg.SlabNum;
sheet[row, 36].Value = rgg.Pause;
sheet[row, 37].Value = rgg.PauseClear;
sheet[row, 38].Value = rgg.GroupTotal;
sheet[row, 39].Value = rgg.DirtyTime;
sheet[row, 40].Value = rgg.Stove.ToString();
sheet[row, 41].Value = rgg.Type;
sheet[row, 42].Value = rgg.TimeToPech;
sheet[row, 43].Value = rgg.NumCar;
sheet[row, 44].Value = rgg.NumForKP;
sheet[row, 45].Value = rgg.DaTime.ToString();

sheet[row, 46].Value =
rgg.TimeFromStove.ToString();
sheet[row, 47].Value =
rgg.TimeToStan.ToString();
sheet[row, 48].Value = rgg.StrategyProk;
sheet[row, 49].Value = rgg.UKOorUTU;
sheet[row, 50].Value =
rgg.TimeCoolingInUTU;
sheet[row, 51].Value = rgg.PlaceCooling;
sheet[row, 52].Value = rgg.TimeStorageZRU;
sheet[row, 53].Value = rgg.TaskPrav;
sheet[row, 54].Value = rgg.CountPass;
sheet[row, 55].Value = rgg.TnachCoolingUPO;
sheet[row, 56].Value = rgg.TkonCoolingUPO;
sheet[row, 57].Value = rgg.VCoolingUKOms;
sheet[row, 58].Value = rgg.VCoolingUKOgrads;
sheet[row, 59].Value = rgg.Numberthing;
sheet[row, 60].Value = rgg.Tehnumber;
} book.Save(fileName, FileFormat.OpenXml);
} catch (Exception exception) {
throw new Exception("Ошибка сохранения
данных в файл Excel: " + exception.Message, exception); } }

public static void
SaveRes(ObservableCollection<Slyab> slyabs, string fileName,
string tm, double dtna, int cid, int count, double kef, int iter, int gen)
{ try {
var book = new C1XLBook();
book.Sheets.Clear();
var cell = 0;
#region Прокатные партии
var sheet = book.Sheets.Add();
sheet.Name = "Прокатные партии";
cell = 0;
sheet[0, cell++].Value = "Марка";
sheet[0, cell++].Value = "Номер проката";
sheet[0, cell++].Value = "Лист, Н";
sheet[0, cell++].Value = "Лист, W";
sheet[0, cell++].Value = "Лист, L";
sheet[0, cell++].Value = "Сляб, Н";
sheet[0, cell++].Value = "Сляб, W";
sheet[0, cell++].Value = "Сляб, L";
sheet[0, cell++].Value = "Длит. черн.";
sheet[0, cell++].Value = "Длит. чист.";
sheet[0, cell++].Value = "Длит. подстуж.";
sheet[0, cell++].Value = "Время прокатки группы";
sheet[0, cell++].Value = "Время черновой паузы";
sheet[0, cell++].Value = "Время чистовой паузы";
sheet[0, cell++].Value = "Оставшееся время";
sheet[0, cell++].Value = "GroupNum";
sheet[0, cell++].Value = "SlyabNum";
sheet[0, cell++].Value = "КП";

```

П2.3. Продолжение

```

sheet[0, cell++].Value = "КП2";
sheet[0, cell++].Value = "КП3";
sheet[0, cell++].Value = "КП4";
sheet[0, cell++].Value = "КП5";
sheet[0, cell++].Value = "КП6";
sheet[0, cell++].Value = "МП";
sheet[0, cell++].Value = "Время черновой прокатки";
sheet[0, cell++].Value = "Вес сляба";
sheet[0, cell++].Value = "Номер штуки";
sheet[0, cell++].Value = "УКО";
sheet[0, cell++].Value = "УТУ";
sheet[0, cell++].Value = "Разм УКО";
sheet[0, cell++].Value = "Разм УТУ";
sheet[0, cell++].Value = "Время в УТУ";
#region Сохраняем данные
int row = 0;
#endregion
foreach (var rgg in slyabs) {
    row++;
    sheet[row, 0].Value = rgg.Marka.ToString();
    sheet[row, 1].Value = rgg.NumberProk;
    sheet[row, 2].Value = rgg.H;
    sheet[row, 3].Value = rgg.W;
    sheet[row, 4].Value = rgg.L;
    sheet[row, 5].Value = rgg.HS;
    sheet[row, 6].Value = rgg.WS;
    sheet[row, 7].Value = rgg.LS;
    sheet[row, 8].Value = rgg.T_chemovoi;
    sheet[row, 9].Value = rgg.T_Chistovoi;
    sheet[row, 10].Value = rgg.T_podst;
    sheet[row, 11].Value = rgg.TimeProkatGroup;
    sheet[row, 12].Value = rgg.ChemPause;
    sheet[row, 13].Value = rgg.ChistPause;
    sheet[row, 14].Value = rgg.Raznost;
    sheet[row, 15].Value = rgg.GroupNumBest;
    sheet[row, 16].Value = rgg.SlabNumBest;
    sheet[row, 17].Value = rgg.KP1;
    sheet[row, 18].Value = rgg.KP2;
    sheet[row, 19].Value = rgg.KP3;
    sheet[row, 20].Value = rgg.KP4;
    sheet[row, 21].Value = rgg.KP5;
    sheet[row, 22].Value = rgg.KP6;
    sheet[row, 23].Value = rgg.MP;
    sheet[row, 24].Value = rgg.TimeChProkat;
    sheet[row, 25].Value = rgg.Weight;
    sheet[row, 26].Value = rgg.Numberthing;
    sheet[row, 27].Value = rgg.prioritetUKO;
    sheet[row, 28].Value = rgg.prioritetUTU;
    sheet[row, 29].Value = rgg.razmInUKObest;
    sheet[row, 30].Value = rgg.razmInUTUbest;
    sheet[row, 31].Value = rgg.TimeCoolingInUTU;
}

} book.Save(fileName, FileFormat.OpenXml);
} catch (Exception exception) {
    throw new Exception("Ошибка сохранения
данных в файл Excel: " + exception.Message, exception); } }

public static void
SaveResSim(ObservableCollection<SlyabSim> slyabs, string
fileName, string tm, double dtna, int cid, int count, double kef, int
iter, int gen) {
    try {
        var book = new C1XLBook();
        book.Sheets.Clear();
        var cell = 0;
        #region Прокатные партии
        var sheet = book.Sheets.Add();
        sheet.Name = "Прокатные партии";
        cell = 0;
        sheet[0, cell++].Value = "Марка";
        sheet[0, cell++].Value = "Номер проката";
        sheet[0, cell++].Value = "Лист, Н";
        sheet[0, cell++].Value = "Лист, W";
        sheet[0, cell++].Value = "Лист, L";

        sheet[0, cell++].Value = "Сляб, Н";
        sheet[0, cell++].Value = "Сляб, W";
        sheet[0, cell++].Value = "Сляб, L";
        sheet[0, cell++].Value = "Длит. черн.";
        sheet[0, cell++].Value = "Длит. чист.";
        sheet[0, cell++].Value = "Длит. подстуж.";
        sheet[0, cell++].Value = "Время прокатки группы";
        sheet[0, cell++].Value = "Время черновой паузы";
        sheet[0, cell++].Value = "Время чистовой паузы";
        sheet[0, cell++].Value = "GroupNum";
        sheet[0, cell++].Value = "SlyabNum";
        sheet[0, cell++].Value = "КП1";
        sheet[0, cell++].Value = "КП2";
        sheet[0, cell++].Value = "КП3";
        sheet[0, cell++].Value = "КП4";
        sheet[0, cell++].Value = "КП5";
        sheet[0, cell++].Value = "КП6";
        sheet[0, cell++].Value = "МП";
        sheet[0, cell++].Value = "Время черновой прокатки";
        sheet[0, cell++].Value = "Вес сляба";
        sheet[0, cell++].Value = "Номер штуки";
        #region Сохраняем данные
        int row = 0;
        #endregion
        foreach (var rgg in slyabs) {
            row++;
            sheet[row, 0].Value = rgg.Marka.ToString();
            sheet[row, 1].Value = rgg.NumberRolling;
            sheet[row, 2].Value = rgg.H;

```

П2.3. Продолжение

```

        sheet[row, 3].Value = rgg.W;
        sheet[row, 4].Value = rgg.L;
        sheet[row, 5].Value = rgg.H_Slyab;
        sheet[row, 6].Value = rgg.W_Slyab;
        sheet[row, 7].Value = rgg.L_Slyab;
        sheet[row, 8].Value =
rgg.RoughtRollingDuration.ToString();
        sheet[row, 9].Value =
rgg.FinishRollingDuration.ToString();
        sheet[row, 10].Value =
rgg.CoolingDuration.ToString();
        sheet[row, 11].Value = rgg.GroupTotal;
        sheet[row, 12].Value = rgg.Pause;
        sheet[row, 13].Value = rgg.PauseClear;
        sheet[row, 14].Value = rgg.GroupNum;
        sheet[row, 15].Value = rgg.SlabNum;
        sheet[row, 16].Value = rgg.KP1;
        sheet[row, 17].Value = rgg.KP2;
        sheet[row, 18].Value = rgg.KP3;
        sheet[row, 19].Value = rgg.KP4;
        sheet[row, 20].Value = rgg.KP5;
        sheet[row, 21].Value = rgg.KP6;
        sheet[row, 22].Value = rgg.MP;
        sheet[row, 23].Value = rgg.DirtyTime;
        sheet[row, 24].Value = rgg.WeightSl;
        sheet[row, 25].Value = rgg.Numberthing;    }
        #endregion
        book.Save(fileName, FileFormat.OpenXml);    }
        catch (Exception exception)    {
            throw new Exception("Ошибка сохранения
данных в файл Excel: " + exception.Message, exception);    }    }
        public static void ReadDIDG(string fileName, out
System.Data.DataTable dataGrid)    {
            var book = new C1XLBook();
            book.Load(fileName, FileFormat.OpenXml);
            var sheet = book.Sheets[0];
            var index = 0;
            var unitName = string.Empty;
            dataGrid = new DataTable();
            for (var i = 0; i < sheet.Rows.Count - 1; i++)    {
                dataGrid.Rows.Add();    }
            for (var i = 0; i < sheet.Columns.Count; i++)    {
                dataGrid.Columns.Add(Convert.ToString(sheet[index,
i].Value),
typeof(string));    }
            index++;
            var index1 = 0;
            while (true)    {
                if (sheet[index, 0].Text == string.Empty) //
считываем пока есть записи    {
                    break;    }
                dataGrid.Rows[index1].SetField(0,
Convert.ToString(sheet[index, 0].Value));
                dataGrid.Rows[index1].SetField(1,
Convert.ToString(sheet[index, 1].Value));
                dataGrid.Rows[index1].SetField(2,
Convert.ToString(sheet[index, 2].Value));
                dataGrid.Rows[index1].SetField(3,
Convert.ToString(sheet[index, 3].Value));
                dataGrid.Rows[index1].SetField(4,
Convert.ToString(sheet[index, 4].Value));
                dataGrid.Rows[index1].SetField(5,
Convert.ToString(sheet[index, 5].Value));
                dataGrid.Rows[index1].SetField(6,
Convert.ToString(sheet[index, 6].Value));
                dataGrid.Rows[index1].SetField(7,
Convert.ToString(sheet[index, 7].Value));
                dataGrid.Rows[index1].SetField(8,
Convert.ToString(sheet[index, 8].Value));
                dataGrid.Rows[index1].SetField(9,
Convert.ToString(sheet[index, 9].Value));
                dataGrid.Rows[index1].SetField(10,
Convert.ToString(sheet[index, 10].Value));
                index++;
                index1++;    }    }
            public static void ReadDT(string fileName,
out ObservableCollection<SlyabSim>
slyabs, int cs)
            {
                slyabs = new ObservableCollection<SlyabSim>();
                var book = new C1XLBook();
                book.Load(fileName, FileFormat.OpenXml);
                var sheet = book.Sheets[0];
                var index = 1;
                var unitName = string.Empty;
                if (cs == 2) {    while (true)    {
                    if (sheet[index, 0].Text == string.Empty) //
считываем пока есть записи    {    break;    }
                    slyabs.Add(new SlyabSim()    {
                        Marka = Convert.ToString(sheet[index, 0].Value),
                        NumberRolling = Convert.ToInt32(sheet[index,
1].Value),
                        H = Convert.ToDouble(sheet[index, 2].Value),
                        W = Convert.ToDouble(sheet[index, 3].Value),
                        L = Convert.ToDouble(sheet[index, 4].Value),
                        H_Slyab = Convert.ToInt32(sheet[index, 5].Value),
                        W_Slyab = Convert.ToInt32(sheet[index, 6].Value),
                        L_Slyab = Convert.ToInt32(sheet[index, 7].Value),
                        RoughtRollingDuration = new TimeSpan(0,
0, Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
8].Value), 0))),

```

П2.3. Продолжение

```

CoolingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
10].Value), 0))),

FinishRollingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
9].Value), 0))),    }; ;

index++;    }    }

else if (cs == 3)    {    while (true)    {

if (sheet[index, 0].Text == string.Empty) //
считываем пока есть записи    {    break; }

slyabs.Add(new SlyabSim()    {

ID = Convert.ToInt32(sheet[index, 1].Value),

W = Convert.ToDouble(sheet[index, 3].Value),

L_Slyab = Convert.ToInt32(sheet[index, 7].Value),

RoughtRollingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
8].Value), 0))),

CoolingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
10].Value), 0))),

FinishRollingDuration = new TimeSpan(0, 0,
Convert.ToInt32(Math.Round(Convert.ToDouble(sheet[index,
9].Value), 0))),    }); ;

index++;    }    }    }

public static void readrasklad(string fileName, out
ObservableCollection<Slyab> sheets) {

sheets = new ObservableCollection<Slyab>();

var book = new C1XLBook();

book.Load(fileName, FileFormat.OpenXml);

var sheet = book.Sheets[0];

var index = 1;

var unitName = string.Empty;

int cnt = sheet.Rows.Count;

while (true)    {

if (sheet[index, 0].Text == string.Empty) //
считываем пока есть записи    {    break;    }

sheets.Add(new Slyab()    {

Marka = Convert.ToString(sheet[index, 0].Value),

LS = Convert.ToInt32(sheet[index, 1].Value),

WS = Convert.ToInt32(sheet[index, 2].Value),

HS = Convert.ToInt32(sheet[index, 3].Value),

Weight = Convert.ToDouble(sheet[index, 4].Value),

numpp = index,

```

```

NumForKP = Convert.ToInt32(sheet[0, 6].Value    ));

index++;    }    }

public static void ReadForBD(string fileName, out
List<string> technumbers) {

var book = new C1XLBook();

book.Load(fileName, FileFormat.OpenXml);

//var sheet = book.Sheets[0];

technumbers = new List<string>();

try    {

for (var i = 0; i < book.Sheets.Count; i++)    {

var sheet = book.Sheets[i];

int cnt = sheet.Rows.Count;

for (var t = 1; t < cnt; t++)    {

technumbers.Add(Convert.ToString(sheet[t, 9].Value));

}    }    }

catch (Exception ex) {

MessageBox.Show("Ошибка считывания с Excel: " +
ex.Message);    }    }

public static void SaveFromBD(string fileName,
DataTable dt)    {    try    {

var book = new C1XLBook();

book.Sheets.Clear();

var sheet = book.Sheets.Add();

sheet.Name = "Результат";

for (var i = 0; i < dt.Columns.Count; i++)    {

sheet[0, i].Value = dt.Columns[i].ColumnName;    }

for (var i = 0; i < dt.Rows.Count; i++)    {

for (var g = 0; g < dt.Columns.Count; g++)

{

if (dt.Rows[i].ItemArray[g].ToString() == "" ||
dt.Rows[i].ItemArray[g].ToString() == null)    {

sheet[i + 1, g].Value = 0;    }

else    {

sheet[i + 1, g].Value =

dt.Rows[i].ItemArray[g];

}    }    }

book.Save(fileName, FileFormat.OpenXml);

}

catch (Exception ex) {

MessageBox.Show("Ошибка записи в Excel: " +
ex.Message);

}    }    }

```

Текст модуля RecMP.cs представлен на рис. П2.4.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Shapes;

namespace OptimizatorStana
{
    public class RecMP
    {
        public Rectangle Rn1 { get; set; }
        private Canvas Cnv { get; set; }
        public double otY { get; set; }
        public double otX { get; set; }
        public Slyab obs { get; set; }

        public RecMP(Rectangle m1, Canvas cnv, Slyab tmp,
double otX, double oty)
        {
            Rn1 = m1;
            Cnv = cnv;
            obs = tmp;
            otY = oty;
            otX = otX;
            Rn1.MouseLeftButtonDown +=
Rn1_MouseLeftButtonDown;
            Cnv.Children.Add(Rn1);
        }
        DataGrid dtg = null;
        private void Rn1_MouseLeftButtonDown(object
sender, System.Windows.Input.MouseButtonEventArgs e)
        {
            List<Slyab> tmp1 = new List<Slyab>();
            tmp1.Add(obs);
            if (dtg == null)
            {
                dtg = new DataGrid();
                dtg.ItemsSource = tmp1;
                dtg.AutoGenerateColumns = false;
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Марка", Binding = new Binding("Marka") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Номер сажки", Binding = new Binding("NumberProk")
});
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Геом Листа Н", Binding = new Binding("H") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Геом Листа W", Binding = new Binding("W") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Геом Листа L", Binding = new Binding("L") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Сляб, Н", Binding = new Binding("HS") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Сляб, W", Binding = new Binding("WS") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Сляб, L", Binding = new Binding("LS") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Н подст", Binding = new Binding("H_podst") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Т нчп", Binding = new Binding("T_nchp") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Т кп", Binding = new Binding("T_kp") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Вес сляба", Binding = new Binding("Weight") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Печь", Binding = new Binding("Stove") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Номер тележки", Binding = new Binding("NumCar")
});
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Время посадки", Binding = new Binding("DaTime") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Время выдачи", Binding = new
Binding("TimeFromStove") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Стратегия прокатки", Binding = new
Binding("StrategyProk") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "УКО/УТУ", Binding = new Binding("UKOorUTU") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Время охлаждения в УТУ", Binding = new
Binding("TimeCoolingInUTU") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Место охлаждения", Binding = new
Binding("PlaceCooling") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Время хранения под ЗРУ", Binding = new
Binding("TimeStorageZRU") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Задание на правку", Binding = new
Binding("TaskPrav") });
                dtg.Columns.Add(new DataGridTextColumn() {
Header = "Количество проходов УПО", Binding = new
Binding("CountPass") });
            }
        }
    }
}

```

Рис. П2.4. Модуль RecMP.cs

П2.4. Продолжение

```

        dtg.Columns.Add(new DataGridTextBoxColumn() {
Header = "Тнач охл УПО", Binding = new
Binding("TnachCoolingUPO") });
        dtg.Columns.Add(new DataGridTextBoxColumn() {
Header = "Ткон охл УПО", Binding = new
Binding("TkonCoolingUPO") });
        dtg.Columns.Add(new DataGridTextBoxColumn() {
Header = "Скорость транспортировки", Binding = new
Binding("SpeedTrans") });
        dtg.Columns.Add(new DataGridTextBoxColumn() {
Header = "Расход воды", Binding = new Binding("RashodWater")
});
        dtg.Columns.Add(new DataGridTextBoxColumn() {
Header = "Тнач охл УКО", Binding = new
Binding("TnachCoolingUKO") });
        dtg.Columns.Add(new DataGridTextBoxColumn() {
Header = "Ткон охл УКО", Binding = new
Binding("TkonCoolingUKO") });

```

```

        dtg.Columns.Add(new DataGridTextBoxColumn() {
Header = "V охл УКО(м/с)", Binding = new
Binding("VCoollingUKOms") });
        dtg.Columns.Add(new DataGridTextBoxColumn() {
Header = "V охл УКО(град/с)", Binding = new
Binding("VCoollingUKOgrads") });
        dtg.Height = 70;
        dtg.Width = 500;
        Canvas.SetLeft(dtg, otX);
        Canvas.SetTop(dtg, otY + 30);
        Cnv.Children.Add(dtg);
    }
    else
    {
        Cnv.Children.Remove(dtg);
        dtg = null;
    } } }

```

Текст модуля MLine.cs представлен на рис. П2.5.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Shapes;

namespace OptimizatorStana
{
    public class MLine
    {
        public Line Ln {get; set;}
        public Line Ln1 { get; set; }
        public Line Ln2 { get; set; }
        public Slyab Slb { get; set; }
        private Canvas Cnv { get; set; }
        public MLine(Line ln, Line ln2, Line ln3, Slyab slb,
Canvas cnv) {
            Ln = ln;
            Ln1 = ln2;
            Ln2 = ln3;
            Slb = slb;
            Cnv = cnv;
            Ln.MouseLeftButtonDown +=
Ln_MouseLeftButtonDown;
            Ln1.MouseLeftButtonDown +=
Ln_MouseLeftButtonDown;
            Ln2.MouseLeftButtonDown +=
Ln_MouseLeftButtonDown;
            Cnv.Children.Add(Ln);
            Cnv.Children.Add(Ln1);
            Cnv.Children.Add(Ln2);
        }
        DataGrid dtg = null;
        private void Ln_MouseLeftButtonDown(object sender,
System.Windows.Input.MouseButtonEventArgs e)
        {
            List<Slyab> tmp1 = new List<Slyab>();
            tmp1.Add(Slb);
            if (dtg == null)
            {
                dtg = new DataGrid();
                dtg.ItemsSource = tmp1;
                dtg.AutoGenerateColumns = false;
                dtg.Columns.Add(new DataGridTextBoxColumn() {
Header = "Марка", Binding = new Binding("Marka")});
                dtg.Columns.Add(new DataGridTextBoxColumn() {
Header = "Номер прокатки", Binding = new
Binding("NumberProk") });
                dtg.Columns.Add(new DataGridTextBoxColumn() {
Header = "Геом Листа H", Binding = new Binding("H") });
            }
        }
    }
}

```

Рис. П2.5. Модуль MLine.cs

П2.5. Продолжение

```

        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Геом Листа W", Binding = new Binding("W") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Геом Листа L", Binding = new Binding("L") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Сляб, H", Binding = new Binding("HS") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Сляб, W", Binding = new Binding("WS") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Сляб, L", Binding = new Binding("LS") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Н подст", Binding = new Binding("H_podst") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Т нчп", Binding = new Binding("T_nchp") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Т кп", Binding = new Binding("T_kp") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Вес сляба", Binding = new Binding("Weight") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Печь", Binding = new Binding("Stove") });
        if (Slb.Stove == "КП") {
            dtg.Columns.Add(new DataGridViewTextBoxColumn()
{ Header = "Номер КП", Binding = new Binding("NumForKP") });
        }
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Штука в печи", Binding = new
Binding("NumberInStove") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Стратегия прокатки", Binding = new
Binding("StrategyProk") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "УКО|УТУ", Binding = new Binding("UKOorUTU") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Время охлаждения в УТУ", Binding = new
Binding("TimeCoolingInUTU") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Место охлаждения", Binding = new
Binding("PlaceCooling") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Время хранения под ЗРУ", Binding = new
Binding("TimeStorageZRU") });

```

```

        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Задание на правку", Binding = new
Binding("TaskPrav") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Количество проходов УПО", Binding = new
Binding("CountPass") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Тнач охл УПО", Binding = new
Binding("TnachCoolingUPO") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Ткон охл УПО", Binding = new
Binding("TkonCoolingUPO") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Скорость транспортировки", Binding = new
Binding("SpeedTrans") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Расход воды", Binding = new Binding("RashodWater")
});
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Тнач охл УКО", Binding = new
Binding("TnachCoolingUKO") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "Ткон охл УКО", Binding = new
Binding("TkonCoolingUKO") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "V охл УКО(м/с)", Binding = new
Binding("VCoolingUKOms") });
        dtg.Columns.Add(new DataGridViewTextBoxColumn() {
Header = "V охл УКО(град/с)", Binding = new
Binding("VCoolingUKOgrads") });
        dtg.Height = 70;
        dtg.Width = 500;
        Canvas.SetLeft(dtg, Ln.X1);
        Canvas.SetTop(dtg, Ln.Y1 + 10);
        Cnv.Children.Add(dtg);
    }
    else
    {
        Cnv.Children.Remove(dtg);
        dtg = null;
    }
}
}

```

Текст модуля MainWindow.xaml представлен на рис. П2.6.

```
<Window xClass="OptimizerStana.MainWindow"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

xmlns:d="http://schemas.microsoft.com/expression/blend/2008"

xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"

        xmlns:local="clr-namespace:OptimizerStana"
        mc:Ignorable="d"

        Title="Оптимизатор          стана"          Height="800"
Width="930"

        Loaded="Window_Loaded">

<TabControl>

    <TabItem Header="Оптимизация">

        <Grid Background="#FFD6D3D3">

            <Grid.RowDefinitions>

<RowDefinition Height="70"></RowDefinition>
<RowDefinition Height="70"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition Height="30"></RowDefinition>
<RowDefinition></RowDefinition>

</Grid.RowDefinitions>

<Grid.ColumnDefinitions>

<ColumnDefinition/>
<ColumnDefinition/>
<ColumnDefinition/>
<ColumnDefinition/>

</Grid.ColumnDefinitions>

<Button Grid.Column="0" Grid.Row="1"
Content="Выбрать файл" Width="150" Height="50"
Click="Button_Click_15"/>

<Button Grid.Column="1" Grid.Row="1"
Content="Алгоритм 1" Height="50" Click="Button_Click_11"
Margin="0,2,10,6,78,4,9,4"/>


```

```

<Button Grid.Column="1" Grid.Row="1"
Content="Алгоритм 2" Height="50" Click="Button_Click_12"
Grid.ColumnSpan="2" Margin="156.2,10.6,142.9,4"/>
<Button Grid.Column="2" Grid.Row="1"
Content="Алгоритм 3" Height="50" Click="Button_Click_13"
Margin="92.6,10.6,0,9.4"/>
<Button Grid.Column="3" Grid.Row="0"
Content="Выгрузить" MinWidth="25" MinHeight="28"
Grid.ColumnSpan="1" Click="Button_Click_1"
Margin="10,12,74.6,10.4"/>
<TextBlock Text="{Binding CountIDTB}"
Grid.Column="3" Grid.Row="0" VerticalAlignment="Top"
HorizontalAlignment="Right" Margin="0,0,5,0" FontSize="7"/>
<TextBlock Text="{Binding CountTB}"
Grid.Column="3" Grid.Row="0" VerticalAlignment="Top"
HorizontalAlignment="Right" Margin="0,10,5,0" FontSize="7"/>
<TextBlock Text="{Binding KoeffTB}"
Grid.Column="3" Grid.Row="0" VerticalAlignment="Top"
HorizontalAlignment="Right" Margin="0,20,5,0" FontSize="7"/>
<TextBlock Text="{Binding PKPTB}"
Grid.Column="3" Grid.Row="0" VerticalAlignment="Top"
HorizontalAlignment="Right" Margin="0,30,5,0" FontSize="7"/>
<TextBlock Text="{Binding PMPTB}"
Grid.Column="3" Grid.Row="0" VerticalAlignment="Top"
HorizontalAlignment="Right" Margin="0,40,5,0" FontSize="7"/>
<Label Grid.Column="0" Grid.Row="1"
Content="Количество итераций" Margin="0,70,54.4,30.2"
Grid.RowSpan="3"/>
<TextBox xName="iteration" Grid.Column="0"
Grid.Row="2" Height="25" Margin="181,2.6,123.8,2.8"
Text="{Binding IterationTB}" Grid.ColumnSpan="2"/>
<Label Grid.Column="0" Grid.Row="3"
Content="Допустимая разница ширин"
Margin="0,0,2,54.4,0.2"/>
<TextBox xName="CountFiles" Grid.Row="1"
Grid.Column="3" Width="80" Height="25"
HorizontalAlignment="Right" Text="40"/>
<Label Grid.Row="1" Grid.Column="3"
Content="Кол-во файлов" Width="100" Height="25"
HorizontalAlignment="Left" Margin="37.2,22.6,0,22.4"/>
<TextBox xName="PauseClearBlack"
Grid.Column="2" Grid.Row="2" Height="25"
Margin="214.2,2.6,91.6,2.8" Text="{Binding PauseMezhduTB}"
Grid.ColumnSpan="2" />
<Label Grid.Column="2" Grid.Row="2"
Content="Пауза между чистой и черновой"
Margin="6.4,2.6,0,2.8" Height="25" MaxWidth="208"
Width="203" HorizontalAlignment="Left"/>
<Label Grid.Column="2" Grid.Row="3"
Content="Пауза между слябами черновой"
Margin="5.4,3.2,21.8,2.2" Height="25" />

```

Рис. П2.6. Модуль MainWindow.xaml

П2.6. Продолжение

```

        <Label      Grid.Column="2"      Grid.Row=" 4"
Content="Пауза между слябами чистой"
Margin="5,4,2,8,21.8,0.6" Height="25"/>

        <Label      Grid.Column="2"      Grid.Row=" 5"
Content="Доп пауза между черновой и чистой"
Margin="5,4,2,4,21.8,3" Height="25" />

        <Label      Grid.Column="0"      Grid.Row=" 6"
Content="Порядок посадки в КП" Margin="0,2,54.8,3. 4"
Height="25" />

        <Label      Grid.Column="0"      Grid.Row=" 7"
Content="Время на перегон тележки" Margin="0,2,54.8,3. 4"
Height="25"/>

        <!--<Label      Grid.Column="2"      Grid.Row=" 7"
Content="Макс. количество в КП" Margin="5,2,54.8,3. 4"
Height="25" />-->

        <Label      Grid.Column="2"      Grid.Row=" 7"
Content="Время посадки 1-го сляба в МП"
Margin="4,8,1,6,39.8,3.8" Height="25" />

        <Label      Grid.Column="0"      Grid.Row=" 8"
Content="Время посадки 1-го сляба в КП"
Margin="0,1,2,54.6,4.2" Height="25" />

        <Label      Grid.Column="2"      Grid.Row=" 8"
Content="Пауза между группами" Margin="4,6,1,2,40,2.2" />

        <!--<Label      Grid.Column="2"      Grid.Row=" 10"
Content="Время на загрузку печи" Margin="4,6,1,2,40,2.2" />-->

        <Label      Grid.Column="0"      Grid.Row=" 9"
Content="Макс время выдачи, ч" Margin="0,1,2,54.6,4. 2"
Height="25" />

        <Label      Grid.Column="0"      Grid.Row=" 10"
Content="Время на перемещение МП" Margin="0,1,2,54.6,4. 2"
Height="25" />

        <Label      Grid.Column="0"      Grid.Row=" 11"
Content="Время на перемещение КП1" Margin="0,1,2,54.6,4. 2"
Height="25" />

        <Label      Grid.Column="0"      Grid.Row=" 12"
Content="Время на перемещение КП2" Margin="0,1,2,54.6,4. 2"
Height="25" />

        <Label      Grid.Column="0"      Grid.Row=" 13"
Content="Время на перемещение КП3" Margin="0,1,2,54.6,4. 2"
Height="25" />

        <Label      Grid.Column="2"      Grid.Row=" 9" Content="Время на
перемещение КП4" Margin="4,6,1,2,40,2.2" />

        <Label      Grid.Column="2"      Grid.Row=" 10" Content="Время на
перемещение КП5" Margin="4,6,1,2,40,2.2" />

        <Label      Grid.Column="2"      Grid.Row=" 6" Content="Допустимая
разница длины" Margin="4,6,1,2,40,2.2" />

        <Label      Grid.Column="2"      Grid.Row=" 11" Content="Время
на перемещение КП6" Margin="4,6,1,2,40,2.2" />

        <Label      Grid.Column="0"      Grid.Row=" 14"
Content="Отступ от начала печи" Margin="0,1,2,54.6,4. 2"
Height="25" />

```

```

        <Label      Grid.Column="0"      Grid.Row=" 15"
Content="Отступ от конца печи" Margin="0,1,2,54.6,4. 2"
Height="25" />

        <!--<Label      Grid.Column="2"      Grid.Row=" 14"
Content="Время ПП" Margin="4,6,1,2,40,2.2"/>-->

        <TextBox      xName="OtNach"      Grid.Column="0"
Grid.Row="15"      Height="25"      Margin="181,3,2,123.8,3. 2"
Text="{Binding OtstupEndTB}" Grid.ColumnSpan="2"/>

        <TextBox      xName="OtEnd"      Grid.Column="0"
Grid.Row="14"      Height="25"      Margin="181,3,2,123.8,3. 2"
Text="{Binding OtstupNachTB}" Grid.ColumnSpan="2"/>

        <TextBox      xName="WidthMax"
Grid.Column="0"      Grid.Row="3"      Height="24"
Margin="181,3,2,123.8,3.2"      VerticalAlignment="Center"
Text="{Binding ShirinaTB}" Grid.ColumnSpan="2"/>

        <TextBox      xName="PauseBlack"
Grid.Column="2"      Grid.Row="3"      Height="25"
Margin="214,2,2,6,91.6,2.8" Text="{Binding PauseDirtyTB}"
Grid.ColumnSpan="2"/>

        <TextBox      xName="PauseClear"
Grid.Column="2"      Grid.Row="4"      Height="25"
Margin="214,2,3,2,91.6,2.2" Text="{Binding PauseClearTB}"
Grid.ColumnSpan="2"/>

        <TextBox      xName="DopPause"      Grid.Column="2"
Grid.Row="5"      Height="25"      Margin="214,2,2,8,91.6,2. 6"
Text="{Binding DopPauseTB}" Grid.ColumnSpan="2"/>

        <TextBox      xName="TimeForMP"
Grid.Column="2"      Grid.Row="7"      Height="25"
Margin="214,2,2,8,91.6,2.6" Text="{Binding TimeForMPTB}"
Grid.ColumnSpan="2"/>

        <TextBox      xName="TimeForKP"
Grid.Column="0"      Grid.Row="8"      Height="25"
Margin="181,3,2,123.8,3.2" Text="{Binding TimeForKPTB}"
Grid.ColumnSpan="2"/>

        <TextBox      xName="MaxTimeFrom"
Grid.Column="0"      Grid.Row="9"      Height="25"
Margin="181,3,2,123.8,3.2" Text="{Binding MaxTimeFromTB}"
Grid.ColumnSpan="2"/>

        <TextBox      xName="TimeMP"      Grid.Column="0"
Grid.Row="10"      Height="25"      Margin="181,3,2,123.8,3. 2"
Text="{Binding TimeMPTB}" Grid.ColumnSpan="2"/>

        <TextBox      xName="TimeKP1"      Grid.Column="0"
Grid.Row="11"      Height="25"      Margin="181,3,2,123.8,3. 2"
Text="{Binding TimeKP1TB}" Grid.ColumnSpan="2"/>

        <TextBox      xName="TimeKP2"      Grid.Column="0"
Grid.Row="12"      Height="25"      Margin="181,3,2,123.8,3. 2"
Text="{Binding TimeKP2TB}" Grid.ColumnSpan="2"/>

        <TextBox      xName="TimeKP3"      Grid.Column="0"
Grid.Row="13"      Height="25"      Margin="181,3,2,123.8,3. 2"
Text="{Binding TimeKP3TB}" Grid.ColumnSpan="2"/>

```

П2.6. Продолжение

```

        <TextBox xName="PauseForGroups"
Grid.Column="2" Grid.Row="8" Height="25"
Margin="216,6,2,8,89,6,2,6" Text="{Binding
PauseForGroupsTB}" Grid.ColumnSpan="2"/>
        <!--<TextBox xName="TimeLoadStove"
Grid.Column="2" Grid.Row="10" Height="25"
Margin="214,2,2,8,91,6,2,6" Text="{Binding
TimeLoadStoveTB}" Grid.ColumnSpan="2"/>-->
        <TextBox xName="TimeKP4" Grid.Column="2"
Grid.Row="9" Height="25" Margin="214,2,2,8,91,6,2,6"
Text="{Binding TimeKP4TB}" Grid.ColumnSpan="2"/>
        <TextBox xName="TimeKP5" Grid.Column="2"
Grid.Row="10" Height="25" Margin="214,2,2,8,91,6,2,6"
Text="{Binding TimeKP5TB}" Grid.ColumnSpan="2"/>
        <TextBox xName="LengthMax"
Grid.Column="2" Grid.Row="6" Height="25"
Margin="214,2,2,8,91,6,2,6" Text="{Binding
LengthTB}" Grid.ColumnSpan="2"/>
        <TextBox xName="TimeKP6" Grid.Column="2"
Grid.Row="11" Height="25" Margin="214,2,2,8,91,6,2,6"
Text="{Binding TimeKP6TB}" Grid.ColumnSpan="2"/>
        <Label Grid.Column="0" Grid.Row="3"
Content="Количество поколений" Margin="0,30,2,54,4,0,6"
Grid.RowSpan="2"/>
        <TextBox xName="AllTime" Grid.Column="0"
Grid.Row="4" Height="25" Margin="181,2,6,123,8,2,8"
Grid.ColumnSpan="2" Text="{Binding KolpokolTB}"/>
        <Label Grid.Column="0" Grid.Row="5"
Content="Общее время с оптимизацией"
Margin="0,0,4,54,4,0"/>
        <Label Grid.Column="2" Grid.Row="12"
Content="Минимальное расстояние в печи"
Margin="4,6,1,2,40,2,2" Grid.ColumnSpan="2"/>
        <Label Grid.Column="2" Grid.Row="13"
Content="Время МП (ч)" Margin="4,6,1,2,40,2,2"
Grid.ColumnSpan="2"/>
        <Label Grid.Column="2" Grid.Row="14"
Content="Количество сортировок" Margin="4,6,1,2,40,2,2"
Grid.ColumnSpan="2"/>
        <TextBox xName="srtcount" Grid.Column="2"
Grid.Row="14" Height="25" Margin="214,2,2,8,91,6,2,6"
Text="{Binding SortCountTB}" Grid.ColumnSpan="2"/>
        <TextBox xName="MPTime" Grid.Column="2"
Grid.Row="13" Height="25" Margin="214,2,2,8,91,6,2,6"
Text="{Binding TimeHoursMPTB}" Grid.ColumnSpan="2"/>
        <ProgressBar xName="progBar"
Grid.Column="2" Grid.Row="15" Value="{Binding ProgBarCh}"
Maximum="{Binding KolpokolTB}" Grid.ColumnSpan="2"/>
        <TextBox xName="AllTimeOptimiz"
Grid.Column="0" Grid.Row="5" Height="25"

```

```

Margin="181,2,6,123,8,2,8" Text="{Binding TimeWithOptTB}"
Grid.ColumnSpan="2"/>
        <TextBox xName="Posad" Grid.Column="0"
Grid.Row="6" Height="25" Margin="181,2,6,123,8,2,8"
Text="{Binding OrderTB}" Grid.ColumnSpan="2"/>
        <!--<TextBox xName="MaxSlabs"
Grid.Column="2" Grid.Row="7" Height="25"
Margin="214,2,2,8,91,6,2,6" Text="{Binding
MaxSlabsTB}" Grid.ColumnSpan="2"/>-->
        <TextBox xName="TimeToCar"
Grid.Column="0" Grid.Row="7" Height="25"
Margin="181,2,6,123,8,2,8" Text="{Binding
TimeCarTB}" Grid.ColumnSpan="2"/>
        <TextBox xName="MinDist" Grid.Column="2"
Grid.Row="12" Height="25" Margin="214,2,2,8,91,6,2,6"
Text="{Binding MinDist}" Grid.ColumnSpan="2"/>
        <!--<TextBox xName="PPR" Grid.Column="2"
Grid.Row="14" Height="25" Margin="214,2,2,8,91,6,2,6"
Grid.ColumnSpan="2" Text="{Binding PPRTB}"/>-->
        <GroupBox Grid.ColumnSpan="3"
Header="Формирование прокатных партий"
HorizontalAlignment="Left" Height="72" Margin="-2,-4,0,0"
VerticalAlignment="Top" Width="523" BorderBrush="Black"
Foreground="Black">
        <Grid Margin="0,0,-1,8,-0,4">
        <Grid.ColumnDefinitions>
        <ColumnDefinition/>
        <ColumnDefinition/>
        <ColumnDefinition/>
        </Grid.ColumnDefinitions>
        <Button xName="Alg1" Grid.Column="0"
Grid.Row="0" Content="Алгоритм 1" MinWidth="25"
MinHeight="28" Width="150" Height="47" Grid.ColumnSpan="1"
Click="Button_Click" />
        <Button xName="Alg2" Grid.Column="1"
Grid.Row="0" Content="Алгоритм 2" MinWidth="25"
MinHeight="28" Width="150" Height="47" Grid.ColumnSpan="1"
Click="Button_Click_2"/>
        <Button xName="Alg3" Grid.Column="2"
Grid.Row="0" Content="Алгоритм 3" MinWidth="25"
MinHeight="28" Width="150" Height="47" Grid.ColumnSpan="1"
Click="Button_Click_4"/>
        </Grid>
        </GroupBox>
        <Button xName="Rasklad" IsEnabled="False"
Grid.Column="2" Grid.Row="0" Content="Раскладка по печам"
MinWidth="25" MinHeight="28" Height="47"
Grid.ColumnSpan="1" Click="Button_Click_8"
Margin="69,6,13,20,10,4"/>
        </Grid>
    </TabItem>
    <TabItem Header="Таблица">

```

П2.6. Продолжение

```

        <DataGrid      Grid.Column="0"      Grid.Row="9"
xName="dgdate"      AutoGenerateColumns="True"
Margin="10,9,8,9,6,10,4" Grid.ColumnSpan="3" />
    </TabItem>
    <TabItem Header="Группы">
        <ScrollViewer
HorizontalScrollBarVisibility="Visible"
VerticalScrollBarVisibility="Visible">
            <Grid Margin="0">
                <Grid.RowDefinitions>
                    <RowDefinition Height="30"/>
                </Grid.RowDefinitions>
                <Button      Height="25"      Width="100"
Grid.Row="0"      Content="Сохранить"      Click="Button_Click_9"
VerticalAlignment="Top"      HorizontalAlignment="Left"
Margin="10,3,0,0"/>
                <!--<Button      Height="25"      Width="100"
Grid.Row="0"      Content="Нарисовать"      Click="Button_Click_3"
VerticalAlignment="Top"      HorizontalAlignment="Left"
Margin="10,5,0,0"/>-->
                <!--<Button      Height="25"      Width="126"
Grid.Row="0"      Content="Нарисовать      (Simplex)"
Click="Button_Click_7"      VerticalAlignment="Top"
HorizontalAlignment="Left"      Margin="125,5,0,0" />-->
                <Grid Grid.Row="1" Background="White">
                    <Border      BorderBrush="Black"
BorderThickness="1">
                        <Canvas      xName="Canvas"
VerticalAlignment="Top"      HorizontalAlignment="Left"
Height="15000" Width="5000">
                            </Canvas>
                        </Border>
                    </Grid>
                </Grid>
            </ScrollViewer>
        </TabItem>
        <TabItem Header="Печи">
            <Grid>
                <Grid.RowDefinitions>
                    <RowDefinition Height="30"/>
                </Grid.RowDefinitions>
                <Button Height="25" Width="100" Grid.Row="0"
Content="Сохранить"      Click="Button_Click_10"
VerticalAlignment="Top"      HorizontalAlignment="Left"
Margin="10,3,0,0"/>
                <CheckBox      xName="Mtb"      Grid.Row="0"
Content="Масштабирование" Margin="124,7,654,6,5,4"/>
                <!--<Button      Height="25"      Width="100"
Grid.Row="0"      Content="Нарисовать"      Click="Button_Click_5"
Margin="5,5,5,0"      VerticalAlignment="Top"
HorizontalAlignment="Left" />

```

```

        <Button      Height="25"      Width="130"
Grid.Row="0"      Content="Нарисовать      (Simplex)"
Click="Button_Click_6"      Margin="130,5,5,0"
VerticalAlignment="Top" HorizontalAlignment="Left" />-->
        <Grid Grid.Row="1" Background="White">
            <Border      BorderBrush="Black"
BorderThickness="1">
                <ScrollViewer      xName="scrl1"
HorizontalScrollBarVisibility="Visible"
VerticalScrollBarVisibility="Visible">
                    <Canvas      xName="Canvass"
VerticalAlignment="Top"      HorizontalAlignment="Left"
Height="10000"      Width="10000"
MouseWheel="canvas1_MouseWheel">
                        </Canvas>
                    </ScrollViewer>
                </Border>
            </Grid>
        </TabItem>
        <TabItem Header="Раскладка">
            <ScrollViewer
HorizontalScrollBarVisibility="Visible"
VerticalScrollBarVisibility="Visible">
                <Grid>
                    <Grid.RowDefinitions>
                        <RowDefinition Height="30"/>
                    </Grid.RowDefinitions>
                    <Button      Height="25"      Width="100"
Grid.Row="0"      Content="Сохранить"      Click="Button_Click_14"
VerticalAlignment="Top"      HorizontalAlignment="Left"
Margin="10,3,0,0"/>
                    <Label      Margin="125,4,0,0"
FontSize="11">Маркировка: [Номер стопы], Марка, Номер
штуки, Размер (Т х Ш х Д), № гр, № сляба</Label>
                    <Grid Grid.Row="1" Background="White">
                        <Border      BorderBrush="Black"
BorderThickness="1">
                            <Canvas      xName="CanvasRasklad"
VerticalAlignment="Top"      HorizontalAlignment="Left"
Height="2000" Width="5000">
                                </Canvas>
                            </Border>
                        </Grid>
                    </Grid>
                </ScrollViewer>
            </TabItem>
        <TabItem Header="Подключение к БД">
            <Grid Background="#FFD6D3D3">
                <Grid.RowDefinitions>
                    <RowDefinition Height="30"/>
                </Grid.RowDefinitions>
                <RowDefinition Height="30"/>
                <RowDefinition Height="30"/>
                <RowDefinition Height="30"/>
            </Grid.RowDefinitions>
            <Grid.ColumnDefinitions>

```

П2.6. Продолжение

```

        <ColumnDefinition/>
        <ColumnDefinition/>
        <ColumnDefinition/>
        <ColumnDefinition/>
    </Grid.ColumnDefinitions>
    <Label      Grid.Column="0"      Grid.Row="0"
Content="Имя      пользователя"      Margin="5.4,2.4,21.8,3"
Height="25"/>
    <Label      Grid.Column="0"      Grid.Row="1"
Content="Порт" Margin="5.4,2.4,21.8,3" Height="25"/>
    <Label      Grid.Column="0"      Grid.Row="2"
Content="Хост" Margin="5.4,2.4,21.8,3" Height="25"/>
    <Label      Grid.Column="2"      Grid.Row="0"
Content="Пароль" Margin="5.4,2.4,21.8,3" Height="25"/>
    <Label      Grid.Column="2"      Grid.Row="1"
Content="ID Сервера" Margin="5.4,2.4,21.8,3" Height="25"/>
    <TextBox  xName="Name"      Grid.Column="1"
Grid.Row="0" Height="25" Text="{Binding NameTB}"/>
    <TextBox  xName="Password"  Grid.Column="3"
Grid.Row="0" Height="25" Text="{Binding PasswordTB}"/>
    <TextBox  xName="Port"      Grid.Column="1"
Grid.Row="1" Height="25" Text="{Binding PortTB}"/>
    <TextBox  xName="IDServ"    Grid.Column="3"
Grid.Row="1" Height="25" Text="{Binding IDSTB}"/>
    <TextBox  xName="Host"      Grid.Column="1"
Grid.Row="2" Height="25" Text="{Binding HostTB}"/>
    <Button   xName="BtnDB"      Grid.Column="0"
Grid.Row="3"      Width="125"      Click="BtnDB_Click"
Margin="0,5,0,0" Content="Подключение к БД"/>

```

```

        <Button   xName="ForFile"  Grid.Column="2"
Grid.Row="3"      Width="125"      Click="ForFile_Click"
Margin="0,5,0,0" Content="Выбрать файл"/>
        <Label   xName="InfoBD"   Grid.Column="1"
Grid.Row="3"      Height="25"      Margin="5.4,2.4,21.8,3"
Content="{Binding InfoBDTB}"/>
    </Grid>
    </TabItem>
    <TabItem Header="Для 1 печи">
        <Grid>
            <Grid.RowDefinitions>
                <RowDefinition Height="auto"/>
                <RowDefinition Height="auto"/>
            </Grid.RowDefinitions>
            <Grid.ColumnDefinitions>
                <ColumnDefinition/>
                <ColumnDefinition/>
            </Grid.ColumnDefinitions>
            <Button   Grid.Column="0"      Grid.Row="0"
Width="150"      Height="40"      Content="Выбрать файл"
Click="Button_Click_16"/>
            <Button   xName="ras"      Grid.Column="1"
Grid.Row="0"      Width="150"      Height="40"      Content="Разложить"
Click="Button_Click_17" IsEnabled="False"/>
        </Grid>
    </TabItem>
</TabControl>
</Window>

```

Спецификации

Настоящее приложение содержит спецификацию на разработанную программную документацию и компоненты программного обеспечения. Спецификация на разработанную программную документацию приведена в табл. ПЗ.1.

Таблица ПЗ.1

Спецификации на разработанную программную документацию

Обозначение	Наименование	Примечание
1	2	3
Документация		
Кустов_ВКР.docx	Расчетно-пояснительная записка выпускной квалификационной работы	
Компоненты		
App.xml	Файл конфигурации визуальных ресурсов приложения	
App.cs	Содержит класс App, являющийся стартовой точкой приложения	
MainWindow.xaml	Содержит структуру интерфейса оконной формы	
MainWindow.xaml.cs	Содержит структуру интерфейса оконной формы	
MainViewModel.cs	Модель представления главного окна	
MainModel.cs	Содержит класс MainModel в котором реализована алгоритма оптимизации	
StoveSim.cs	Содержит класс StoveSim, содержащий механизмы для формирования посадки слябов по печам	
IWriter.cs	Класс, позволяющий записывать данные в различные файлы	
ConnectToDB.cs	Класс, позволяющий подключиться к БД	
WriteToDB.cs	Класс, для работы с БД	
WriteToExcel.cs	Класс для записи данных в Excel файл	
SaveModel.cs	Класс для сохранения данных	
BaseModel.cs	Базовый класс модели	
Slyab.cs	Содержит класс модели сляба	

Продолжение табл. ПЗ.1

1	2	3
RollingGroup.cs	Содержит класс модели прокатной группы	
Canva.cs	Содержит класс для отрисовки элементов на канвасе	
Line.cs	Класс создания линии на канвасе	
RecMP.cs	Содержит класс создания прямоугольника на канвасе для МП	
RectangleInfo.cs	Содержит класс создания прямоугольника с информацией на канвасе для КП	

Руководство пользователя

1. Описание программного обеспечения

Программный продукт предназначен для планирования прокатки и нагрева слабов на стане 5000. Результаты работы программного обеспечения может записываться в базу данных или Excel файл.

Программное обеспечение предназначается для операторов, которые следят за работой стана 5000.

2. Минимальные системные требования

Для корректной работы системы необходимы персональные компьютеры, удовлетворяющие следующим минимальным требованиям:

- объем оперативной памяти 16 Гб;
- монитор с разрешением 1920x1080;
- операционная система Windows10 Домашняя;
- твердотельный накопитель объемом 1 Тб;
- процессор AMD Ryzen 5 2600 (4 ядер, 8 потоков, тактовая 3.4 ГГц);
- видеокарта Gigabyte GeForce GTX 1660Ti с поддержкой DirectX 12.

3. Запуск программы

Для запуска программы необходимо открыть папку с файлами программы и в ней найти файл «OptimizerStana.exe» (рис. П4.1). После чего открыть его.

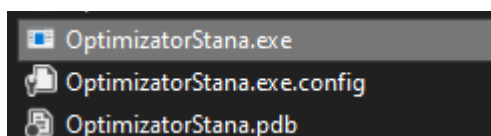


Рис. П4.1. Файл запуска ПО

После чего перед пользователем откроется главное окно программы (рис. П4.2).

Оптимизатор стана

Оптимизация | Таблица | Группы | Печи | Раскладка | Подключение к БД

Формирование прокатных партий

Старт

Выгрузить результат

Загрузить файл

Общее время с оптимизацией:	98585	Пауза между чистой и черновой	35
Порядок посадки в КП	1 2 3 4 5	Пауза между слябами черновой	30
Отступ от конца печи	100	Пауза между слябами чистой	40
Отступ от начала печи	100	Доп. пауза между черновой и чистой	15
Количество сортировок	3	Допустимая разница длины	10000
Время на перегон тележки	320	Время посадки 1-го сляба в МП	14.02.21 15:00
Время посадки 1-го сляба в КП		Пауза между группами	40
Макс. время выдачи, ч	6	Время на перемещение КП4	170
Время на перемещение МП	100	Время на перемещение КП5	160
Время на перемещение КП1	200	Время на перемещение КП6	150
Время на перемещение КП2	190	Минимальное расстояние в печи	700
Время на перемещение КП3	180	Время МП (ч)	8

Рис. П4.2. Главное окно программы

Сначала пользователь выбирает файл с данными, необходимыми для работы программы (список слябов), чтобы выбрать файл необходимо нажать кнопку «Выбрать файл». Далее пользователь заполняет необходимые поля согласно технологическим требованиям (например, вводит порядок КП), в некоторых текстовых полях стоят значения по умолчанию, которые редко изменяются. После того как пользователь загрузил файл и ввел необходимые параметры, необходимо запустить алгоритм, нажав кнопку «Старт». После чего шкала процесса начнет заполняться зеленым, когда шкала заполнится значит работа алгоритма закончена и можно переходить к просмотру результатов.

При нажатии на вкладку «Таблица» будут показаны результаты в табличной форме (рис. П4.3).

Оптимизатор стана

Оптимизация	Таблица	Группы	Печи	Раскладка	Подключение к БД	Для 1 печи
-------------	---------	--------	------	-----------	------------------	------------

№	Марка	Номер садки	Лист, Н	Лист, W	Лист, L	Сляб, Н	Сляб, W	Сляб, L	Н пс
1	S355K2	34768	15	2500	7500	313	1715	1828	50..7
2	S355J2	34763	15	2450	11980	313	1990	1662	50..7
3	09Г2С	34931	130	2600	5400	313	1995	3611	210..
4	S355J2	34901	16	2500	6000	313	1995	1523	50..8
5	S355J2	34763	15	2450	11980	313	1990	1663	50..7
6	S355J2	34535	16	2270	11970	313	1710	1878	50..7
7	S355J2	34916	16	2270	11970	313	1715	1873	50..7
8	S355J2	34772	16	2270	11970	313	1995	1598	50..7
9	S355J2	34772	16	2270	11970	313	1995	1598	50..7

Рис. П4.3. Вкладка «Таблица»

Чтобы посмотреть сформированные группы в виде рисунка необходимо открыть вкладку «Группы» (рис. П4.4).



Рис. П4.4. Вкладка «Группы»

Для того, чтобы посмотреть в графическом виде как был сформирован посад слябов в печи, необходимо открыть вкладку «Печи» (рис. П4.5).

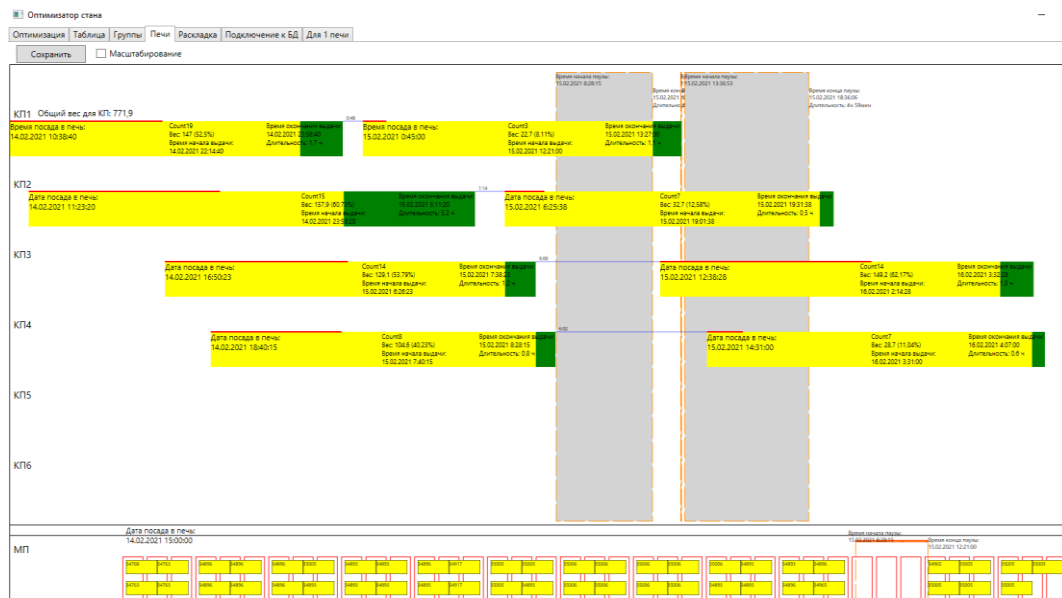


Рис. П4.5. Вкладка «Печи»

Также создана экспериментальная вкладка «Раскладка» в которой пользователь может раскладку слябов внутри камерных печей (рис. П4.6).



Рис. П4.6. Вкладка «Раскладка»

Во время работы у пользователя будут выскакивать информационные сообщения, например, сообщение о том, что партии сформированы (рис. П4.7) и раскладка слябов по печам завершена (рис. П4.8).

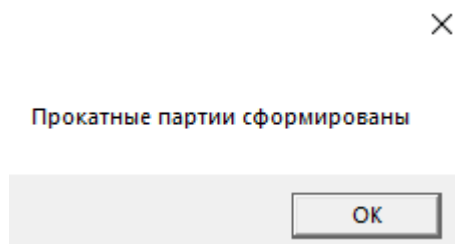


Рис. П4.7. Сообщение об успешном формировании партий

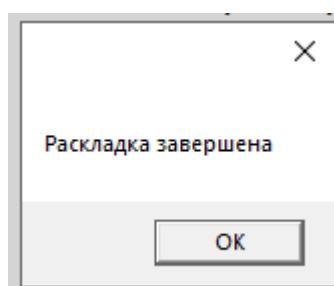


Рис. П4.8. Сообщение об успешной раскладке слябов